

Python scientific computing ecosystem

Contents

- Why Python?
- Before starting: Installing a working environment
- The workflow: interactive environments and text editors

Authors: *Fernando Perez, Emmanuelle Gouillart, Gaël Varoquaux, Valentin Haenel*

Why Python?

The scientist's needs

- Get data (simulation, experiment control),
- Manipulate and process data,
- Visualize results, quickly to understand, but also with high quality figures, for reports or publications.

Python's strengths

- **Batteries included** Rich collection of already existing **bricks** of classic numerical methods, plotting or data processing tools. We don't want to re-program the plotting of a curve, a Fourier transform or a fitting algorithm. Don't reinvent the wheel!
- **Easy to learn** Most scientists are not paid as programmers, neither have they been trained so. They need to be able to draw a curve, smooth a signal, do a Fourier transform in a few minutes.
- **Easy communication** To keep code alive within a lab or a company it should be as readable as a book by collaborators, students, or maybe customers. Python syntax is simple, avoiding

strange symbols or lengthy routine specifications that would divert the reader from mathematical or scientific understanding of the code.

- **Efficient code** Python numerical modules are computationally efficient. But needless to say that a very fast code becomes useless if too much time is spent writing it. Python aims for quick development times and quick execution times.
- **Universal** Python is a language used for many different problems. Learning Python avoids learning a new software for each new problem.

How does Python compare to other solutions?

Table 1 Compiled languages (C, C++, Fortran ...)

Pros	Very fast. For heavy computations, it's difficult to outperform these languages.
Cons	Painful usage: no interactivity during development, mandatory compilation steps, verbose syntax, manual memory management. These are difficult languages for non programmers.

Table 2 Matlab scripting language

Pros	• Very rich collection of libraries with numerous algorithms, for many different domains. Fast execution because these libraries are often written in a compiled language. • Pleasant development environment: comprehensive help, integrated editor, etc. • Commercial support is available.
Cons	• Base language is quite poor and can become restrictive for advanced users. • Not free and not everything is open sourced.

Table 3 Julia

Pros	• Fast code, yet interactive and simple to read and write. • Easily connects to Python or C.
Cons	• Ecosystem limited to numerical computing. • Still young.

Table 4 Other scripting languages: Scilab, Octave, R, IDL, etc.

Pros	• Open-source, free, or at least cheaper than Matlab. • Some features can be very advanced (statistics in R, etc.)
Cons	• Fewer available algorithms than in Matlab, and the language is not more advanced. • Some software are dedicated to one domain. Ex: Gnuplot to draw curves. These programs are very powerful, but they are restricted to a single type of usage, such as plotting.

Table 5 Python

Pros	• Very rich scientific computing libraries • Well thought out language, allowing to write very readable and well structured code: we “code what we think”. • Many libraries beyond scientific computing (web server, serial port access, etc.) • Free and open-source software, widely spread, with a vibrant community. • A variety of powerful environments to work in, such as IPython, Spyder, Jupyter notebooks, Pycharm, Visual Studio Code
Cons	• Not all the algorithms that can be found in more specialized software or toolboxes.

The scientific Python ecosystem

Unlike Matlab, or R, Python does not come with a pre-bundled set of modules for scientific computing. Below are the basic building blocks that can be combined to obtain a scientific computing environment:

Python, a generic and modern computing language

- The language: flow control, data types (`string`, `int`), data collections (lists, dictionaries), etc.
- Modules of the standard library: string processing, file management, simple network protocols.
- A large number of specialized modules or applications written in Python: web framework, etc. ... and scientific computing.
- Development tools (automatic testing, documentation generation)

See also

[Chapter on Python language](#)

Core numeric libraries

- **NumPy**: numerical computing with powerful **numerical arrays** objects, and routines to manipulate them. <https://numpy.org/>

See also

[Chapter on Numpy](#)

- **SciPy** : high-level numerical routines. Optimization, regression, interpolation, etc <https://scipy.org/>

See also

[Chapter on SciPy](#)

- **Matplotlib** : 2-D visualization, “publication-ready” plots <https://matplotlib.org/>

See also

[Chapter on Matplotlib](#)

Advanced interactive environments:

- **IPython**, an advanced **Python console** <https://ipython.org/>
- **Jupyter**, **notebooks** in the browser <https://jupyter.org/>

Domain-specific packages,

- **pandas**, **statsmodels**, **seaborn** for [statistics](#)
- **sympy** for [symbolic computing](#)
- **scikit-image** for [image processing](#)
- **scikit-learn** for [machine learning](#)

and many more packages not documented in the Scientific Python Lectures.

See also

- [Chapters on advanced topics](#)
- [Chapters on packages and applications](#)

► Show code cell source

Before starting: Installing a working environment

Python comes in many flavors, and there are many ways to install it. However, we recommend to install a scientific-computing distribution, that comes readily with optimized versions of scientific modules.

Under Linux

If you have a recent distribution, most of the tools are probably packaged, and it is recommended to use your package manager.

Other systems

There are several fully-featured scientific Python distributions:

- [Anaconda](#)
- [WinPython](#)

The workflow: interactive environments and text editors

Interactive work to test and understand algorithms: In this section, we describe a workflow combining interactive work and consolidation.

Python is a general-purpose language. As such, there is not one blessed environment to work in, and not only one way of using it. Although this makes it harder for beginners to find their way, it makes it possible for Python to be used for programs, in web servers, or embedded devices.

Interactive work

We recommend an interactive work with the [IPython](#) console, or its offspring, the [Jupyter notebook](#). They are handy to explore and understand algorithms.

Start `ipython`:

```
In [2]: print('Hello world')
Hello world
```

Under the notebook

To execute code, press “shift enter”

Getting help by using the `?` operator after an object:

```
In [3]: print?
Signature: print(*args, sep=' ', end='\n', file=None, flush=False)
Docstring:
Prints the values to a stream, or to sys.stdout by default.

sep
    string inserted between values, default a space.
end
    string appended after the last value, default a newline.
file
    a file-like object (stream); defaults to the current sys.stdout.
flush
    whether to forcibly flush the stream.
Type:      builtin_function_or_method
```

See also

- IPython user manual: <https://ipython.readthedocs.io/en/stable/>
- Jupyter Notebook QuickStart: <https://docs.jupyter.org/en/latest/start/index.html>

Elaboration of the work in an editor

As you move forward, it will be important to not only work interactively, but also to create and reuse Python files. For this, a powerful code editor will get you far. Here are several good easy-to-use editors:

- [Spyder](#): integrates an IPython console, a debugger, a profiler...
- [PyCharm](#): integrates an IPython console, notebooks, a debugger... (freely available, but commercial)
- [Visual Studio Code](#): integrates a Python console, notebooks, a debugger, ...

Some of these are shipped by the various scientific Python distributions, and you can find them in the menus.

As an exercise, create a file `my_file.py` in a code editor, and add the following lines:

```
s = 'Hello world'
print(s)
```

Now, you can run it in IPython console or a notebook and explore the resulting variables:

```
In [1]: %run my_file.py
Hello world

In [2]: %whos
Variable  Type      Data/Info
-----
s         str       Hello world
```



From a script to functions

While it is tempting to work only with scripts, that is a file full of instructions following each other, do plan to progressively evolve the script to a set of functions:

- A script is not reusable, functions are.
- Thinking in terms of functions helps breaking the problem in small blocks.

IPython and Jupyter Tips and Tricks

The user manuals contain a wealth of information. Here we give a quick introduction to four useful features: *history*, *tab completion*, *magic functions*, and *aliases*.

Command history

Like a UNIX shell, the IPython console supports command history. Type the *up* and *down* cursor keys to navigate previously typed commands:

```
In [3]: x = 10
```

```
In [4]: <UP>
```

```
In [4]: x = 10
```

Tab completion

Tab completion is a convenient way to explore the structure of any object you're dealing with. Simply type `object_name.<TAB>` to view the object's attributes. Besides Python objects and keywords, tab completion also works on file and directory names.*

```
In [5]: x = 10
```

```
In [6]: x.<TAB>
as_integer_ratio() conjugate()      imag          to_bytes()
bit_count()        denominator    numerator
bit_length()       from_bytes()   real
```

Magic functions

The console and the notebooks support so-called *magic* functions by prefixing a command with the `%` character. For example, the `run` and `whos` functions from the previous section are magic functions. Note that, the setting `automagic`, which is enabled by default, allows you to omit the preceding `%` sign. Thus, you can just type the magic function and it will work.

Other useful magic functions are:

`%cd` to change the current directory

```
In [1]: cd /tmp
/tmp
```

`%cpaste` allows you to paste code, especially code from websites which has been prefixed with the standard Python prompt (e.g. `>>>`) or with an ipython prompt, (e.g. `in [3]`):

```
In [2]: %cpaste
Pasting code; enter '--' alone on the line to stop or use Ctrl-D.
:>>> for i in range(3):
...     print(i)
:--
0
1
2
```

`%timeit` allows you to time the execution of short snippets using the `timeit` module from the standard library:

```
In [3]: %timeit x = 10
10000000 loops, best of 3: 39 ns per loop
```

See also

[Chapter on optimizing code](#)

`%debug` allows you to enter post-mortem debugging. That is to say, if the code you try to execute, raises an exception, using `%debug` will enter the debugger at the point where the exception was thrown. For example, consider the following code.

```
def func(a, b):
    c = a * 3
    d = b * 20
    return c / d

func(2, 3)
```

0.1

All good, but now you try:

```
func(3, 0)
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 func(3, 0)

Cell In[2], line 4, in func(a, b)
      2 c = a * 3
      3 d = b * 20
----> 4 return c / d

ZeroDivisionError: division by zero
```

You run the code and see the error, but perhaps you want to go in and have a look at what the values of `c` and `d` are at the time of the error.

You can next type the `%debug` magic to enter the debugger, where you can print out values inside the function, before exiting the debugger with `q` followed by Return.

```
In [4]: %debug
>
/var/folders/hd/rfxyn9gx4bl39bvzrgn3rtr0000gn/T/ipykernel_62633/2015602957.py(
2)func()
      1 def func(a, b):
----> 2     return a / b
      3
      4 func(10, 0)

ipdb>
```

(where `ipdb` is the debugger prompt).

See also

[Chapter on debugging](#)

Aliases

IPython and Jupyter ship with various *aliases* which emulate common UNIX command line tools such as `ls` to list files, `cp` to copy files and `rm` to remove files (a full list of aliases is shown when typing `alias`).



Getting help

- The built-in cheat-sheet is accessible via the `%quickref` magic function.
- A list of all available magic functions is shown when typing `%magic`.

First steps

Start the **IPython** shell (an enhanced interactive Python shell):

- by typing “ipython” from a Linux/Mac terminal, or from the Windows cmd shell,
- **or** by starting the program from a menu, e.g. the [Anaconda Navigator](#), the [Python\(x,y\)](#) menu if you have installed one of these scientific-Python suites.

Note



If you don't have IPython installed on your computer, other Python shells are available, such as the plain Python shell started by typing “python” in a terminal, or the Idle interpreter. However, we advise to use the IPython shell because of its enhanced features, especially for interactive scientific computing.

Once you have started the interpreter, type

```
print("Hello, world!")
```

```
Hello, world!
```

Note



The message “Hello, world!” is then displayed. You just executed your first Python instruction, congratulations!

To get yourself started, type the following stack of instructions

```
a = 3
b = 2*a
type(b)
```

```
int
```



```
print(b)
```

```
6
```

```
a*b
```

```
18
```

```
b = 'hello'  
type(b)
```

```
str
```

```
b + b
```

```
'hellohello'
```

```
2*b
```

```
'hellohello'
```

Note



Two variables `a` and `b` have been defined above. Note that one does not declare the type of a variable before assigning its value. In C, conversely, one should write:

```
int a = 3;
```

In addition, the type of a variable may change, in the sense that at one point in time it can be equal to a value of a certain type, and a second point in time, it can be equal to a value of a different type. `b` was first equal to an integer, but it became equal to a string when it was assigned the value `'hello'`. Operations on integers (`b=2*a`) are coded natively in Python, and so are some operations on strings such as additions and multiplications, which amount respectively to concatenation and repetition.

Basic types

Contents

- Numerical types
- Containers
- Assignment operator

Numerical types

Note



Python supports the following numerical, scalar types:

Floats:

```
c = 2.1  
type(c)
```

```
float
```

Complex:

```
a = 1.5 + 0.5j  
a.real
```

```
1.5
```

```
a.imag
```

```
0.5
```

```
type(1. + 0j)
```

complex

Booleans:

```
3 > 4
```

False

```
test = (3 > 4)  
test
```

False

```
type(test)
```

bool

Note



A Python shell can therefore replace your pocket calculator, with the basic arithmetic operations , , , ,  (modulo) natively implemented

```
7 * 3.
```

21.0

```
2**10
```

1024

```
8 % 3
```

2

Type conversion (casting):

```
float(1)
```

1.0

Containers

Note



Python provides many efficient types of containers, in which collections of objects can be stored.

Lists

Note



A list is an ordered collection of objects, that may have different types. For example:

```
colors = ['red', 'blue', 'green', 'black', 'white']  
type(colors)
```

list

Indexing: accessing individual objects contained in the list:

```
colors[2]
```

'green'

Counting from the end with negative indices:

```
colors[-1]
```

```
'white'
```

```
colors[-2]
```

```
'black'
```

⚠ Warning

Indexing starts at 0 (as in C), not at 1 (as in Fortran or Matlab)!

Slicing: obtaining sublists of regularly-spaced elements:

```
colors
```

```
['red', 'blue', 'green', 'black', 'white']
```

```
colors[2:4]
```

```
['green', 'black']
```

⚠ Warning

Note that `colors[start:stop]` contains the elements with indices `i` such as `start <= i < stop` (`i` ranging from `start` to `stop-1`). Therefore, `colors[start:stop]` has `(stop - start)` elements.

Slicing syntax: `colors[start:stop:stride]`

i Note

All slicing parameters are optional:

```
colors
```

```
['red', 'blue', 'green', 'black', 'white']
```

```
colors[3:]
```

```
['black', 'white']
```

```
colors[:3]
```

```
['red', 'blue', 'green']
```

```
colors[::2]
```

```
['red', 'green', 'white']
```

Lists are *mutable* objects and can be modified:

```
colors[0] = 'yellow'  
colors
```

```
['yellow', 'blue', 'green', 'black', 'white']
```

```
colors[2:4] = ['gray', 'purple']  
colors
```

```
['yellow', 'blue', 'gray', 'purple', 'white']
```

Note

The elements of a list may have different types:

```
colors = [3, -200, 'hello']  
colors
```

```
[3, -200, 'hello']
```

```
colors[1], colors[2]
```

```
(-200, 'hello')
```

Note

For collections of numerical data that all have the same type, it is often **more efficient** to use the `array` type provided by the `numpy` module. A NumPy array is a chunk of memory containing fixed-sized items. With NumPy arrays, operations on elements can be faster because elements are regularly spaced in memory and more operations are performed through specialized C functions instead of Python loops.

Note

Python offers a large panel of functions to modify lists, or query them. Here are a few examples; for more details, see <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

Add and remove elements:

```
colors = ['red', 'blue', 'green', 'black', 'white']
colors.append('pink')
colors
```

```
['red', 'blue', 'green', 'black', 'white', 'pink']
```

```
colors.pop() # removes and returns the last item
```

```
'pink'
```

```
colors
```

```
['red', 'blue', 'green', 'black', 'white']
```

```
colors.extend(['pink', 'purple']) # extend colors, in-place
colors
```

```
['red', 'blue', 'green', 'black', 'white', 'pink', 'purple']
```

```
colors = colors[:-2]
colors
```

```
['red', 'blue', 'green', 'black', 'white']
```

Reverse:

```
rcolors = colors[::-1]
rcolors
```

```
['white', 'black', 'green', 'blue', 'red']
```

```
rcolors2 = list(colors) # new object that is a copy of colors in a different
memory area
rcolors2
```

```
['red', 'blue', 'green', 'black', 'white']
```

```
rcolors2.reverse() # in-place; reversing rcolors2 does not affect colors
rcolors2
```

```
['white', 'black', 'green', 'blue', 'red']
```

Concatenate and repeat lists:

```
rcolors + colors
```



```
['white',  
 'black',  
 'green',  
 'blue',  
 'red',  
 'red',  
 'blue',  
 'green',  
 'black',  
 'white']
```

```
rcolors * 2
```

```
['white',  
 'black',  
 'green',  
 'blue',  
 'red',  
 'white',  
 'black',  
 'green',  
 'blue',  
 'red']
```

Sort:

```
sorted(rcolors) # new object
```

```
['black', 'blue', 'green', 'red', 'white']
```

```
rcolors
```

```
['white', 'black', 'green', 'blue', 'red']
```

```
rcolors.sort() # in-place  
rcolors
```

```
['black', 'blue', 'green', 'red', 'white']
```

Methods and Object-Oriented Programming

The notation `rcolors.method()` (e.g. `rcolors.append(3)` and `colors.pop()`) is our first example of object-oriented programming (OOP). Being a `list`, the object `rcolors` owns the *method* `function` that is called using the notation `..`. No further knowledge of OOP than understanding the notation `.` is necessary for going through this tutorial.

Discovering methods:

Reminder: in Ipython: tab-completion (press tab)

```
rcolors.<TAB>
              append()  count()  insert()  reverse()
              clear()   extend()  pop()     sort()
              copy()    index()   remove()
```

Strings

Different string syntaxes (simple, double or triple quotes):

```
s = 'Hello, how are you?'
s = "Hi, what's up"
s = '''Hello,
    how are you'''          # tripling the quotes allows the
                             # string to span more than one line
s = """Hi,
what's up?"""
```

However, if you try to run this code:

```
'Hi, what's up?'
```

— you will get a syntax error. (Try it.) (Why?)

This syntax error can be avoided by enclosing the string in double quotes instead of single quotes. Alternatively, one can prepend a backslash to the second single quote. Other uses of the backslash are, e.g., the newline character `\n` and the tab character `\t`.

Note



Strings are collections like lists. Hence they can be indexed and sliced, using the same syntax and rules.

Indexing:

```
a = "hello"  
a[0]
```

```
'h'
```

```
a[1]
```

```
'e'
```

```
a[-1]
```

```
'o'
```

Note



(Remember that negative indices correspond to counting from the right end.)

Slicing:

```
a = "hello, world!"  
a[3:6] # 3rd to 6th (excluded) elements: elements 3, 4, 5
```

```
'lo, '
```

```
a[2:10:2] # Syntax: a[start:stop:step]
```

```
'lo o'
```

```
a[::3] # every three characters, from beginning to end
```

```
'hl r!'
```

Note

Accents and special characters can also be handled as in Python 3 strings consist of Unicode characters.

A string is an **immutable object** and it is not possible to modify its contents. One may however create new strings from the original one.

```
a = "hello, world!"  
a[2] = 'z'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[46], line 2  
      1 a = "hello, world!"  
----> 2 a[2] = 'z'  
  
TypeError: 'str' object does not support item assignment
```

```
a.replace('l', 'z', 1)
```

```
'hezlo, world!'
```

```
a.replace('l', 'z')
```

```
'hezzo, worzd!'
```

Note

Strings have many useful methods, such as `a.replace` as seen above. Remember the `a.` object-oriented notation and use tab completion or `help(str)` to search for new methods.

See also

Python offers advanced possibilities for manipulating strings, looking for patterns or formatting. The interested reader is referred to

<https://docs.python.org/3/library/stdtypes.html#string-methods> and

<https://docs.python.org/3/library/string.html#format-string-syntax>

String formatting:

```
'An integer: %i; a float: %f; another string: %s' % (1, 0.1, 'string') # with  
more values use tuple after %
```

```
'An integer: 1; a float: 0.100000; another string: string'
```

```
i = 102  
filename = 'processing_of_dataset_%d.txt' % i    # no need for tuples with just  
one value after %  
filename
```

```
'processing_of_dataset_102.txt'
```

Dictionaries

Note

A dictionary is basically an efficient table that **maps keys to values**.

```
tel = {'emmanuelle': 5752, 'sebastian': 5578}  
tel['francis'] = 5915  
tel
```

```
{'emmanuelle': 5752, 'sebastian': 5578, 'francis': 5915}
```

```
tel['sebastian']
```

```
5578
```

```
tel.keys()
```

```
dict_keys(['emmanuelle', 'sebastian', 'francis'])
```

```
tel.values()
```

```
dict_values([5752, 5578, 5915])
```

```
'francis' in tel
```

```
True
```

Note



It can be used to conveniently store and retrieve values associated with a name (a string for a date, a name, etc.). See <https://docs.python.org/3/tutorial/datastructures.html#dictionaries> for more information.

A dictionary can have keys (resp. values) with different types:

```
d = {'a':1, 'b':2, 3:'hello'}  
d
```

```
{'a': 1, 'b': 2, 3: 'hello'}
```

More container types

Tuples

Tuples are basically immutable lists. The elements of a tuple are written between parentheses, or just separated by commas:

```
t = 12345, 54321, 'hello!'
t[0]
```

```
12345
```

```
t
u = (0, 2)
```

Sets: unordered, unique items:

```
s = set(('a', 'b', 'c', 'a'))
s
```

```
{'a', 'b', 'c'}
```

```
s.difference(('a', 'b'))
```

```
{'c'}
```

Assignment operator

Note



[Python library reference](#) says:

Assignment statements are used to (re)bind names to values and to modify attributes or items of mutable objects.

In short, it works as follows (simple assignment):

1. an expression on the right hand side is evaluated, the corresponding object is created/obtained
2. a **name** on the left hand side is assigned, or bound, to the r.h.s. object

Things to note:

- A single object can have several names bound to it:

```
a = [1, 2, 3]
b = a
a
```

```
[1, 2, 3]
```

```
b
```

```
[1, 2, 3]
```

```
a is b
```

```
True
```

```
b[1] = 'hi!'
a
```

```
[1, 'hi!', 3]
```

- to change a list *in place*, use indexing/slices:

```
a = [1, 2, 3]
a
```

```
[1, 2, 3]
```

```
a = ['a', 'b', 'c'] # Creates another object.
a
```

```
['a', 'b', 'c']
```

```
id(a)
```

```
140076740845632
```



```
a[:] = [1, 2, 3] # Modifies object in place.  
a
```

```
[1, 2, 3]
```

```
id(a)
```

```
140076740845632
```

- the key concept here is **mutable vs. immutable**
 - mutable objects can be changed in place
 - immutable objects cannot be modified once created

See also

A very good and detailed explanation of the above issues can be found in David M. Beazley's article [Types and Objects in Python](#).

Control Flow

Contents

- if/elif/else
- for/range
- while/break/continue
- Conditional Expressions
- Advanced iteration
- List Comprehensions

Controls the order in which the code is executed.

if/elif/else

```
if 2**2 == 4:  
    print("Obvious!")
```

Obvious!

Blocks are delimited by indentation

Note

Type the following lines in your Python interpreter, and be careful to **respect the indentation depth**. The Jupyter / IPython shell automatically increases the indentation depth after a colon `:` sign; to decrease the indentation depth, go four spaces to the left with the Backspace key. Press the Enter key twice to leave the logical block.

```
a = 10
```

```
if a == 1:
    print(1)
elif a == 2:
    print(2)
else:
    print("A lot")
```

A lot

Indentation is compulsory in scripts as well. As an exercise, re-type the previous lines with the same indentation in a script `condition.py`, and execute the script with `run condition.py` in IPython.

for/range

Iterating with an index:

```
for i in range(4):
    print(i)
```

0
1
2
3

But most often, it is more readable to iterate over values:

```
for word in ('cool', 'powerful', 'readable'):
    print('Python is %s' % word)
```

Python is cool
Python is powerful
Python is readable

while/break/continue

Typical C-style while loop (Mandelbrot problem):

```
z = 1 + 1j
while abs(z) < 100:
    z = z**2 + 1
z
```

(-134+352j)

More advanced features

break out of enclosing for/while loop:

```
z = 1 + 1j

while abs(z) < 100:
    if z.imag == 0:
        break
    z = z**2 + 1
```

continue the next iteration of a loop.:

```
a = [1, 0, 2, 4]
for element in a:
    if element == 0:
        continue
    print(1. / element)
```

1.0
0.5
0.25

Conditional Expressions

if <OBJECT>:

Evaluates to **False** for:

- any number equal to zero (0, 0.0, 0+0j)
- an empty container (list, tuple, set, dictionary, ...)
- **False**, **None**

Evaluates to `True` for:

- everything else

Examples:

```
a = 10
if a:
    print("Evaluated to `True`")
else:
    print('Evaluated to `False`')
```

Evaluated to `True`

```
a = []
if a:
    print("Evaluated to `True`")
else:
    print('Evaluated to `False`')
```

Evaluated to `False`

`a == b:`

Tests equality, with logics::

```
1 == 1.
```

True

`a is b`

Tests identity: both sides **are the same object**:

```
a = 1
b = 1.
a == b
```

True

```
a is b
```

False

```
a = 'A string'
b = a
a is b
```

True

a in b

For any collection **b**: **b** contains **a** :

```
b = [1, 2, 3]
2 in b
```

True

```
5 in b
```

False

If **b** is a dictionary, this tests that **a** is a key of **b**.

```
b = {'first': 0, 'second': 1}
# Tests for key.
'first' in b
```

True

```
# Does not test for value.
0 in b
```

False

Advanced iteration

Iterate over any sequence:

You can iterate over any sequence (string, list, keys in a dictionary, lines in a file, ...):

```
vowels = 'aeiouy'
```

```
for i in 'powerful':  
    if i in vowels:  
        print(i)
```

```
o  
e  
u
```

```
message = "Hello how are you?"  
message.split() # returns a list
```

```
['Hello', 'how', 'are', 'you?']
```

```
for word in message.split():  
    print(word)
```

```
Hello  
how  
are  
you?
```

Note



Few languages (in particular, languages for scientific computing) allow to loop over anything but integers/indices. With Python it is possible to loop exactly over the objects of interest without bothering with indices you often don't care about. This feature can often be used to make code more readable.

⚠ Warning

It is not safe to modify the sequence you are iterating over.

Keeping track of enumeration number

Common task is to iterate over a sequence while keeping track of the item number.

We could use while loop with a counter as above. Or a for loop:

```
words = ('cool', 'powerful', 'readable')
for i in range(0, len(words)):
    print((i, words[i]))
```

```
(0, 'cool')
(1, 'powerful')
(2, 'readable')
```

But, Python provides a built-in function - `enumerate` - for this:

```
for index, item in enumerate(words):
    print((index, item))
```

```
(0, 'cool')
(1, 'powerful')
(2, 'readable')
```

Looping over a dictionary

Use `items`:

```
d = {'a': 1, 'b': 1.2, 'c': 1j}
```

```
for key, val in d.items():
    print('Key: %s has value: %s' % (key, val))
```



```
Key: a has value: 1  
Key: b has value: 1.2  
Key: c has value: 1j
```

List Comprehensions

Instead of creating a list by means of a loop, one can make use of a list comprehension with a rather self-explaining syntax.

```
[i**2 for i in range(4)]
```

```
[0, 1, 4, 9]
```

Exercise 1



Compute the decimals of Pi using the Wallis formula:

$$\pi = 2 \prod_{i=1}^{\infty} \frac{4i^2}{4i^2 - 1}$$

Solution to [Exercise 1](#)



```
from functools import reduce

pi = 3.14159265358979312

my_pi = 1.0

for i in range(1, 100000):
    my_pi *= 4 * i**2 / (4 * i**2 - 1.0)

my_pi *= 2

print(pi)
print(my_pi)
print(abs(pi - my_pi))
```

```
3.141592653589793
3.141584799578707
7.854011085939305e-06
```

```
num = 1
den = 1
for i in range(1, 100000):
    tmp = 4 * i * i
    num *= tmp
    den *= tmp - 1

better_pi = 2 * (num / den)

print(pi)
print(better_pi)
print(abs(pi - better_pi))
print(abs(my_pi - better_pi))
```

```
3.141592653589793
3.1415847995787067
7.854011086383395e-06
4.440892098500626e-16
```

Solution in a single line using more advanced constructs (reduce, lambda, list comprehensions):

```
print(
    2
    * reduce(
        lambda x, y: x * y,
        [float(4 * (i**2)) / ((4 * (i**2)) - 1) for i in range(1,
100000)],
    )
)
```

3.141584799578707

Defining functions

Contents

- Function definition
- Return statement
- Parameters
- Passing by value
- Global variables
- Variable number of parameters
- Docstrings
- Functions are objects
- Methods
- Exercises

Function definition

```
def test():  
    print('in test function')
```

```
test()
```

```
in test function
```

Warning

Function blocks must be indented in the same way as other control-flow blocks.

Return statement

Functions *always* return values:

```
def disk_area(radius):  
    return 3.14 * radius * radius
```

```
disk_area(1.5)
```

```
7.0649999999999995
```

But - if you do not specify an explicit return value, functions return the special Python value `None`.

```
def another_func(a):  
    # Do nothing.  
    # Notice there is no "return" statement.  
    pass
```

```
result = another_func(10)  
# Check whether result returned is None value.  
result is None
```

```
True
```

Note

Note the syntax to define a function:

- the `def` keyword;
- is followed by the function's **name**, then
- the arguments of the function are given between parentheses followed by a colon.
- the function body;
- and `return object` for optionally returning values.

Parameters

Mandatory parameters (positional arguments)

```
def double_it(x):  
    return x * 2
```

```
double_it(3)
```

6

```
double_it()
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[9], line 1  
----> 1 double_it()  
  
TypeError: double_it() missing 1 required positional argument: 'x'
```

Optional parameters (keyword or named arguments)

```
def double_it(x=2):  
    return x * 2
```

```
double_it()
```

4

```
double_it(3)
```

6

Keyword arguments allow you to specify *default values*.

Warning: default values are evaluated when the function is defined, not when it is called. This can be problematic when using mutable types (e.g. dictionary or list) and modifying them in the function body, since the modifications will be persistent across invocations of the function.

Using an immutable type in a keyword argument:

```
bigx = 10  
def double_it(x=bigx):  
    return x * 2
```

```
bigx = 1e9 # Now really big
double_it()
```

20

Using an mutable type in a keyword argument (and modifying it inside the function body):

```
def add_to_dict(args={'a': 1, 'b': 2}):
    for i in args.keys():
        args[i] += 1
    print(args)
```

add_to_dict

```
<function __main__.add_to_dict(args={'a': 1, 'b': 2})>
```

add_to_dict()

```
{'a': 2, 'b': 3}
```

add_to_dict()

```
{'a': 3, 'b': 4}
```

add_to_dict()

```
{'a': 4, 'b': 5}
```

More involved example implementing python's slicing:

```
def slicer(seq, start=None, stop=None, step=None):
    """Implement basic python slicing."""
    return seq[start:stop:step]
```

```
rhyme = 'one fish, two fish, red fish, blue fish'.split()
rhyme
```

```
['one', 'fish,', 'two', 'fish,', 'red', 'fish,', 'blue', 'fish']
```

```
 slicer(rhyme)
```

```
 ['one', 'fish,', 'two', 'fish,', 'red', 'fish,', 'blue', 'fish']
```

```
 slicer(rhyme, step=2)
```

```
 ['one', 'two', 'red', 'blue']
```

```
 slicer(rhyme, 1, step=2)
```

```
 ['fish,', 'fish,', 'fish,', 'fish']
```

```
 slicer(rhyme, start=1, stop=4, step=2)
```

```
 ['fish,', 'fish,']
```

The order of the keyword arguments does not matter:

```
 slicer(rhyme, step=2, start=1, stop=4)
```

```
 ['fish,', 'fish,']
```

— but it is good practice to use the same ordering as the function's definition.

Keyword arguments are a very convenient feature for defining functions with a variable number of arguments, especially when default values are to be used in most calls to the function.

Passing by value

Note



Can you modify the value of a variable inside a function? Most languages (C, Java, ...) distinguish “passing by value” and “passing by reference”. In Python, such a distinction is somewhat artificial, and it is a bit subtle whether your variables are going to be modified or not. Fortunately, there exist clear rules.

Parameters to functions are references to objects, which are passed by value. When you pass a variable to a function, python passes the reference to the object to which the variable refers (the **value**). Not the variable itself.

If the **value** passed in a function is immutable, the function does not modify the caller’s variable. If the **value** is mutable, the function may modify the caller’s variable in-place:

```
def try_to_modify(x, y, z):  
    x = 23  
    y.append(42)  
    z = [99] # new reference  
    print(x)  
    print(y)  
    print(z)
```

```
a = 77      # immutable variable  
b = [99]    # mutable variable  
c = [28]  
try_to_modify(a, b, c)
```

```
23  
[99, 42]  
[99]
```

```
print(a)
```

```
77
```

```
print(b)
```

```
[99, 42]
```

```
print(c)
```

```
[28]
```

Functions have a local variable table called a *local namespace*.

The variable `x` only exists within the function `try_to_modify`.

Global variables

Variables declared outside the function can be referenced within the function:

```
x = 5
def addx(y):
    return x + y
```

```
addx(10)
```

```
15
```

But these “global” variables cannot be modified within the function, unless declared **global** in the function.

This doesn't work:

```
def setx(y):
    x = y
    print('x is %d' % x)
```

```
setx(10)
```

```
x is 10
```

```
x
```

```
5
```

This works:

```
def setx(y):  
    global x  
    x = y  
    print('x is %d' % x)
```

```
setx(10)
```

```
x is 10
```

```
x
```

```
10
```

Variable number of parameters

Special forms of parameters:

- `*args`: any number of positional arguments packed into a tuple
- `**kwargs`: any number of keyword arguments packed into a dictionary

```
def variable_args(*args, **kwargs):  
    print('args is', args)  
    print('kwargs is', kwargs)
```

```
variable_args('one', 'two', x=1, y=2, z=3)
```

```
args is ('one', 'two')  
kwargs is {'x': 1, 'y': 2, 'z': 3}
```

Docstrings

Documentation about what the function does and its parameters. General convention:

```
def funcname(params):  
    """Concise one-line sentence describing the function.  
  
    Extended summary which can contain multiple paragraphs.  
    """  
    # function body  
    pass
```

```
# Also assessible in Jupyter / IPython with "funcname?"  
help(funcname)
```

Help on function funcname in module __main__:

```
funcname(params)  
    Concise one-line sentence describing the function.  
  
    Extended summary which can contain multiple paragraphs.
```

Note

Docstring guidelines

For the sake of standardization, the [Docstring Conventions](#) webpage documents the semantics and conventions associated with Python docstrings.

Also, the NumPy and SciPy modules have defined a precise standard for documenting scientific functions, that you may want to follow for your own functions, with a

[Parameters](#) section, an [Examples](#) section, etc. See

<https://numpydoc.readthedocs.io/en/latest/format.html#docstring-standard>

Functions are objects

Functions are first-class objects, which means they can be:

- assigned to a variable
- an item in a list (or any collection)
- passed as an argument to another function.

```
va = variable_args  
va('three', x=1, y=2)
```

```
args is ('three',)
kwargs is {'x': 1, 'y': 2}
```

Methods

Methods are functions attached to objects. You've seen these in our examples on *lists*, *dictionaries*, *strings*, etc...

Exercises

Exercise 2



Write a function that displays the first terms of the Fibonacci sequence, defined by:

$$\begin{aligned}U_0 &= 0 \\U_1 &= 1 \\U_{n+2} &= U_{n+1} + U_n\end{aligned}$$

Solution to [Exercise 2](#)



```
def fib(n):  
    """Display the n first terms of Fibonacci sequence"""  
    a, b = 0, 1  
    i = 0  
    while i < n:  
        print(b)  
        a, b = b, a+b  
        i +=1
```

```
fib(10)
```

```
1  
1  
2  
3  
5  
8  
13  
21  
34  
55
```

Exercise 3

Implement the [Quicksort algorithm, as defined by Wikipedia](#)

```
function quicksort(array)  
    var list less, greater  
    if length(array) < 2  
        return array  
    select and remove a pivot value pivot from array  
    for each x in array  
        if x < pivot + 1 then append x to less  
        else append x to greater  
    return concatenate(quicksort(less), pivot, quicksort(greater))
```

Solution to [Exercise 3](#)

```
def qsort(lst):
    """Quick sort: returns a sorted copy of the list."""
    if len(lst) <= 1:
        return lst
    pivot, rest = lst[0], lst[1:]

    # Could use list comprehension:
    # less_than = [ lt for lt in rest if lt < pivot ]

    less_than = []
    for lt in rest:
        if lt < pivot:
            less_than.append(lt)

    # Could use list comprehension:
    # greater_equal = [ ge for ge in rest if ge >= pivot ]

    greater_equal = []
    for ge in rest:
        if ge >= pivot:
            greater_equal.append(ge)
    return qsort(less_than) + [pivot] + qsort(greater_equal)

# And now check that qsort does sort:
assert qsort(range(10)) == list(range(10))
assert qsort(range(10)[::-1]) == list(range(10))
assert qsort([1, 4, 2, 5, 3]) == sorted([1, 4, 2, 5, 3])
```

Reusing code: scripts and modules

Contents

- Scripts
- Importing objects from modules
- Creating modules
- Introspection
- ‘__main__’ and module loading
- Scripts or modules? How to organize your code
- Packages
- Good practices

For now, we have typed all instructions in the interpreter. For longer sets of instructions we need to change track and write the code in text files (using a text editor), that we will call either *scripts* or *modules*. Use your favorite text editor (provided it offers syntax highlighting for Python), or the editor that comes with the Scientific Python Suite you may be using.

Scripts

Note



Let us first write a *script*, that is a file with a sequence of instructions that are executed each time the script is called. Instructions may be e.g. copied-and-pasted from the interpreter (but take care to respect indentation rules!).

The extension for Python files is `.py`. Write or copy-and-paste the following lines in a file called

`test.py`


```
# Contents of test.py
message = "Hello how are you?"
for word in message.split():
    print(word)
```

Note

Let us now execute the script interactively, that is inside the Jupyter or IPython interpreter. This is maybe the most common use of scripts in scientific computing.

In Jupyter or IPython, the syntax to execute a script is `%run script.py`. For example:

```
%run test.py
```

```
Hello
how
are
you?
```

```
message
```

```
'Hello how are you?'
```

The script has been executed. Moreover the variables defined in the script (such as `message`) are now available inside the interpreter's namespace.

Note

Other interpreters also offer the possibility to execute scripts (e.g., `execfile` in the plain Python interpreter, etc.).

It is also possible In order to execute this script as a *standalone program*, by executing the script inside a shell terminal (Linux/Mac console or cmd Windows console). For example, if we are in the same directory as the [test.py](#) file, we can execute this in a console:

```
$ python test.py
Hello
how
are
you?
```

Tip

Standalone scripts may also take command-line arguments

```
# Contents of my_file.py
import sys

print(sys.argv)
```

```
$ python my_file.py test arguments
['file.py', 'test', 'arguments']
```

Warning

Don't implement option parsing like this yourself. Use a dedicated module such as `argparse`.

Importing objects from modules

```
import os
os
```

```
<module 'os' (frozen)>
```

```
os.listdir('.')
```

```
['reusing_code.md',  
 'test.py',  
 'my_file.py',  
 'exceptions.md',  
 'demo2.py',  
 'python_language.md',  
 'python-logo.png',  
 'basic_types.md',  
 'demo.py',  
 'standard_library.md',  
 'data.txt',  
 'functions.md',  
 'solutions',  
 'oop.md',  
 'workfile',  
 'control_flow.md',  
 'first_steps.md',  
 'io.md']
```

And also:

```
from os import listdir
```

Importing shorthands:

```
import numpy as np
```

Warning

The following code is an example of what is called the *star import* and please, **Do not use** it

```
from os import *
```

- Makes the code harder to read and understand: where do symbols come from?
- Makes it impossible to guess the functionality by the context and the name (hint: `os.name` is the name of the OS), and to profit usefully from tab completion.
- Restricts the variable names you can use: `os.name` might override `name`, or vice-versa.
- Creates possible name clashes between modules.
- Makes the code impossible to statically check for undefined symbols.

Modules are a good way to organize code in a hierarchical way. Actually, all the scientific computing tools we are going to use are modules:

```
import numpy as np # Module for data arrays
import scipy as sp # Module for scientific computing

# Use Numpy
np.linspace(0, 10, 6)
```

```
array([ 0.,  2.,  4.,  6.,  8., 10.])
```

Creating modules

Note

If we want to write larger and better organized programs (compared to simple scripts), where some objects are defined, (variables, functions, classes) and that we want to reuse several times, we have to create our own *modules*.

Let us create a module `demo` contained in the file `demo.py`:

```
"A demo module."

def print_b():
    "Prints b."
    print("b")

def print_a():
    "Prints a."
    print("a")

c = 2
d = 2
```

Note

In this file, we defined two functions `print_a` and `print_b`. Suppose we want to call the `print_a` function from the interpreter. We could execute the file as a script, but since we just want to have access to the function `print_a`, we are rather going to **import it as a module**. The syntax is as follows.

```
import demo

demo.print_a()
```

a

```
demo.print_b()
```

b

Importing the module gives access to its objects, using the `module.object` syntax. Don't forget to put the module's name before the object's name, otherwise Python won't recognize the instruction.

Introspection

```
help(demo)
```

Help on module demo:

NAME

demo - A demo module.

FUNCTIONS

print_a()
Prints a.

print_b()
Prints b.

DATA

c = 2
d = 2

FILE

/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/demo.py

You can get the same output (in Jupyter / IPython) from:

```
demo?
```

An example session:

```
In [4]: demo?
Type:          module
Base Class:    <type 'module'>
String Form:   <module 'demo' from 'demo.py'>
Namespace:    Interactive
File:
/home/varoquau/Projects/Python_talks/scipy_2009_tutorial/source/demo.py
Docstring:
    A demo module.
```

```
In [5]: who
demo
```

```
In [6]: whos
Variable      Type      Data/Info
-----
demo          module    <module 'demo' from 'demo.py'>
```

```
In [7]: dir(demo)
Out[7]:
['__builtins__',
'__doc__',
'__file__',
'__name__',
'__package__',
'c',
'd',
'print_a',
'print_b']
```

```
In [8]: demo.<TAB>
demo.c      demo.print_a  demo.py
demo.d      demo.print_b  demo.pyc
```

Importing objects from modules into the main namespace

```
In [9]: from demo import print_a, print_b
```

```
In [10]: whos
Variable      Type      Data/Info
-----
demo          module    <module 'demo' from 'demo.py'>
print_a       function  <function print_a at 0xb7421534>
print_b       function  <function print_b at 0xb74214c4>
```

```
In [11]: print_a()
a
```

⚠ Warning

Module caching

Modules are cached: if you modify `demo.py` and re-import it in the old session, you will get the old one.

Solution

```
In [10]: importlib.reload(demo)
```

'__main__' and module loading

i Note

Sometimes we want code to be executed when a module is run directly, but not when it is imported by another module. `if __name__ == '__main__':` allows us to check whether the module is being run directly.

File `demo2.py`:

```
def print_b():  
    "Prints b."  
    print("b")  
  
def print_a():  
    "Prints a."  
    print("a")  
  
# print_b() runs on import  
print_b()  
  
if __name__ == "__main__":  
    # print_a() is only executed when the module is run directly.  
    print_a()
```

Importing it:

```
import demo2
```

```
b
```

Importing it again in the same session:

```
import demo2
```

Running it:

```
%run demo2
```

```
b  
a
```

Scripts or modules? How to organize your code

Note

Rule of thumb

- Sets of instructions that are called several times should be written inside **functions** for better code reusability.
- Functions (or other bits of code) that are called from several scripts should be written inside a **module**, so that only the module is imported in the different scripts (do not copy-and-paste your functions in the different scripts!).

How modules are found and imported

When the `import mymodule` statement is executed, the module `mymodule` is searched in a given list of directories. This list includes a list of installation-dependent default path (e.g.,

`/usr/lib64/python3.11`) as well as the list of directories specified by the environment variable `PYTHONPATH`.

The list of directories searched by Python is given by the `sys.path` variable

```
import sys  
sys.path
```



```
['/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12.zip',  
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12',  
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/lib-dynload',  
 '',  
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/site-packages']
```

Modules must be located in the search path, therefore you can:

- write your own modules within directories already defined in the search path (e.g. `$HOME/.venv/lectures/lib64/python3.11/site-packages`). You may use symbolic links (on Linux) to keep the code somewhere else.
- modify the environment variable `PYTHONPATH` to include the directories containing the user-defined modules.

Tip

On Linux/Unix, add the following line to a file read by the shell at startup (e.g. `/etc/profile`, `.profile`)

```
export PYTHONPATH=$PYTHONPATH:/home/emma/user_defined_modules
```

On Windows, <https://support.microsoft.com/kb/310519> explains how to handle environment variables.

- or modify the `sys.path` variable itself within a Python script.

Tip

```
import sys  
new_path = '/home/emma/user_defined_modules'  
if new_path not in sys.path:  
    sys.path.append(new_path)
```

This method is not very robust, however, because it makes the code less portable (user-dependent path) and because you have to add the directory to your `sys.path` each time you want to import from a module in this directory.

See also

See <https://docs.python.org/3/tutorial/modules.html> for more information about modules.

Packages

A directory that contains many modules is called a *package*. A package is a module with submodules (which can have submodules themselves, etc.). A special file called `__init__.py` (which may be empty) tells Python that the directory is a Python package, from which modules can be imported.

```
$ ls
_build_utils/      fft/               _lib/              odr/               spatial/
cluster/           fftpack/           linalg/            optimize/          special/
conftest.py        __init__.py        linalg.pxd         optimize.pxd       special.pxd
constants/         integrate/         meson.build        setup.py           stats/
datasets/          interpolate/       misc/              signal/
_distributor_init.py io/                ndimage/           sparse/
$ cd ndimage
$ ls
_filters.py  __init__.py  _measurements.py  morphology.py  src/
filters.py   _interpolation.py  measurements.py  _ni_docstrings.py  tests/
_fourier.py  interpolation.py  meson.build      _ni_support.py    utils/
fourier.py   LICENSE.txt     _morphology.py   setup.py
```

From Jupyter / IPython:

```
import scipy as sp

sp.__file__
```

```
'/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/site-
packages/scipy/__init__.py'
```

```
sp.version.version
```

```
'1.17.0'
```

```
# Also available as sp.ndimage.binary_dilation?
help(sp.ndimage.binary_dilation)
```

Help on function `binary_dilation` in module `scipy.ndimage._morphology`:

```
binary_dilation(input, structure=None, iterations=1, mask=None, output=None,
border_value=0, origin=0, brute_force=False, *, axes=None)
    Multidimensional binary dilation with the given structuring element.
```

Parameters

`input` : array_like

Binary array_like to be dilated. Non-zero (True) elements form the subset to be dilated.

`structure` : array_like, optional

Structuring element used for the dilation. Non-zero elements are considered True. If no structuring element is provided an element is generated with a square connectivity equal to one.

`iterations` : int, optional

The dilation is repeated `iterations`` times (one, by default). If `iterations` is less than 1, the dilation is repeated until the result does not change anymore. Only an integer of iterations is accepted.

`mask` : array_like, optional

If a mask is given, only those elements with a True value at the corresponding mask element are modified at each iteration.

`output` : ndarray, optional

Array of the same shape as `input`, into which the output is placed. By default, a new array is created.

`border_value` : int (cast to 0 or 1), optional

Value at the border in the output array.

`origin` : int or tuple of ints, optional

Placement of the filter, by default 0.

`brute_force` : boolean, optional

[... salida de 6049 caracteres recortada en el PDF; completa en la versión web ...]

Good practices

- Use **meaningful** object **names**
- **Indentation: no choice!**

💡 Tip

Indenting is compulsory in Python! Every command block following a colon bears an additional indentation level with respect to the previous line with a colon. One must therefore indent after `def f():` or `while:`. At the end of such logical blocks, one decreases the indentation depth (and re-increases it if a new block is entered, etc.)

Strict respect of indentation is the price to pay for getting rid of `{` or `;` characters that delineate logical blocks in other languages. Improper indentation leads to errors such as

```
-----  
IndentationError: unexpected indent (test.py, line 2)
```

All this indentation business can be a bit confusing in the beginning. However, with the clear indentation, and in the absence of extra characters, the resulting code is very nice to read compared to other languages.

- **Indentation depth:** Inside your text editor, you may choose to indent with any positive number of spaces (1, 2, 3, 4, ...). However, it is considered good practice to **indent with 4 spaces**. You may configure your editor to map the `Tab` key to a 4-space indentation.
- **Style guidelines**

Long lines: you should not write very long lines that span over more than (e.g.) 80 characters. Long lines can be broken with the `\` character

```
long_line = "Here is a very very long line \  
that we break in two parts."
```

Spaces

Write well-spaced code: put whitespaces after commas, around arithmetic operators, etc.:

```
a = 1 # yes  
a=1 # too cramped
```

A certain number of rules for writing “beautiful” code (and more importantly using the same conventions as anybody else!) are given in the [Style Guide for Python Code](#).

Quick read

If you want to do a first quick pass through the Scientific Python Lectures to learn the ecosystem, you can directly skip to the next chapter: [NumPy: creating and manipulating numerical data](#).

The remainder of this chapter is not necessary to follow the rest of the intro part. But be sure to come back and finish this chapter later.

Input and Output

Contents

- Iterating over a file

To be exhaustive, here are some information about input and output in Python. Since we will use the NumPy methods to read and write files, **you may skip this chapter at first reading**.

We write or read **strings** to/from files (other types must be converted to strings). To write in a file:

```
f = open('workfile', 'w') # opens the workfile file
type(f)
```

```
_io.TextIOWrapper
```

```
f.write('This is a test \nand another test')
f.close()
```

To read from a file

```
f = open('workfile', 'r')
s = f.read()
print(s)
```

```
This is a test
and another test
```

```
f.close()
```

See also

For more details: <https://docs.python.org/3/tutorial/inputoutput.html>

Iterating over a file

```
f = open('workfile', 'r')

for line in f:
    print(line)
```

```
This is a test

and another test
```

```
f.close()
```

File modes

- Read-only: `r`
- Write-only: `w`
 - Note: Create a new file or *overwrite* existing file.
- Append a file: `a`
- Read and Write: `r+`
- Binary mode: `b`
 - Note: Use for binary files, especially on Windows.

Standard Library

Contents

- `os` module: operating system functionality
- `shutil`: high-level file operations
- `glob`: Pattern matching on files
- `sys` module: system-specific information
- `pickle`: easy persistence
- Exercises

Note

Reference document for this section:

- The Python Standard Library documentation:
<https://docs.python.org/3/library/index.html>
- Python Essential Reference, David Beazley, Addison-Wesley Professional

`os` module: operating system functionality

“A portable way of using operating system dependent functionality.”

Directory and file manipulation

Current directory:

```
import os
os.getcwd()
```



```
 '/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language'
```

List a directory:

```
os.listdir(os.curdir)
```

```
['reusing_code.md',  
 'test.py',  
 'my_file.py',  
 'exceptions.md',  
 'demo2.py',  
 'python_language.md',  
 'python-logo.png',  
 'basic_types.md',  
 'demo.py',  
 'standard_library.md',  
 '__pycache__',  
 'data.txt',  
 'functions.md',  
 'solutions',  
 'oop.md',  
 'workfile',  
 'control_flow.md',  
 'first_steps.md',  
 'io.md']
```

Make a directory:

```
os.mkdir('junkdir')  
'junkdir' in os.listdir(os.curdir)
```

True

Rename the directory:

```
os.rename('junkdir', 'foodir')  
'junkdir' in os.listdir(os.curdir)
```

False

```
'foodir' in os.listdir(os.curdir)
```

True

```
os.rmdir('foodir')  
'foodir' in os.listdir(os.curdir)
```

False

Delete a file:

```
fp = open('junk.txt', 'w')  
fp.close()  
'junk.txt' in os.listdir(os.curdir)
```

True

```
os.remove('junk.txt')  
'junk.txt' in os.listdir(os.curdir)
```

False

`os.path`: path manipulations

`os.path` provides common operations on pathnames.

```
fp = open('junk.txt', 'w')  
fp.close()  
a = os.path.abspath('junk.txt')  
a
```

```
'/home/runner/work/scientific-python-lectures/scientific-python-  
lectures/intro/language/junk.txt'
```

```
os.path.split(a)
```

```
(' /home/runner/work/scientific-python-lectures/scientific-python-  
lectures/intro/language',  
 'junk.txt')
```

```
os.path.dirname(a)
```

```
'/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language'
```

```
os.path.basename(a)
```

```
'junk.txt'
```

```
os.path.splitext(os.path.basename(a))
```

```
('junk', '.txt')
```

```
os.path.exists('junk.txt')
```

```
True
```

```
os.path.isfile('junk.txt')
```

```
True
```

```
os.path.isdir('junk.txt')
```

```
False
```

```
os.path.expanduser('~/.local')
```

```
'/home/runner/local'
```

```
os.path.join(os.path.expanduser('~'), 'local', 'bin')
```

```
'/home/runner/local/bin'
```

Running an external command

```
return_code = os.system('ls')
```

```
__pycache__
basic_types.md
control_flow.md
data.txt
demo.py
demo2.py
exceptions.md
first_steps.md
functions.md
io.md
junk.txt
my_file.py
oop.md
python-logo.png
python_language.md
reusing_code.md
solutions
standard_library.md
test.py
workfile
```

Note

Alternative to `os.system`

A noteworthy alternative to `os.system` is the [sh module](#). Which provides much more convenient ways to obtain the output, error stream and exit code of the external command.

```
import sh
com = sh.ls()

print(com)
basic_types.md      exceptions.md      oop.md
standard_library.md
control_flow.md     first_steps.md   python_language.md
demo2.py            functions.md      python-logo.png
demo.py             io.md            reusing_code.md

type(com)
Out[33]: str
```

Walking a directory

`os.path.walk` generates a list of filenames in a directory tree.

```
for dirpath, dirnames, filenames in os.walk(os.curdir):  
    for fp in filenames:  
        print(os.path.abspath(fp))
```

```
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/reusing_code.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/test.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/my_file.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/exceptions.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/demo2.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/python_language.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/python-logo.png
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/basic_types.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/demo.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/standard_library.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/data.txt
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/functions.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/oop.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/workfile
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/control_flow.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/junk.txt
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/first_steps.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/io.md
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/demo.cpython-312.pyc
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/demo2.cpython-312.pyc
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/path_site.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/dir_sort.py
/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/language/test_dir_sort.py
```

Environment variables:

```
In [2]: os.environ.keys()
Out[2]: KeysView(environ({'SHELL': '/bin/bash', 'PWD': '/home/mb312',
'LOGNAME': 'mb312', 'HOME': '/home/mb312', 'TERM': 'xterm', 'USER': 'mb312',
'SHLVL': '1', 'PATH':
'/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/l
ocal/games:/snap/bin', 'MAIL': '/var/mail/mb312', '_': '/usr/bin/python3',
'LC_CTYPE': 'C.UTF-8'}))

In [3]: os.environ['SHELL']
Out[3]: '/bin/bash'
```

shutil: high-level file operations

The `shutil` provides useful file operations:

- `shutil.rmtree`: Recursively delete a directory tree.
- `shutil.move`: Recursively move a file or directory to another location.
- `shutil.copy`: Copy files or directories.

glob: Pattern matching on files

The `glob` module provides convenient file pattern matching.

Find all files ending in `.txt`:

```
import glob
glob.glob('*.txt')
```

```
['data.txt', 'junk.txt']
```

sys module: system-specific information

System-specific information related to the Python interpreter.

Which version of Python are you running and where is it installed:

```
import sys
sys.platform
```

```
'linux'
```

```
sys.version
```

```
'3.12.12 (main, Oct 10 2025, 01:01:16) [GCC 13.3.0]'
```

```
sys.prefix
```

```
'/opt/hostedtoolcache/Python/3.12.12/x64'
```

`sys.argv` gives you a **list of command line arguments** passed to a Python script. It is useful when you call as script with e.g. `python my_script.py some arguments`. Inside the `my_arguments.py` script, you can get the passed arguments (here ['some', 'arguments']) with `sys.argv`.

`sys.path` is a list of strings that specifies the search path for modules. Initialized from `PYTHONPATH`:

```
sys.path
```

```
['/opt/hostedtoolcache/Python/3.12.12/x64/lib/python312.zip',
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12',
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/lib-dynload',
 '',
 '/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/site-packages']
```

`pickle`: easy persistence

Useful to store arbitrary objects to a file. Not safe or fast!

```
import pickle
l = [1, None, 'Stan']
with open('test.pkl', 'wb') as file:
    pickle.dump(l, file)
```



```
with open('test.pkl', 'rb') as file:  
    out = pickle.load(file)
```

out

[1, None, 'Stan']

Exercises

Exercise 4 ^

Write a function that will load the column of numbers in `data.txt` and calculate the min, max and sum values. Use no modules except those in the standard library; specifically, do not use Numpy.

 [data.txt](#):

```
10.2  
43.1  
32.6  
32.5  
61.3  
58.2
```

Solution to [Exercise 4](#)



```
def load_data(filename):
    fp = open(filename)
    data_string = fp.read()
    fp.close()

    data = []
    for x in data_string.split():
        # Data is read in as a string. We need to convert it to floats
        data.append(float(x))

    # Could instead use the following one line with list
    # comprehensions!
    # data = [float(x) for x in data_string.split()]
    return data
```

```
data = load_data("data.txt")
# Python provides these basic math functions.
print(f"min: {min(data):f}")
print(f"max: {max(data):f}")
print(f"sum: {sum(data):f}")
```

```
min: 10.200000
max: 61.300000
sum: 237.900000
```

Exercise 5

Implement a *script* that takes a directory name as argument, and returns the list of '.py' files, sorted by name length.

Hint: try to understand the docstring of `list.sort`

Solution to [Exercise 5](#)

```
"""
Script to list all the '.py' files in a directory, in the order of file
name length.
"""

import os
import sys

def filter_and_sort(file_list):
    """Out of a list of file names, returns only the ones ending by
    '.py', ordered with increasing file name length.
    """
    file_list = [filename for filename in file_list if
filename.endswith(".py")]

    def key(item):
        return len(item)

    file_list.sort(key=key)
    return file_list

if __name__ == "__main__":
    file_list = os.listdir(sys.argv[-1])
    sorted_file_list = filter_and_sort(file_list)
    print(sorted_file_list)
```

Exercise 6

Write a program to search your `PYTHONPATH` for the module `site.py`.

Solution to [Exercise 6](#)

```
"""Script to search the PYTHONPATH for the module site.py"""

import os
import sys
import glob

def find_module(module):
    result = []
    # Loop over the list of paths in sys.path
    for subdir in sys.path:
        # Join the subdir path with the module we're searching for
        pth = os.path.join(subdir, module)
        # Use glob to test if the pth exists
        res = glob.glob(pth)
        # glob returns a list, if it is not empty, the pth exists
        if len(res) > 0:
            result.append(res)
    return result

if __name__ == "__main__":
    result = find_module("site.py")
    print(result)
```

Exception handling in Python

Contents

- Exceptions
- Catching exceptions
- Raising exceptions

It is likely that you have raised Exceptions if you have typed all the previous commands of the tutorial. For example, you may have raised an exception if you entered a command with a typo.

Exceptions are raised by different kinds of errors arising when executing Python code. In your own code, you may also catch errors, or define custom error types. You may want to look at the descriptions of the [the built-in Exceptions](#) when looking for the right exception type.

Exceptions

Exceptions are raised by errors in Python:

```
1/0
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[1], line 1  
----> 1 1/0  
  
ZeroDivisionError: division by zero
```

```
1 + 'e'
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 1 + 'e'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
d = {1:1, 2:2}
d[3]
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[3], line 2
      1 d = {1:1, 2:2}
----> 2 d[3]

KeyError: 3
```

```
l = [1, 2, 3]
l[4]
```

```
-----
IndexError                              Traceback (most recent call last)
Cell In[4], line 2
      1 l = [1, 2, 3]
----> 2 l[4]

IndexError: list index out of range
```

```
l.foobar
```

```
-----
AttributeError                           Traceback (most recent call last)
Cell In[5], line 1
----> 1 l.foobar

AttributeError: 'list' object has no attribute 'foobar'
```

As you can see, there are **different types** of exceptions for different errors.

Catching exceptions

try/except

```
In [1]: while True:
.....:     try:
.....:         x = int(input('Please enter a number: '))
.....:         break
.....:     except ValueError:
.....:         print('That was no valid number. Try again...')
.....:

Please enter a number: a
That was no valid number. Try again...
Please enter a number: 1

In [2]: x
Out[9]: 1
```

try/finally

```
In [1]: try:
.....:     x = int(input('Please enter a number: '))
.....:     finally:
.....:         print('Thank you for your input')
.....:

Please enter a number: a
Thank you for your input
-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 2
      1 try:
----> 2     x = int(input('Please enter a number: '))
      3 finally:
      4     print('Thank you for your input')
ValueError: invalid literal for int() with base 10: 'a'
```

Important for resource management (e.g. closing a file)

Easier to ask for forgiveness than for permission

```
def print_sorted(collection):  
    try:  
        collection.sort()  
    except AttributeError:  
        pass # The pass statement does nothing  
    print(collection)
```

```
print_sorted([1, 3, 2])
```

```
[1, 2, 3]
```

```
print_sorted(set((1, 3, 2)))
```

```
{1, 2, 3}
```

```
print_sorted('132')
```

```
132
```

Raising exceptions

Capturing and re-raising an exception:

```
def filter_name(name):  
    try:  
        name = name.encode('ascii')  
    except UnicodeError as e:  
        if name == 'Gaël':  
            print('OK, Gaël')  
        else:  
            raise e  
    return name  
  
filter_name('Gaël')
```

```
OK, Gaël
```



```
'Gaël'
```

```
filter_name('Stéfan')
```

```
-----  
UnicodeEncodeError                                Traceback (most recent call last)  
Cell In[11], line 1  
----> 1 filter_name('Stéfan')  
  
Cell In[10], line 8, in filter_name(name)  
      6         print('OK, Gaël')  
      7     else:  
----> 8         raise e  
      9     return name  
  
Cell In[10], line 3, in filter_name(name)  
      1 def filter_name(name):  
      2     try:  
----> 3         name = name.encode('ascii')  
      4     except UnicodeError as e:  
      5         if name == 'Gaël':  
  
UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 2:  
ordinal not in range(128)
```

Exceptions to pass messages between parts of the code:

```
def achilles_arrow(x):  
    if abs(x - 1) < 1e-3:  
        raise StopIteration  
    x = 1 - (1-x)/2.  
    return x  
  
x = 0  
  
while True:  
    try:  
        x = achilles_arrow(x)  
    except StopIteration:  
        break  
  
x
```

```
0.9990234375
```

Use exceptions to notify certain conditions are met (e.g. `StopIteration`) or not (e.g. custom error raising).

Object-oriented programming (OOP)

Python supports object-oriented programming (OOP). The goals of OOP are:

- to organize the code, and
- to reuse code in similar contexts.

Here is a small example: we create a *Student* class, which is an object gathering several custom functions (*methods*) and variables (*attributes*), we will be able to use:

```
class Student(object):  
    def __init__(self, name):  
        self.name = name  
    def set_age(self, age):  
        self.age = age  
    def set_major(self, major):  
        self.major = major
```

```
anna = Student('anna')  
anna.set_age(21)  
anna.set_major('physics')
```

In the previous example, the *Student* class has `__init__`, `set_age` and `set_major` methods. Its attributes are `name`, `age` and `major`. We can call these methods and attributes with the following notation: `classinstance.method` or `classinstance.attribute`. The `__init__` constructor is a special method we call with: `MyClass(init parameters if any)`.

Now, suppose we want to create a new class *MasterStudent* with the same methods and attributes as the previous one, but with an additional `internship` attribute. We won't copy the previous class, but **inherit** from it:

```
class MasterStudent(Student):  
    internship = 'mandatory, from March to June'
```

```
james = MasterStudent('james')  
james.internship
```

```
'mandatory, from March to June'
```

```
james.set_age(23)  
james.age
```

```
23
```

The MasterStudent class inherited from the Student attributes and methods.

Thanks to classes and object-oriented programming, we can organize code with different classes corresponding to different objects we encounter (an Experiment class, an Image class, a Flow class, etc.), with their own methods and attributes. Then we can use inheritance to consider variations around a base class and **reuse** code. Ex : from a Flow base class, we can create derived StokesFlow, TurbulentFlow, PotentialFlow, etc.

The NumPy array object

Contents

- What are NumPy and NumPy arrays?
- Creating arrays
- Basic data types
- Bool
- Strings
- Much more:
- Basic visualization
- Indexing and slicing
- Copies and views
- Fancy indexing

► Show code cell source

What are NumPy and NumPy arrays?

NumPy arrays

NumPy provides:

- An extension package to Python for multi-dimensional arrays.
- An implementation that is closer to hardware (efficiency).
- Package designed for scientific computation (convenience).
- An implementation of *array oriented computing*.

```
import numpy as np

a = np.array([0, 1, 2, 3])
a
```

```
array([0, 1, 2, 3])
```

Note

For example, An array containing:

- values of an experiment/simulation at discrete time steps
- signal recorded by a measurement device, e.g. sound wave
- pixels of an image, grey-level or colour
- 3-D data measured at different X-Y-Z positions, e.g. MRI scan
- ...

Why it is useful: Memory-efficient container that provides fast numerical operations.

```
L = range(1000)
%timeit [i**2 for i in L]
```

45.6 μ s $\pm$ 15.5 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

```
a = np.arange(1000)
%timeit a**2
```

812 ns $\pm$ 1.53 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

NumPy Reference documentation

On the web:

<https://numpy.org/doc/>

Interactive help:

```
In [5]: np.array?
String Form:<built-in function array>
Docstring:
array(object, dtype=None, copy=True, order=None, subok=False, ndmin=0, ...
```

You can also use the Python builtin `help` command to show the docstring for a function:

```
help(np.array)
```

Help on built-in function array in module numpy:

```
array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0,
ndmax=0, like=None)
    array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0,
          ndmax=0, like=None)
```

Create an array.

Parameters

object : array_like

An array, any object exposing the array interface, an object whose `__array__` method returns an array, or any (nested) sequence. If object is a scalar, a 0-dimensional array containing object is returned.

dtype : data-type, optional

The desired data-type for the array. If not given, NumPy will try to use

a default `dtype` that can represent the values (by applying promotion

rules when necessary.)

copy : bool, optional

If `True` (default), then the array data is copied. If `None`, a copy will only be made if `__array__` returns a copy, if obj is a nested sequence, or if a copy is needed to satisfy any of the other requirements (`dtype`, `order`, etc.). Note that any copy of the data is shallow, i.e., for arrays with object dtype, the new array will point to the same objects. See Examples for `ndarray.copy`. For `False` it raises a `ValueError` if a copy cannot be avoided. Default: `True`.

order : {'K', 'A', 'C', 'F'}, optional

Specify the memory layout of the array. If object is not an array, the

[... salida de 5865 caracteres recortada en el PDF; completa en la versión web ...]

Looking for something:

```
In [6]: np.con*?  
np.concatenate  
np.conj  
np.conjugate  
np.convolve
```

Import conventions

The recommended convention to import NumPy is:

```
import numpy as np
```

Creating arrays

Manual construction of arrays

- **1-D:**

```
a = np.array([0, 1, 2, 3])  
a
```

```
array([0, 1, 2, 3])
```

```
a.ndim
```

```
1
```

```
a.shape
```

```
(4,)
```

```
len(a)
```


4

- **2-D, 3-D, ...:**

```
b = np.array([[0, 1, 2], [3, 4, 5]])    # 2 x 3 array
b
```

```
array([[0, 1, 2],
       [3, 4, 5]])
```

```
b.ndim
```

2

```
b.shape
```

(2, 3)

```
len(b)    # returns the size of the first dimension
```

2

```
c = np.array([[1], [2]], [[3], [4]])
c
```

```
array([[1],
       [2],

       [[3],
        [4]])
```

```
c.shape
```

(2, 2, 1)

Exercise 7



- Create a simple two dimensional array. First, redo the examples from above. And then create your own: how about odd numbers counting backwards on the first row, and even numbers on the second?
- Use the functions `len()`, `numpy.shape()` on these arrays. How do they relate to each other? And to the `ndim` attribute of the arrays?

Functions for creating arrays

Note



In practice, we rarely enter items one by one...

Evenly spaced:

```
a = np.arange(10) # 0 .. n-1 (!)
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
b = np.arange(1, 9, 2) # start, end (exclusive), step
b
```

```
array([1, 3, 5, 7])
```

— or by number of points

```
c = np.linspace(0, 1, 6) # start, end, num-points
c
```

```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

```
d = np.linspace(0, 1, 5, endpoint=False)
d
```

```
array([0. , 0.2, 0.4, 0.6, 0.8])
```

Common arrays

```
a = np.ones((3, 3)) # reminder: (3, 3) is a tuple
a
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

```
b = np.zeros((2, 2))
b
```

```
array([[0., 0.],
       [0., 0.]])
```

```
c = np.eye(3)
c
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

```
d = np.diag(np.array([1, 2, 3, 4]))
d
```

```
array([[1, 0, 0, 0],
       [0, 2, 0, 0],
       [0, 0, 3, 0],
       [0, 0, 0, 4]])
```

- [numpy.random](#): random numbers (Mersenne Twister PRNG):

```
rng = np.random.default_rng(27446968)
a = rng.random(4) # uniform in [0, 1]
a
```

```
array([0.64613018, 0.48984931, 0.50851229, 0.22563948])
```

```
b = rng.standard_normal(4)      # Gaussian
b
```

```
array([-0.38250769, -0.61536465,  0.98131732,  0.59353096])
```

Exercise 8



- Experiment with `arange`, `linspace`, `ones`, `zeros`, `eye` and `diag`.
- Create different kinds of arrays with random numbers.
- Try setting the seed before creating an array with random values.
- Look at the function `np.empty`. What does it do? When might this be useful?

Exercise 9



- construct an array containing: 1 2 3 4 5
- construct an array containing: -5, -4, -3, -2, -1
- Construct: 2 4 6 8
- Construct 15 equispaced numbers in range [0, 10]

Solution to [Exercise 9](#)



```
np.arange(1, 6)
```

```
array([1, 2, 3, 4, 5])
```

```
np.arange(-5, 0)
```

```
array([-5, -4, -3, -2, -1])
```

```
np.arange(2, 10, 2)
```

```
array([2, 4, 6, 8])
```

```
np.linspace(0, 10, 15)
```

```
array([ 0.          ,  0.71428571,  1.42857143,  2.14285714,  2.85714286,  
       3.57142857,  4.28571429,  5.          ,  5.71428571,  6.42857143,  
       7.14285714,  7.85714286,  8.57142857,  9.28571429, 10.          ])
```

Basic data types

You may have noticed that, in some instances, array elements are displayed with a trailing dot (e.g. `2.` vs `2`). This is due to a difference in the data-type used:

```
a = np.array([1, 2, 3])  
a.dtype
```

```
dtype('int64')
```

```
b = np.array([1., 2., 3.])  
b.dtype
```

```
dtype('float64')
```

Note

Different data-types allow us to store data more compactly in memory, but most of the time we simply work with floating point numbers. Note that, in the example above, NumPy auto-detects the data-type from the input.

You can explicitly specify which data-type you want:

```
c = np.array([1, 2, 3], dtype=float)
c.dtype
```

```
dtype('float64')
```

The **default** data type is floating point:

```
a = np.ones((3, 3))
a.dtype
```

```
dtype('float64')
```

There are also other types:

Bool

```
e = np.array([True, False, False, True])
e.dtype
```

```
dtype('bool')
```

Strings

```
f = np.array(['Bonjour', 'Hello', 'Hallo'])
f.dtype      # <--- strings containing max. 7 letters
```

```
dtype('<U7')
```

Much more:

- `int32`
- `int64`
- `uint32`
- `uint64`
- ...

Basic visualization

Now that we have our first data arrays, we are going to visualize them.

Start by launching IPython:

```
$ ipython # or ipython3 depending on your install
```

Or the notebook:

```
$ jupyter notebook
```

If you are using IPython enable interactive plots with:

```
%matplotlib
```

```
Using matplotlib backend: module://matplotlib_inline.backend_inline
```

Interactive plots are enabled automatically in the Jupyter Notebook.

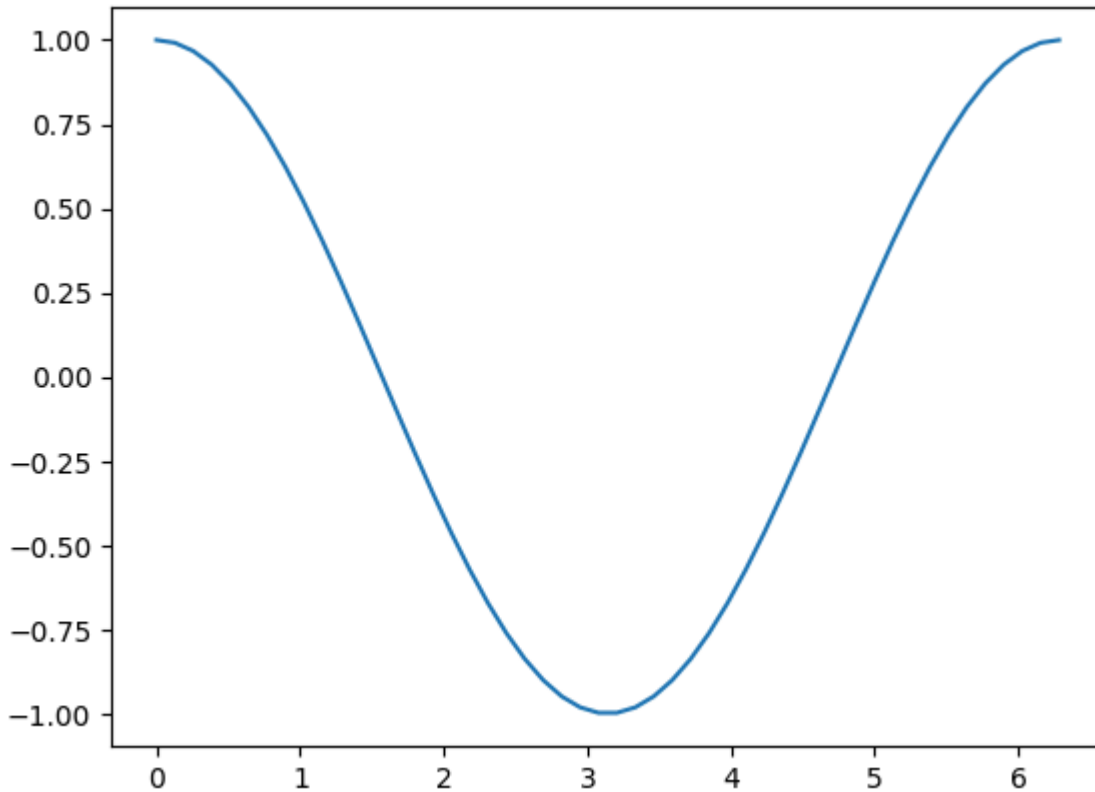
Matplotlib is a 2D plotting package. We can import its functions as below:

```
import matplotlib.pyplot as plt # the tidy way
```

And then use (note that you have to use `show` explicitly if you have not enabled interactive plots with `%matplotlib`):

```
# Example data
x = np.linspace(0, 2 * np.pi)
y = np.cos(x)

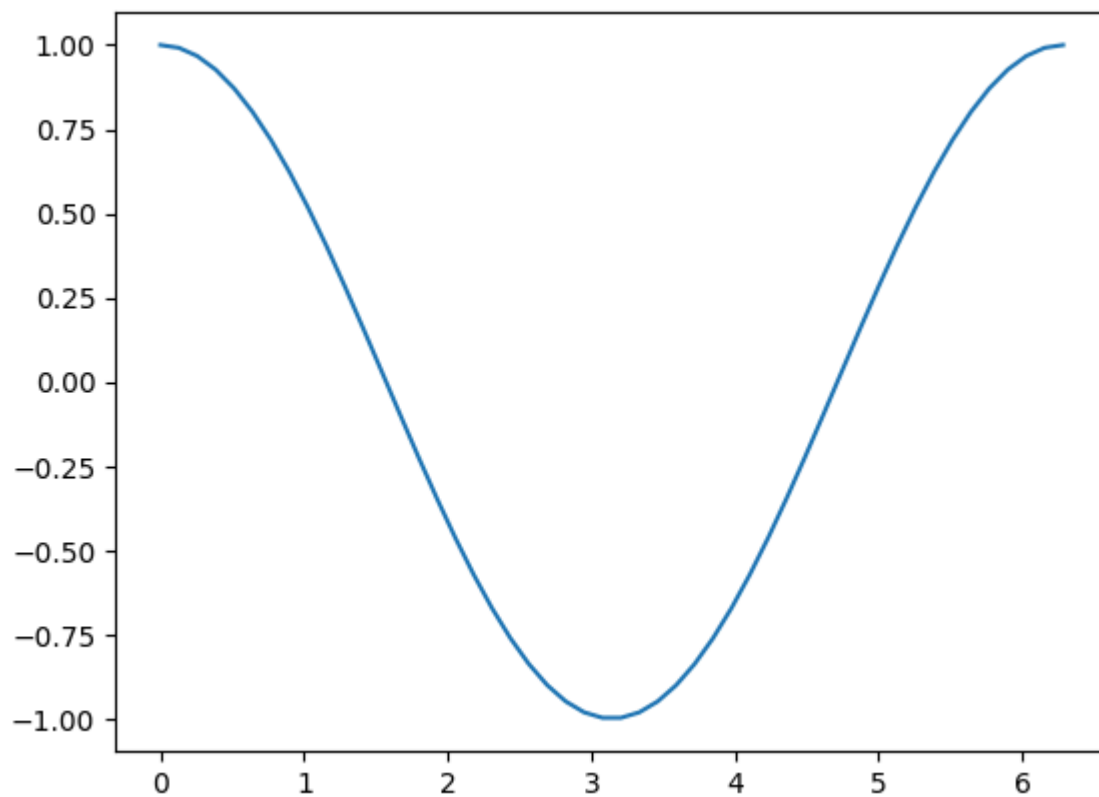
plt.plot(x, y)      # line plot
plt.show()          # <-- shows the plot (not needed with interactive plots)
```



Or, if you have enabled interactive plots with `%matplotlib`:

```
plt.plot(x, y)      # line plot
```

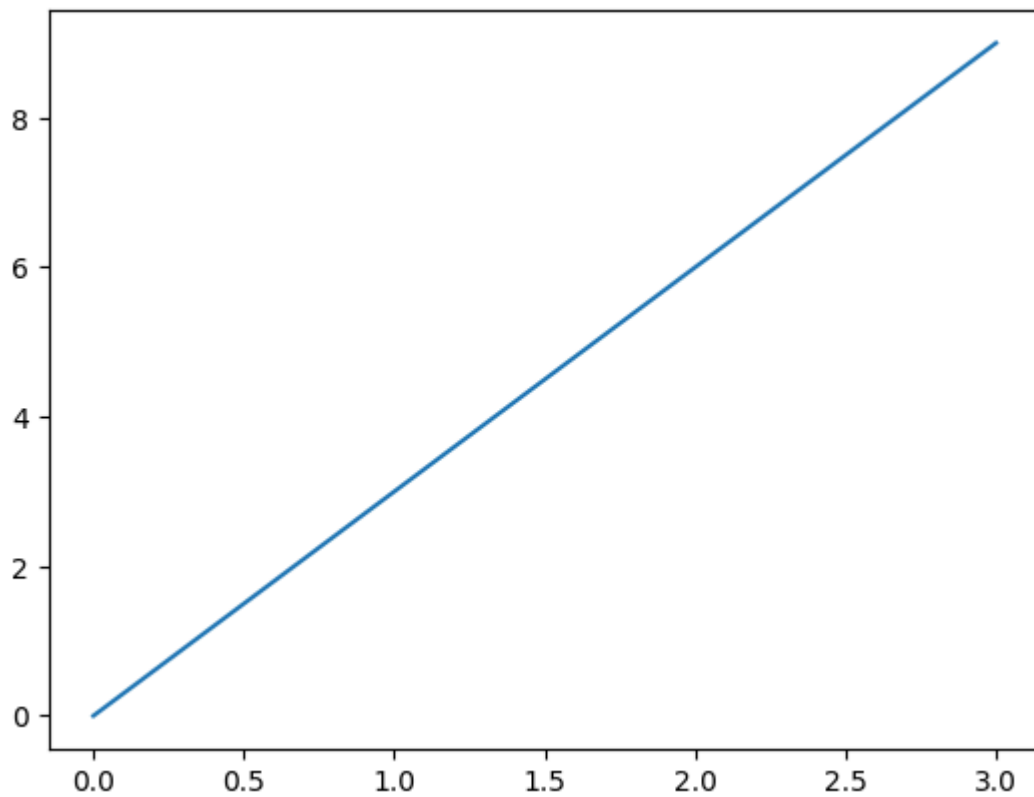
```
[<matplotlib.lines.Line2D at 0x7ff2f83c8ef0>]
```

- **1D plotting:**

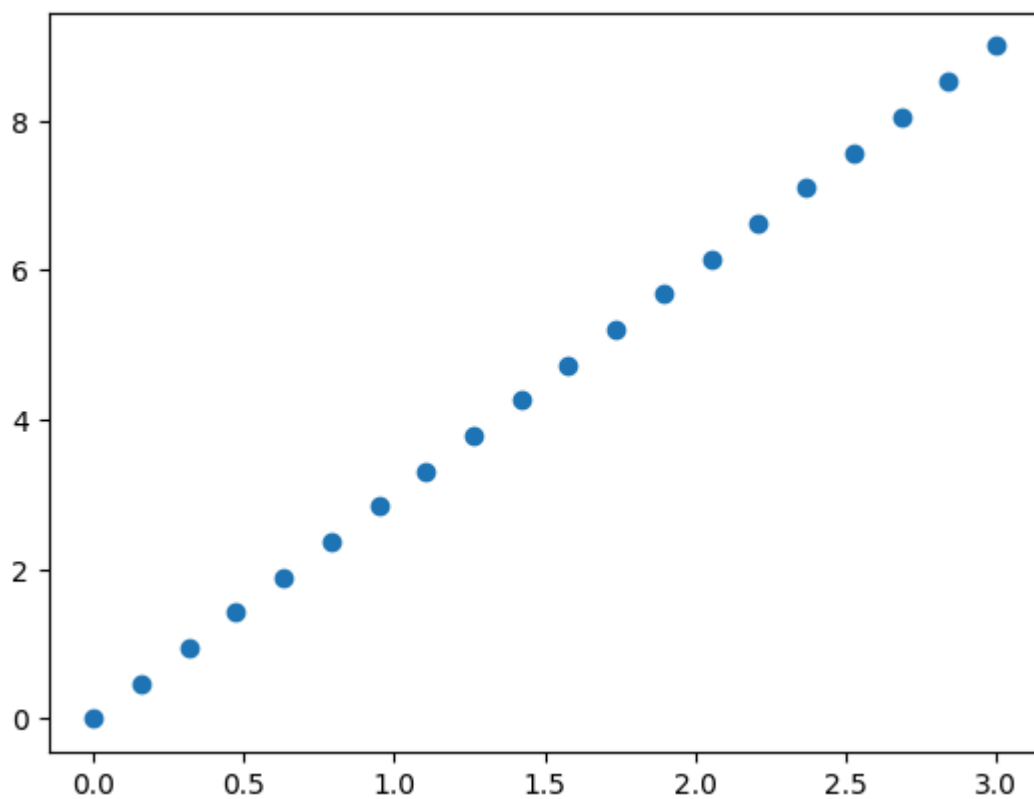
```
x = np.linspace(0, 3, 20)
y = np.linspace(0, 9, 20)
plt.plot(x, y)      # line plot
```

```
[<matplotlib.lines.Line2D at 0x7ff2f829d1f0>]
```



```
plt.plot(x, y, 'o') # dot plot
```

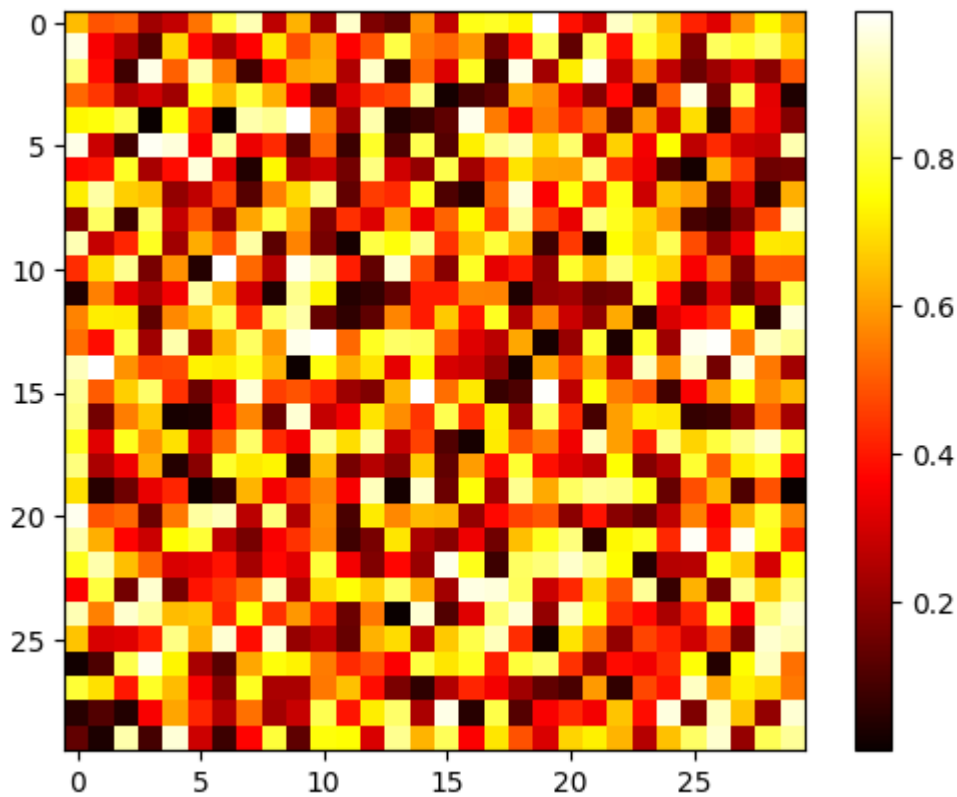
```
[<matplotlib.lines.Line2D at 0x7ff2f82f2750>]
```



- **2D arrays** (such as images):

```
rng = np.random.default_rng(27446968)
image = rng.random((30, 30))
plt.imshow(image, cmap=plt.cm.hot)
plt.colorbar()
```

<matplotlib.colorbar.Colorbar at 0x7ff2f83cb050>



See also

More in the: [matplotlib chapter](#)

Exercise 10

- Plot some simple arrays: a cosine as a function of time and a 2D matrix.
- Try using the `gray` colormap on the 2D matrix.

Indexing and slicing

The items of an array can be accessed and assigned to the same way as other Python sequences (e.g. lists):

```
a = np.arange(10)
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
a[0], a[2], a[-1]
```

```
(np.int64(0), np.int64(2), np.int64(9))
```

Warning

Indices begin at 0, like other Python sequences (and C/C++). In contrast, in Fortran or Matlab, indices begin at 1.

The usual python idiom for reversing a sequence is supported:

```
a[::-1]
```

```
array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

For multidimensional arrays, indices are tuples of integers:

```
a = np.diag(np.arange(3))
a
```

```
array([[0, 0, 0],
       [0, 1, 0],
       [0, 0, 2]])
```

```
a[1, 1]
```

```
np.int64(1)
```

```
a[2, 1] = 10 # third line, second column
a
```

```
array([[ 0,  0,  0],
       [ 0,  1,  0],
       [ 0, 10,  2]])
```

```
a[1]
```

```
array([0, 1, 0])
```

Note

- In 2D, the first dimension corresponds to **rows**, the second to **columns**.
- for multidimensional `a`, `a[0]` is interpreted by taking all elements in the unspecified dimensions.

Slicing: Arrays, like other Python sequences can also be sliced:

```
a = np.arange(10)
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
a[2:9:3] # [start:end:step]
```

```
array([2, 5, 8])
```

Note that the last index is not included! :

```
a[:4]
```

```
array([0, 1, 2, 3])
```

All three slice components are not required: by default, `start` is 0, `end` is the last and `step` is 1:

```
a[1:3]
```

```
array([1, 2])
```

```
a[::2]
```

```
array([0, 2, 4, 6, 8])
```

```
a[3:]
```

```
array([3, 4, 5, 6, 7, 8, 9])
```

A small illustrated summary of NumPy indexing and slicing...

```
>>> a[0, 3:5]
```

```
array([3, 4])
```

```
>>> a[4:, 4:]
```

```
array([[44, 55],  
       [54, 55]])
```

```
>>> a[:, 2]
```

```
a([2, 12, 22, 32, 42, 52])
```

```
>>> a[2::2, ::2]
```

```
array([[20, 22, 24],  
       [40, 42, 44]])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

You can also combine assignment and slicing:

```
a = np.arange(10)  
a[5:] = 10  
a
```

```
array([ 0,  1,  2,  3,  4, 10, 10, 10, 10, 10])
```

```
b = np.arange(5)  
a[5:] = b[::-1]  
a
```

```
array([0, 1, 2, 3, 4, 4, 3, 2, 1, 0])
```

Exercise 11

- Try the different flavours of slicing, using `start`, `end` and `step`: starting from a linspace, try to obtain odd numbers counting backwards, and even numbers counting forwards.
- Reproduce the slices in the diagram above. You may use the following expression to create the array:

```
np.arange(6) + np.arange(0, 51, 10)[: , np.newaxis]
```

Exercise 12

An exercise on array creation.

Create the following arrays (with correct data types):

```
[[1, 1, 1, 1],  
 [1, 1, 1, 1],  
 [1, 1, 1, 2],  
 [1, 6, 1, 1]]  
  
[[0., 0., 0., 0., 0.],  
 [2., 0., 0., 0., 0.],  
 [0., 3., 0., 0., 0.],  
 [0., 0., 4., 0., 0.],  
 [0., 0., 0., 5., 0.],  
 [0., 0., 0., 0., 6.]]
```

Par on course: 3 statements for each.

Hint: Individual array elements can be accessed similarly to a list, e.g. `a[1]` or `a[1, 2]`.

Hint: Examine the docstring for `diag`.

Solution to [Exercise 12](#)

```
a = np.ones((4, 4), dtype=int)
a[3, 1] = 6
a[2, 3] = 2
a
```

```
array([[1, 1, 1, 1],
       [1, 1, 1, 1],
       [1, 1, 1, 2],
       [1, 6, 1, 1]])
```

```
b = np.zeros((6, 5))
b[1:] = np.diag(np.arange(2, 7))
b
```

```
array([[0., 0., 0., 0., 0.],
       [2., 0., 0., 0., 0.],
       [0., 3., 0., 0., 0.],
       [0., 0., 4., 0., 0.],
       [0., 0., 0., 5., 0.],
       [0., 0., 0., 0., 6.]])
```

Exercise 13

Exercise on tiling for array creation.

Skim through the documentation for `np.tile`, and use this function to construct the array:

```
[[4, 3, 4, 3, 4, 3],
 [2, 1, 2, 1, 2, 1],
 [4, 3, 4, 3, 4, 3],
 [2, 1, 2, 1, 2, 1]]
```


Solution to [Exercise 13](#)

```
block = np.array([[4, 3], [2, 1]])  
a = np.tile(block, (2, 3))  
a
```

```
array([[4, 3, 4, 3, 4, 3],  
       [2, 1, 2, 1, 2, 1],  
       [4, 3, 4, 3, 4, 3],  
       [2, 1, 2, 1, 2, 1]])
```

Copies and views

A slicing operation creates a **view** on the original array, which is just a way of accessing array data. Thus the original array is not copied in memory. You can use `np.may_share_memory()` to check if two arrays share the same memory block. Note however, that this uses heuristics and may give you false positives.

When modifying the view, the original array is modified as well:

```
a = np.arange(10)  
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
b = a[::2]  
b
```

```
array([0, 2, 4, 6, 8])
```

```
np.may_share_memory(a, b)
```

```
True
```

```
b[0] = 12  
b
```

```
array([12,  2,  4,  6,  8])
```

a # (!)

```
array([12,  1,  2,  3,  4,  5,  6,  7,  8,  9])
```

```
a = np.arange(10)
c = a[:, :2].copy() # force a copy
c[0] = 12
a
```

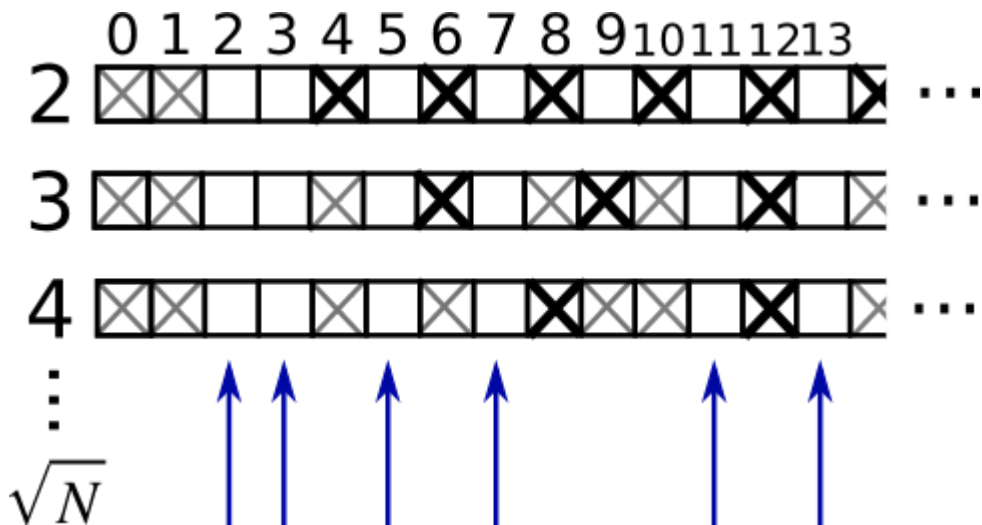
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.may_share_memory(a, c)
```

False

This behavior can be surprising at first sight... but it allows to save both memory and time.

Worked example: Prime number sieve



Compute prime numbers in 0–99, with a sieve

First — construct a shape (100,) boolean array `is_prime`, filled with True in the beginning:

```
is_prime = np.ones((100,), dtype=bool)
```

Next, cross out 0 and 1 which are not primes:

```
is_prime[:2] = 0
```

For each integer `j` starting from 2, cross out its higher multiples:

```
N_max = int(np.sqrt(len(is_prime) - 1))
for j in range(2, N_max + 1):
    is_prime[2*j::j] = False
```

Skim through `help(np.nonzero)`, and print the prime numbers

- Follow-up:
 - Move the above code into a script file named `prime_sieve.py`
 - Run it to check it works
 - Use the optimization suggested in [the sieve of Eratosthenes](#):

1. Skip `j` which are already known to not be primes
2. The first number to cross out is j^2

Fancy indexing

Note

NumPy arrays can be indexed with slices, but also with boolean or integer arrays (**masks**). This method is called *fancy indexing*. It creates **copies not views**.

Using boolean masks

```
rng = np.random.default_rng(27446968)
a = rng.integers(0, 21, 15)
a
```

```
array([ 3, 13, 12, 10, 10, 10, 18,  4,  8,  5,  6, 11, 12, 17,  3])
```

```
(a % 3 == 0)
```

```
array([ True, False,  True, False, False, False,  True, False, False,
        False,  True, False,  True, False,  True])
```

```
mask = (a % 3 == 0)
extract_from_a = a[mask] # or, a[a%3==0]
extract_from_a      # extract a sub-array with the mask
```

```
array([ 3, 12, 18,  6, 12,  3])
```

Indexing with a mask can be very useful to assign a new value to a sub-array:

```
a[a % 3 == 0] = -1
a
```

```
array([-1, 13, -1, 10, 10, 10, -1,  4,  8,  5, -1, 11, -1, 17, -1])
```

Indexing with an array of integers

```
a = np.arange(0, 100, 10)
a
```

```
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
```

Indexing can be done with an array of integers, where the same index is repeated several time:

```
a[[2, 3, 2, 4, 2]] # note: [2, 3, 2, 4, 2] is a Python list
```

```
array([20, 30, 20, 40, 20])
```

New values can be assigned with this kind of indexing:

```
a[[9, 7]] = -100
a
```

```
array([ 0, 10, 20, 30, 40, 50, 60, -100, 80, -100])
```

Tip

When a new array is created by indexing with an array of integers, the new array has the same shape as the array of integers:

```
a = np.arange(10)
idx = np.array([[3, 4], [9, 7]])
idx.shape
```

```
(2, 2)
```

```
a[idx]
```

```
array([[3, 4],
       [9, 7]])
```

The image below illustrates various fancy indexing applications

```
>>> a[(0,1,2,3,4), (1,2,3,4,5)]
array([1, 12, 23, 34, 45])
```

```
>>> a[3:, [0,2,5]]
array([[30, 32, 35],
       [40, 42, 45],
       [50, 52, 55]])
```

```
>>> mask = np.array([1,0,1,0,0,1], dtype=bool)
>>> a[mask, 2]
array([2, 22, 52])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

Exercise 14

- Again, reproduce the fancy indexing shown in the diagram above.
- Use fancy indexing on the left and array creation on the right to assign values into an array, for instance by setting parts of the array in the diagram above to zero.

We can even use fancy indexing and [broadcasting](#) at the same time:

```
a = np.arange(12).reshape(3,4)
a
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
i = np.array([[0, 1], [1, 2]])
a[i, 2] # same as a[i, 2 * np.ones((2, 2), dtype=int)]
```

```
array([[ 2,  6],
       [ 6, 10]])
```

Numerical operations on arrays

Contents

- Elementwise operations
- Basic reductions
- Interim summary and exercises
- Broadcasting
- Array shape manipulation
- Sorting data
- Summary

► [Show code cell source](#)

Elementwise operations

Basic operations

With scalars:

```
a = np.array([1, 2, 3, 4])  
a + 1
```

```
array([2, 3, 4, 5])
```

```
2 ** a
```

```
array([ 2,  4,  8, 16])
```

All arithmetic operates elementwise:

```
b = np.ones(4) + 1
a - b
```

```
array([-1.,  0.,  1.,  2.])
```

```
a * b
```

```
array([2., 4., 6., 8.])
```

```
j = np.arange(5)
2**(j + 1) - j
```

```
array([ 2,  3,  6, 13, 28])
```

These operations are of course much faster than if you did them in pure python:

```
a = np.arange(10000)
%timeit a + 1
```

3.18 μ s $\pm$ 1.79 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

```
l = range(10000)
%timeit [i+1 for i in l]
```

352 μ s $\pm$ 3.85 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)

Warning: array multiplication is not matrix multiplication

Consider these examples:

```
c = np.ones((3, 3))
c * c          # NOT matrix multiplication!
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

Matrix multiplication:

c @ c

```
array([[3., 3., 3.],  
       [3., 3., 3.],  
       [3., 3., 3.]])
```

Exercise 15

- Try simple arithmetic elementwise operations: add even elements with odd elements
- Time them against their pure python counterparts using `%timeit`.
- Generate:
 - `[2**0, 2**1, 2**2, 2**3, 2**4]`
 - `a_j = 2^(3*j) - j`

Other operations

Comparisons

```
a = np.array([1, 2, 3, 4])  
b = np.array([4, 2, 2, 4])  
a == b
```

```
array([False,  True, False,  True])
```

```
a > b
```

```
array([False, False,  True, False])
```

Array-wise comparisons:

```
a = np.array([1, 2, 3, 4])  
b = np.array([4, 2, 2, 4])  
c = np.array([1, 2, 3, 4])  
np.array_equal(a, b)
```

False

```
np.array_equal(a, c)
```

True

Logical operations

```
a = np.array([1, 1, 0, 0], dtype=bool)
b = np.array([1, 0, 1, 0], dtype=bool)
np.logical_or(a, b)
```

```
array([ True,  True,  True, False])
```

```
np.logical_and(a, b)
```

```
array([ True, False, False, False])
```

Transcendental functions

```
a = np.arange(5)
np.sin(a)
```

```
array([ 0.          ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ])
```

```
np.exp(a)
```

```
array([ 1.          ,  2.71828183,  7.3890561 , 20.08553692, 54.59815003])
```

```
np.log(np.exp(a))
```

```
array([0., 1., 2., 3., 4.])
```

Shape mismatches

```
a = np.arange(4)
a + np.array([1, 2])
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[20], line 2
      1 a = np.arange(4)
----> 2 a + np.array([1, 2])

ValueError: operands could not be broadcast together with shapes (4,) (2,)
```

Broadcasting? We'll return to that [later](#).

Transposition

```
a = np.triu(np.ones((3, 3)), 1)  # see help(np.triu)
a
```

```
array([[0., 1., 1.],
       [0., 0., 1.],
       [0., 0., 0.]])
```

```
a.T
```

```
array([[0., 0., 0.],
       [1., 0., 0.],
       [1., 1., 0.]])
```

Remember, **the transposition is a view**.

The transpose returns a *view* of the original array:

```
a = np.arange(9).reshape(3, 3)
a.T[0, 2] = 999
a.T
```

```
array([[ 0,   3, 999],
       [ 1,   4,   7],
       [ 2,   5,   8]])
```

a

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [999,  7,  8]])
```

Linear algebra

The sub-module `numpy.linalg` implements basic linear algebra, such as solving linear systems, singular value decomposition, etc. However, it is not guaranteed to be compiled using efficient routines, and thus we recommend the use of `scipy.linalg`, as detailed in section [Linear algebra operations: scipy.linalg](#)

Exercise 16

- Look at the help for `np.allclose`. When might this be useful?
- Look at the help for `np.triu` and `np.tril`.

Basic reductions

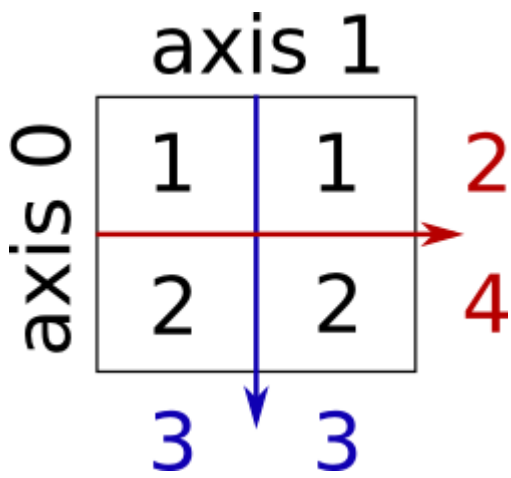
Computing sums

```
x = np.array([1, 2, 3, 4])
np.sum(x)
```

```
np.int64(10)
```

```
x.sum()
```

```
np.int64(10)
```



Sum by rows and by columns:

```
x = np.array([[1, 1], [2, 2]])
x
```

```
array([[1, 1],
       [2, 2]])
```

```
x.sum(axis=0)    # columns (first dimension)
```

```
array([3, 3])
```

```
x[:, 0].sum(), x[:, 1].sum()
```

```
(np.int64(3), np.int64(3))
```

```
x.sum(axis=1)    # rows (second dimension)
```

```
array([2, 4])
```

```
x[0, :].sum(), x[1, :].sum()
```

```
(np.int64(2), np.int64(4))
```

Here is the same idea in higher dimensions:

```
rng = np.random.default_rng(27446968)
x = rng.random((2, 2, 2))
x.sum(axis=2)[0, 1]
```

```
np.float64(0.7341517644004746)
```

```
x[0, 1, :].sum()
```

```
np.float64(0.7341517644004746)
```

Other reductions

These work the same way (and take `axis=`)

Extrema

```
x = np.array([1, 3, 2])
x.min()
```

```
np.int64(1)
```

```
x.max()
```

```
np.int64(3)
```

```
x.argmin() # index of minimum
```

```
np.int64(0)
```

```
x.argmax() # index of maximum
```

```
np.int64(1)
```

Logical operations

```
np.all([True, True, False])
```

```
np.False_
```

```
np.any([True, True, False])
```

```
np.True_
```

This can be used for array comparisons:

```
a = np.zeros((100, 100))  
np.any(a != 0)
```

```
np.False_
```

```
np.all(a == a)
```

```
np.True_
```

```
a = np.array([1, 2, 3, 2])  
b = np.array([2, 2, 3, 2])  
c = np.array([6, 4, 4, 5])  
((a <= b) & (b <= c)).all()
```

```
np.True_
```

Statistics:

```
x = np.array([1, 2, 3, 1])  
y = np.array([[1, 2, 3], [5, 6, 1]])  
x.mean()
```

```
np.float64(1.75)
```

```
np.median(x)
```

```
np.float64(1.5)
```

```
np.median(y, axis=-1) # last axis
```

```
array([2., 5.])
```

```
x.std() # full population standard dev.
```

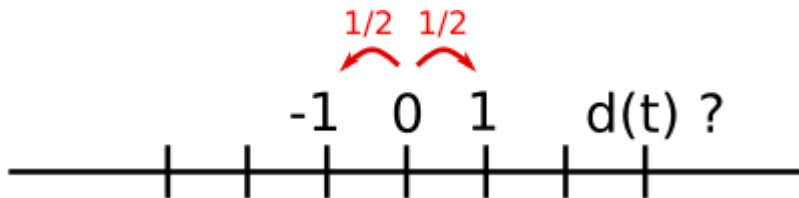
```
np.float64(0.82915619758885)
```

... and many more (best to learn as you go).

Exercise 17



Given there is a `sum`, what other function might you expect to see? What is the difference between `sum` and `cumsum`?



Let us consider a simple 1D random walk process: at each time step a walker jumps right or left with equal probability.

We are interested in finding the typical distance from the origin of a random walker after `t` left or right jumps? We are going to simulate many “walkers” to find this law, and we are going to do so using array computing tricks: we are going to create a 2D array with the “stories” (each walker has a story) in one direction, and the time in the other:

shuffled jumps

Position : cumulated jumps sum

		time →							
stories ↓	1	1	-1	1	-1	1			
	-1	1	1	-1	-1	1			

		time →							
stories ↓	1	2	1	2	1	2			
	-1	0	1	0	-1	0			

```
n_stories = 1000 # number of walkers
t_max = 200 # time during which we follow the walker
```

We randomly choose all the steps 1 or -1 of the walk:

```
t = np.arange(t_max)
rng = np.random.default_rng()
steps = 2 * rng.integers(0, 1 + 1, (n_stories, t_max)) - 1 # +1 because the
high value is exclusive
np.unique(steps) # Verification: all steps are 1 or -1
```

```
array([-1,  1])
```

We build the walks by summing steps along the time:

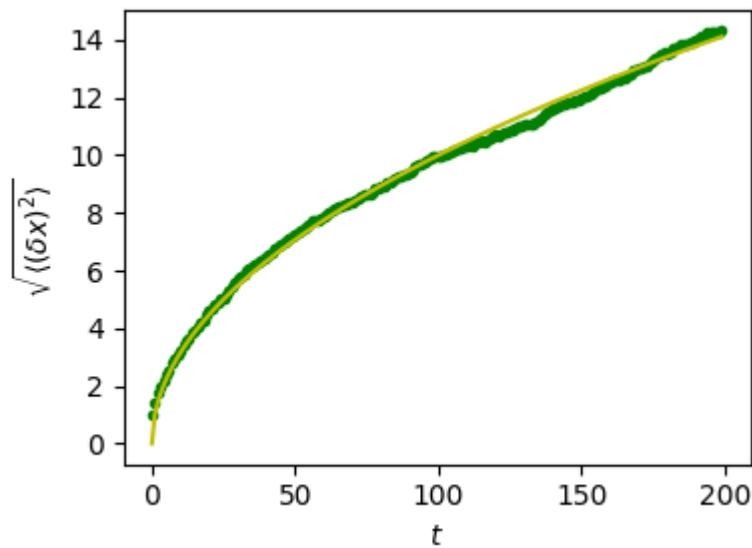
```
positions = np.cumsum(steps, axis=1) # axis = 1: dimension of time
sq_distance = positions**2
```

We get the mean in the axis of the stories:

```
mean_sq_distance = np.mean(sq_distance, axis=0)
```

Plot the results:

```
plt.figure(figsize=(4, 3))
plt.plot(t, np.sqrt(mean_sq_distance), 'g.', t, np.sqrt(t), 'y-')
plt.xlabel(r"$t$")
plt.ylabel(r"$\sqrt{\langle \Delta x \rangle^2}$")
plt.tight_layout() # provide sufficient space for labels
```



We find a well-known result in physics: the Root Mean Square (RMS) distance grows as the square root of the time!

Interim summary and exercises

Operation type	Numpy functions
arithmetic	<code>sum</code> , <code>prod</code> , <code>mean</code> , <code>std</code>
Extrema	<code>min</code> , <code>max</code>
logical	<code>all</code> , <code>any</code>

Also, recall the `axis` argument to select the dimension over which an operation will be applied:

```
arr = np.array([[99, 12], [11, 2]])
arr
```

```
array([[99, 12],
       [11,  2]])
```

```
# Without axis=, operation applied over whole (flatted, 1D) array.
np.min(arr)
```

```
np.int64(2)
```

```
# Operate along first axis (rows).  
np.min(arr, axis=0)
```

```
array([11,  2])
```

```
# Operate along second axis (columns).  
np.min(arr, axis=1)
```

```
array([12,  2])
```

Exercise 18



We load an array from a text file:

```
an_array = np.loadtxt('data/an_array.txt')
```

1. Verify if all elements in `an_array` are equal to 1:
2. Verify if any elements in an array are equal to 1
3. Compute mean and standard deviation.
4. Challenge: write a function `my_std` that computes the standard deviation of the elements in the array, where you are only allowed to use `np.sum` from Numpy in your function. Check your function returns a value close to that from `np.std` (use `np.allclose` for that check).

Solution to [Exercise 18](#)



```
# 1. Verify if all elements in `an_array` are equal to 1:
np.all(an_array == 1)

# 2. Verify if any elements in an array are equal to 1
np.any(an_array == 1)

# 3. Compute mean and standard deviation.
print('Mean', np.mean(an_array))
print('STD', np.std(an_array))
```

```
Mean 4.025
STD 2.1850343246731843
```

```
# 4. Challenge: write a function `my_std` that computes the standard
deviation
# of the elements in the array, where you are only allowed to use
`np.sum` from
# Numpy in your function.

def my_std(a):
    n = a.size
    m = np.sum(a) / n
    return np.sqrt(np.sum((a - m) ** 2) / n)

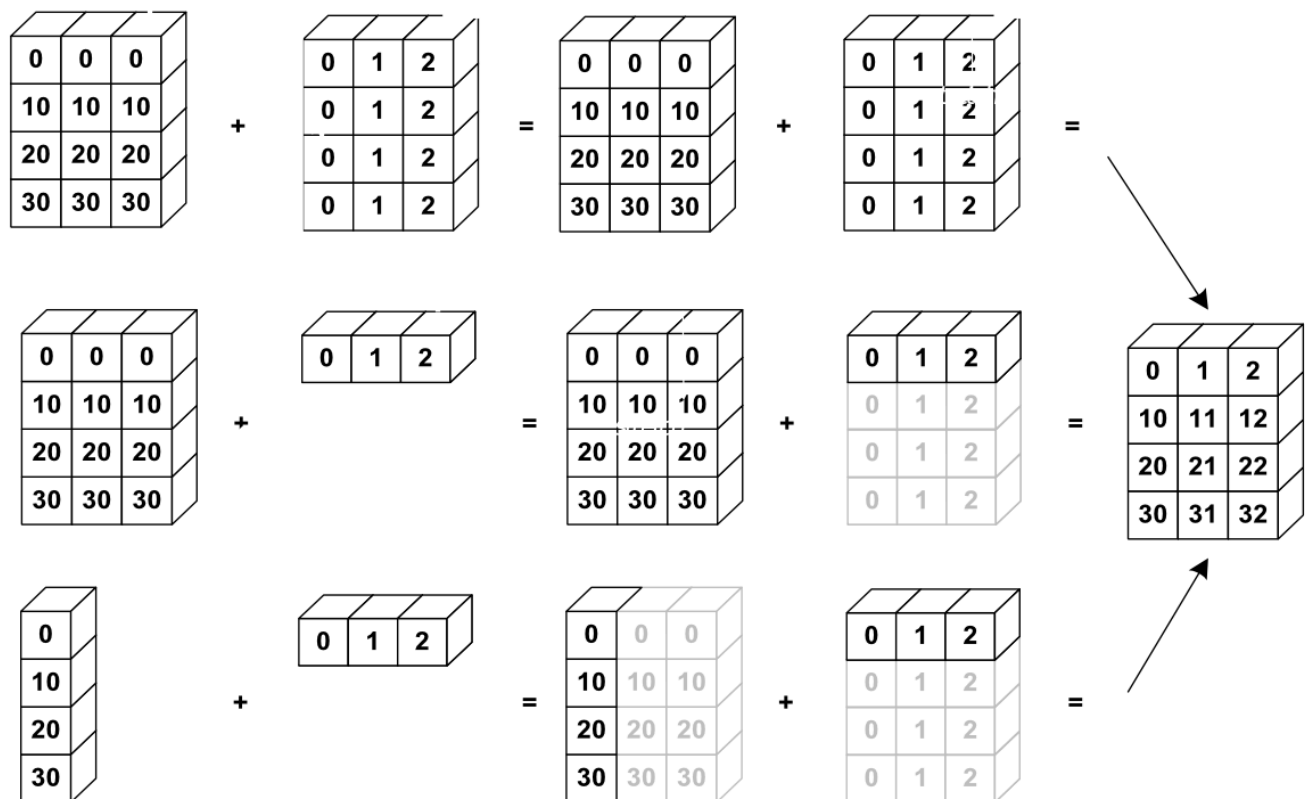
# Check we get the same answers from our function as for Numpy.
assert np.allclose(my_std(an_array), np.std(an_array))
assert np.allclose(my_std(an_array.ravel()), np.std(an_array))

rng = np.random.default_rng()
for i in range(10):
    another_array = rng.uniform(size=(10, 4))
    assert np.allclose(my_std(another_array), np.std(another_array))
```

Broadcasting

- Basic operations on `numpy` arrays (addition, etc.) are elementwise
- This works on arrays of the same size.
- **Nevertheless**, it's also possible to do operations on arrays of different sizes if *NumPy* can transform these arrays so that they all have the same size: this conversion is called **broadcasting**.

The image below gives an example of broadcasting:



Let's verify:

```
a = np.tile(np.arange(0, 40, 10), (3, 1)).T
a
```

```
array([[ 0,  0,  0],
       [10, 10, 10],
       [20, 20, 20],
       [30, 30, 30]])
```

```
b = np.array([0, 1, 2])
a + b
```

```
array([[ 0,  1,  2],
       [10, 11, 12],
       [20, 21, 22],
       [30, 31, 32]])
```

We have already used broadcasting without knowing it!:

```
a = np.ones((4, 5))
a[0] = 2 # we assign an array of dimension 0 to an array of dimension 1
a
```

```
array([[2., 2., 2., 2., 2.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

A useful trick:

```
a = np.arange(0, 40, 10)
a.shape
```

```
(4,)
```

```
a = a[:, np.newaxis] # adds a new axis -> 2D array
a.shape
```

```
(4, 1)
```

```
a
```

```
array([[ 0],
       [10],
       [20],
       [30]])
```

```
a + b
```

```
array([[ 0,  1,  2],
       [10, 11, 12],
       [20, 21, 22],
       [30, 31, 32]])
```

Note

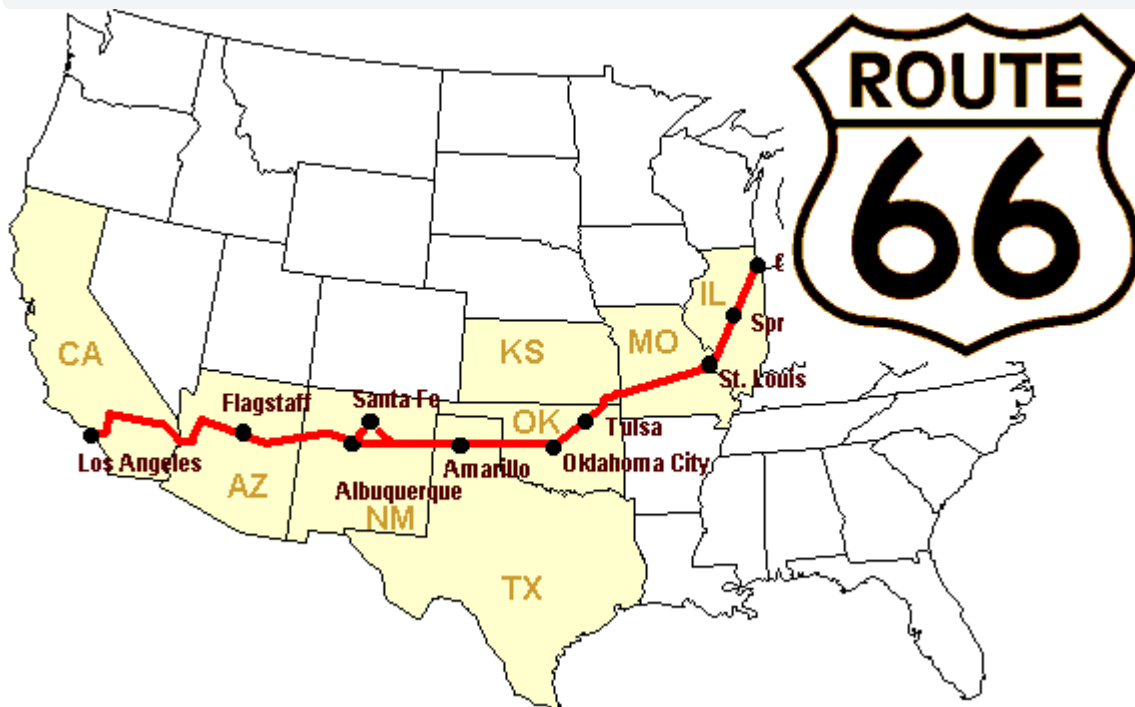
Broadcasting seems a bit magical, but it is actually quite natural to use it when we want to solve a problem whose output data is an array with more dimensions than input data.

Worked Example: Broadcasting

Let's construct an array of distances (in miles) between cities of Route 66: Chicago, Springfield, Saint-Louis, Tulsa, Oklahoma City, Amarillo, Santa Fe, Albuquerque, Flagstaff and Los Angeles.

```
mileposts = np.array([0, 198, 303, 736, 871, 1175, 1475, 1544, 1913, 2448])
distance_array = np.abs(mileposts - mileposts[:, np.newaxis])
distance_array
```

```
array([[ 0, 198, 303, 736, 871, 1175, 1475, 1544, 1913, 2448],
       [198,  0, 105, 538, 673, 977, 1277, 1346, 1715, 2250],
       [303, 105,  0, 433, 568, 872, 1172, 1241, 1610, 2145],
       [736, 538, 433,  0, 135, 439, 739, 808, 1177, 1712],
       [871, 673, 568, 135,  0, 304, 604, 673, 1042, 1577],
       [1175, 977, 872, 439, 304,  0, 300, 369, 738, 1273],
       [1475, 1277, 1172, 739, 604, 300,  0, 69, 438, 973],
       [1544, 1346, 1241, 808, 673, 369, 69,  0, 369, 904],
       [1913, 1715, 1610, 1177, 1042, 738, 438, 369,  0, 535],
       [2448, 2250, 2145, 1712, 1577, 1273, 973, 904, 535,  0]])
```



A lot of grid-based or network-based problems can also use broadcasting. For instance, if we want to compute the distance from the origin of points on a 5x5 grid, we can do

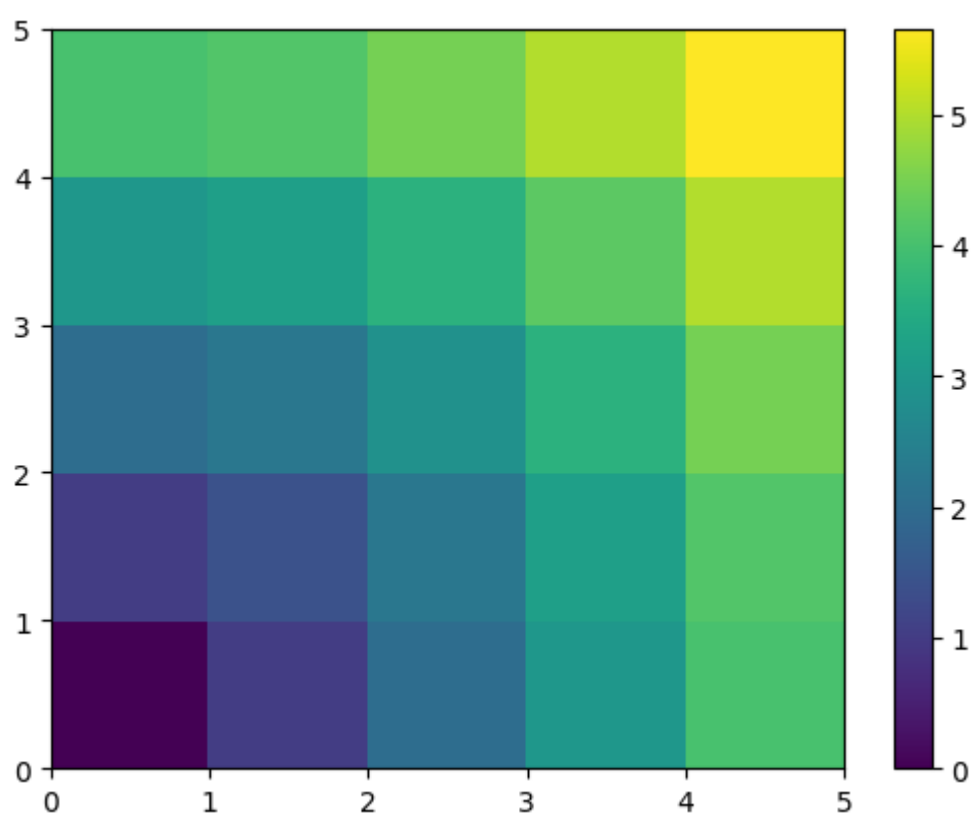
```
x, y = np.arange(5), np.arange(5)[:, np.newaxis]
distance = np.sqrt(x ** 2 + y ** 2)
distance
```

```
array([[0.      , 1.      , 2.      , 3.      , 4.      ],
       [1.41421356, 2.23606798, 3.16227766, 4.12310563],
       [2.23606798, 2.82842712, 3.60555128, 4.47213595],
       [3.16227766, 3.60555128, 4.24264069, 5.      ],
       [4.12310563, 4.47213595, 5.      , 5.65685425]])
```

Or in color:

```
plt.pcolor(distance)
plt.colorbar()
```

<matplotlib.colorbar.Colorbar at 0x7f29ea92bf20>



Remark : the `numpy.ogrid()` function allows to directly create vectors `x` and `y` of the previous example, with two “significant dimensions”:

```
x, y = np.ogrid[0:5, 0:5]
x, y
```

```
(array([[0],
       [1],
       [2],
       [3],
       [4]]),
 array([[0, 1, 2, 3, 4]]))
```



```
x.shape, y.shape
distance = np.sqrt(x ** 2 + y ** 2)
```

So, `np.ogrid` is very useful as soon as we have to handle computations on a grid. On the other hand, `np.mgrid` directly provides matrices full of indices for cases where we can't (or don't want to) benefit from broadcasting:

```
x, y = np.mgrid[0:4, 0:4]
x
```

```
array([[0, 0, 0, 0],
       [1, 1, 1, 1],
       [2, 2, 2, 2],
       [3, 3, 3, 3]])
```

```
y
```

```
array([[0, 1, 2, 3],
       [0, 1, 2, 3],
       [0, 1, 2, 3],
       [0, 1, 2, 3]])
```

See also

[Broadcasting](#): discussion of broadcasting in the [Advanced NumPy](#) chapter.

Array shape manipulation

Flattening

```
a = np.array([[1, 2, 3], [4, 5, 6]])
a.ravel()
```

```
array([1, 2, 3, 4, 5, 6])
```

```
a.T
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```

```
a.T.ravel()
```

```
array([1, 4, 2, 5, 3, 6])
```

Higher dimensions: last dimensions ravel out “first”.

Reshaping

The inverse operation to flattening:

```
a.shape
```

```
(2, 3)
```

```
b = a.ravel()
b = b.reshape((2, 3))
b
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

Or,

```
a.reshape((2, -1))    # unspecified (-1) value is inferred
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

Warning

`ndarray.reshape` **may** return a view (cf `help(np.reshape)`), or copy

For example, consider:

```
b[0, 0] = 99
a
```

```
array([[99,  2,  3],
       [ 4,  5,  6]])
```

Beware: reshape may also return a copy!:

```
a = np.zeros((3, 2))
b = a.T.reshape(3*2)
b[0] = 9
a
```

```
array([[0., 0.],
       [0., 0.],
       [0., 0.]])
```

To understand this you need to learn more about the memory layout of a NumPy array.

Adding a dimension

Indexing with the `np.newaxis` object allows us to add an axis to an array (you have seen this already above in the broadcasting section):

```
z = np.array([1, 2, 3])
z
```

```
array([1, 2, 3])
```

```
z[:, np.newaxis]
```

```
array([[1],
       [2],
       [3]])
```

```
z[np.newaxis, :]
```

```
array([[1, 2, 3]])
```

Dimension shuffling

```
a = np.arange(4*3*2).reshape(4, 3, 2)
a.shape
```

```
(4, 3, 2)
```

```
a[0, 2, 1]
```

```
np.int64(5)
```

```
b = a.transpose(1, 2, 0)
b.shape
```

```
(3, 2, 4)
```

```
b[2, 1, 0]
```

```
np.int64(5)
```

Also creates a view:

```
b[2, 1, 0] = -1
a[0, 2, 1]
```

```
np.int64(-1)
```

Resizing

Size of an array can be changed with `ndarray.resize`:

```
a = np.arange(4)
a.resize((8,))
a
```

```
array([0, 1, 2, 3, 0, 0, 0, 0])
```

However, it must not be referred to somewhere else:

```
b = a
a.resize((4,))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[90], line 2
      1 b = a
----> 2 a.resize((4,))

ValueError: cannot resize an array that references or is referenced
by another array in this way.
Use the np.resize function or refcheck=False
```

Exercise 19

- Look at the docstring for `reshape`, especially the notes section which has some more information about copies and views.
- Use `flatten` as an alternative to `ravel`. What is the difference? (Hint: check which one returns a view and which a copy)
- Experiment with `transpose` for dimension shuffling.

Sorting data

Sorting along an axis:

```
a = np.array([[4, 3, 5], [1, 2, 1]])
b = np.sort(a, axis=1)
b
```

```
array([[3, 4, 5],
       [1, 1, 2]])
```

Note

Sorts each row separately!

In-place sort:

```
a.sort(axis=1)
a
```

```
array([[3, 4, 5],
       [1, 1, 2]])
```

Sorting with fancy indexing:

```
a = np.array([4, 3, 1, 2])
j = np.argsort(a)
j
```

```
array([2, 3, 1, 0])
```

```
a[j]
```

```
array([1, 2, 3, 4])
```

Finding minima and maxima:

```
a = np.array([4, 3, 1, 2])
j_max = np.argmax(a)
j_min = np.argmin(a)
j_max, j_min
```

```
(np.int64(0), np.int64(2))
```

Exercise 20

- Try both in-place and out-of-place sorting.
- Try creating arrays with different dtypes and sorting them.
- Use `all` or `array_equal` to check the results.
- Look at `np.random.shuffle` for a way to create sortable input quicker.
- Combine `ravel`, `sort` and `reshape`.
- Look at the `axis` keyword for `sort` and rewrite the previous exercise.

Summary

What do you need to know to get started?

- Know how to create arrays : `array`, `arange`, `ones`, `zeros`.
- Know the shape of the array with `array.shape`, then use slicing to obtain different views of the array: `array[:, :2]`, etc. Adjust the shape of the array using `reshape` or flatten it with `ravel`.
- Obtain a subset of the elements of an array and/or modify their values with masks, with e.g.:

```
a[a < 0] = 0
```

- Know miscellaneous operations on arrays, such as finding the mean or max (`array.max()`, `array.mean()`). No need to retain everything, but have the reflex to search in the documentation (online docs, `help()`)!!
- For advanced use: master the indexing with arrays of integers, as well as broadcasting. Know more NumPy functions to handle various array operations.

Quick read

If you want to do a first quick pass through the Scientific Python Lectures to learn the ecosystem, you can directly skip to the next chapter: [Matplotlib: plotting](#).

The remainder of this chapter is not necessary to follow the rest of the intro part. But be sure to come back and finish this chapter, as well as to do some more [exercises](#).

More elaborate arrays

Contents

- More data types
- Structured data types
- `maskedarray`: dealing with (propagation of) missing data

► Show code cell source

More data types

Casting

“Bigger” type wins in mixed-type operations:

```
np.array([1, 2, 3]) + 1.5
```

```
array([2.5, 3.5, 4.5])
```

Assignment never changes the type!

```
a = np.array([1, 2, 3])  
a.dtype
```

```
dtype('int64')
```

```
a[0] = 1.9      # <-- float is truncated to integer  
a
```

```
array([1, 2, 3])
```


Forced casts:

```
a = np.array([1.7, 1.2, 1.6])
b = a.astype(int) # <-- truncates to integer
b
```

```
array([1, 1, 1])
```

Rounding:

```
a = np.array([1.2, 1.5, 1.6, 2.5, 3.5, 4.5])
b = np.around(a)
b # still floating-point
```

```
array([1., 2., 2., 2., 4., 4.])
```

```
c = np.around(a).astype(int)
c
```

```
array([1, 2, 2, 2, 4, 4])
```

Different data type sizes

Integers (signed):

Class	Bits
<code>int8</code>	8 bits
<code>int16</code>	16b its
<code>int32</code>	32 bits (same as <code>int</code> on 32-bit platform)
<code>int64</code>	64 bits (same as <code>int</code> on 64-bit platform)

```
np.array([1], dtype=int).dtype
```

```
dtype('int64')
```

```
np.iinfo(np.int32).max, 2**31 - 1
```

```
(2147483647, 2147483647)
```

Unsigned integers:

Class	Bits
<code>uint8</code>	8 bits
<code>uint16</code>	16 bits
<code>uint32</code>	32 bits
<code>uint64</code>	64 bits

```
np.iinfo(np.uint32).max, 2**32 - 1
```

```
(4294967295, 4294967295)
```

Floating-point numbers:

Data Type	Size (bits)
<code>float16</code>	16 bits
<code>float32</code>	32 bits
<code>float64</code>	64 bits (same as <code>float</code>)
<code>float96</code>	96 bits, platform-dependent (same as <code>np.longdouble</code>)
<code>float128</code>	128 bits, platform-dependent (same as <code>np.longdouble</code>)

```
np.finfo(np.float32).eps
```

```
np.float32(1.1920929e-07)
```

```
np.finfo(np.float64).eps
```

```
np.float64(2.220446049250313e-16)
```

```
np.float32(1e-8) + np.float32(1) == 1
```

```
np.True_
```

```
np.float64(1e-8) + np.float64(1) == 1
```

```
np.False_
```

Complex floating-point numbers:

Data Type	Size (bits)
<code>complex64</code>	two 32-bit floats
<code>complex128</code>	two 64-bit floats
<code>complex192</code>	two 96-bit floats, platform-dependent
<code>complex256</code>	two 128-bit floats, platform-dependent

Smaller data types

If you don't know you need special data types, then you probably don't.

Comparison on using `float32` instead of `float64`:

- Half the size in memory and on disk
- Half the memory bandwidth required (may be a bit faster in some operations)

```
In [1]: a = np.zeros((int(1e6),), dtype=np.float64)
```

```
In [2]: b = np.zeros((int(1e6),), dtype=np.float32)
```

```
In [3]: %timeit a*a  
1000 loops, best of 3: 1.78 ms per loop
```

```
In [4]: %timeit b*b  
1000 loops, best of 3: 1.07 ms per loop
```

- But: bigger rounding errors — sometimes in surprising places (i.e., don't use them unless you really need them)

Structured data types

Data Type	Description
<code>sensor_code</code>	4-character string
<code>position</code>	float
<code>value</code>	float

```
samples = np.zeros((6,), dtype=[('sensor_code', 'S4'),  
                                ('position', float), ('value', float)])  
samples.ndim
```

```
1
```

```
samples.shape
```

```
(6,)
```

```
samples.dtype.names
```

```
('sensor_code', 'position', 'value')
```

```
samples[:] = [('ALFA', 1, 0.37), ('BETA', 1, 0.11), ('TAU', 1, 0.13),  
              ('ALFA', 1.5, 0.37), ('ALFA', 3, 0.11), ('TAU', 1.2, 0.13)]  
samples
```

```
array([(b'ALFA', 1. , 0.37), (b'BETA', 1. , 0.11), (b'TAU', 1. , 0.13),  
       (b'ALFA', 1.5, 0.37), (b'ALFA', 3. , 0.11), (b'TAU', 1.2, 0.13)],  
      dtype=[('sensor_code', 'S4'), ('position', '<f8'), ('value', '<f8')])
```

Field access works by indexing with field names:

```
samples['sensor_code']
```

```
array([b'ALFA', b'BETA', b'TAU', b'ALFA', b'ALFA', b'TAU'], dtype='|S4')
```

```
samples['value']
```

```
array([0.37, 0.11, 0.13, 0.37, 0.11, 0.13])
```

```
samples[0]
```

```
np.void((b'ALFA', 1.0, 0.37), dtype=[('sensor_code', 'S4'), ('position',  
'<f8'), ('value', '<f8')])
```

```
samples[0]['sensor_code'] = 'TAU'  
samples[0]
```

```
np.void((b'TAU', 1.0, 0.37), dtype=[('sensor_code', 'S4'), ('position', '<f8'),  
( 'value', '<f8')])
```

Multiple fields at once:

```
samples[['position', 'value']]
```

```
array([(1. , 0.37), (1. , 0.11), (1. , 0.13), (1.5, 0.37), (3. , 0.11),
      (1.2, 0.13)],
      dtype={'names': ['position', 'value'], 'formats': ['<f8', '<f8'],
            'offsets': [4, 12], 'itemsize': 20})
```

Fancy indexing works, as usual:

```
samples[samples['sensor_code'] == b'ALFA']
```

```
array([(b'ALFA', 1.5, 0.37), (b'ALFA', 3. , 0.11)],
      dtype=[('sensor_code', 'S4'), ('position', '<f8'), ('value', '<f8')])
```

Note

There are a bunch of other syntaxes for constructing structured arrays, see [here](#) and [here](#).

maskedarray: dealing with (propagation of) missing data

- For floats one could use NaN's, but masks work for all types:

```
x = np.ma.array([1, 2, 3, 4], mask=[0, 1, 0, 1])
x
```

```
masked_array(data=[1, --, 3, --],
             mask=[False, True, False, True],
             fill_value=999999)
```

```
y = np.ma.array([1, 2, 3, 4], mask=[0, 1, 1, 1])
x + y
```

```
masked_array(data=[2, --, --, --],
             mask=[False, True, True, True],
             fill_value=999999)
```

- Masking versions of common functions:

```
np.ma.sqrt([1, -1, 2, -2])
```

```
masked_array(data=[1.0, --, 1.4142135623730951, --],  
             mask=[False, True, False, True],  
             fill_value=1e+20)
```

Note

There are other useful [array siblings](#)

While it is off topic in a chapter on NumPy, let's take a moment to recall good coding practice, which really do pay off in the long run:

Good practices

- Explicit variable names (no need of a comment to explain what is in the variable)
- Style: spaces after commas, around `=`, etc.

A certain number of rules for writing “beautiful” code (and, more importantly, using the same conventions as everybody else!) are given in the [Style Guide for Python Code](#) and the [Docstring Conventions](#) page (to manage help strings).

- Except some rare cases, variable names and comments in English.

Advanced operations

Contents

- Polynomials
- Loading data files

Polynomials

NumPy also contains polynomials in different bases:

For example, $3x^2 + 2x - 1$:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
p = np.poly1d([3, 2, -1])
p(0)
```

```
np.int64(-1)
```

```
p.roots
```

```
array([-1.          ,  0.33333333])
```

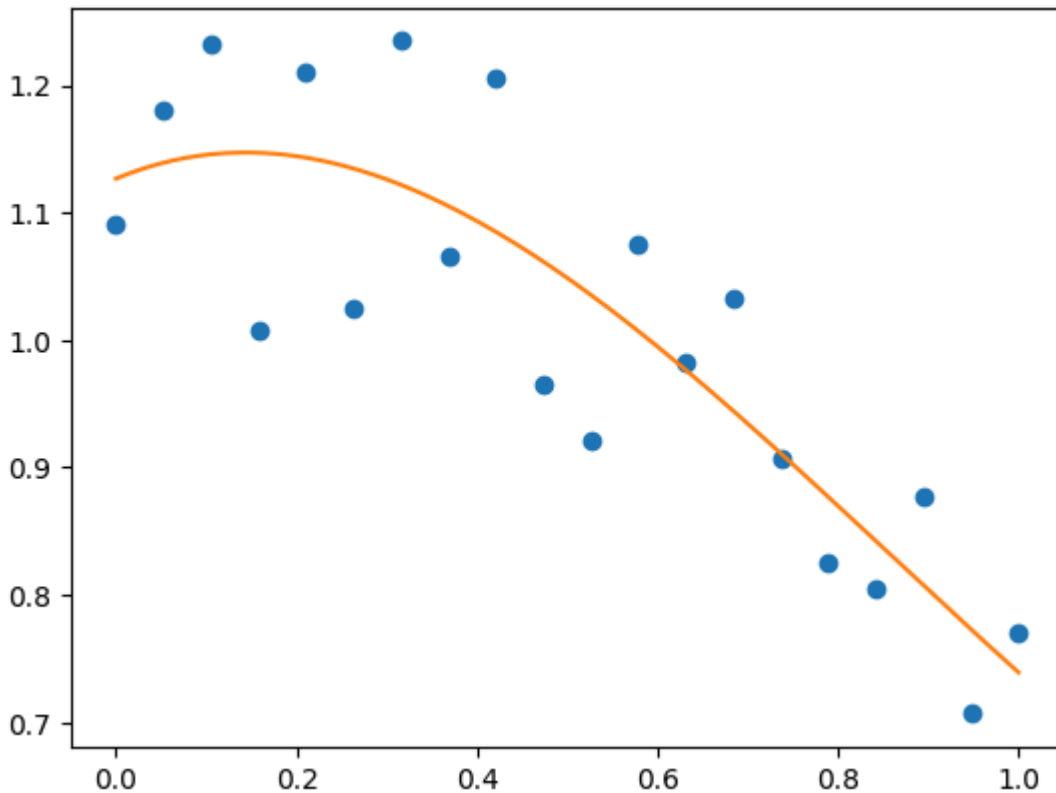
```
p.order
```

```
2
```



```
x = np.linspace(0, 1, 20)
rng = np.random.default_rng()
y = np.cos(x) + 0.3*rng.random(20)
p = np.poly1d(np.polyfit(x, y, 3))

t = np.linspace(0, 1, 200) # use a larger number of points for smoother
plotting
plt.plot(x, y, 'o', t, p(t), '-');
```



See <https://numpy.org/doc/stable/reference/routines.polynomials.poly1d.html> for more.

More polynomials (with more bases)

NumPy also has a more sophisticated polynomial interface, which supports e.g. the Chebyshev basis.

$3x^2 + 2x - 1$:

```
p = np.polynomial.Polynomial([-1, 2, 3]) # coefs in different order!
p(0)
```

```
np.float64(-1.0)
```

```
p.roots()
```

```
array([-1.          ,  0.33333333])
```

```
p.degree() # In general polynomials do not always expose 'order'
```

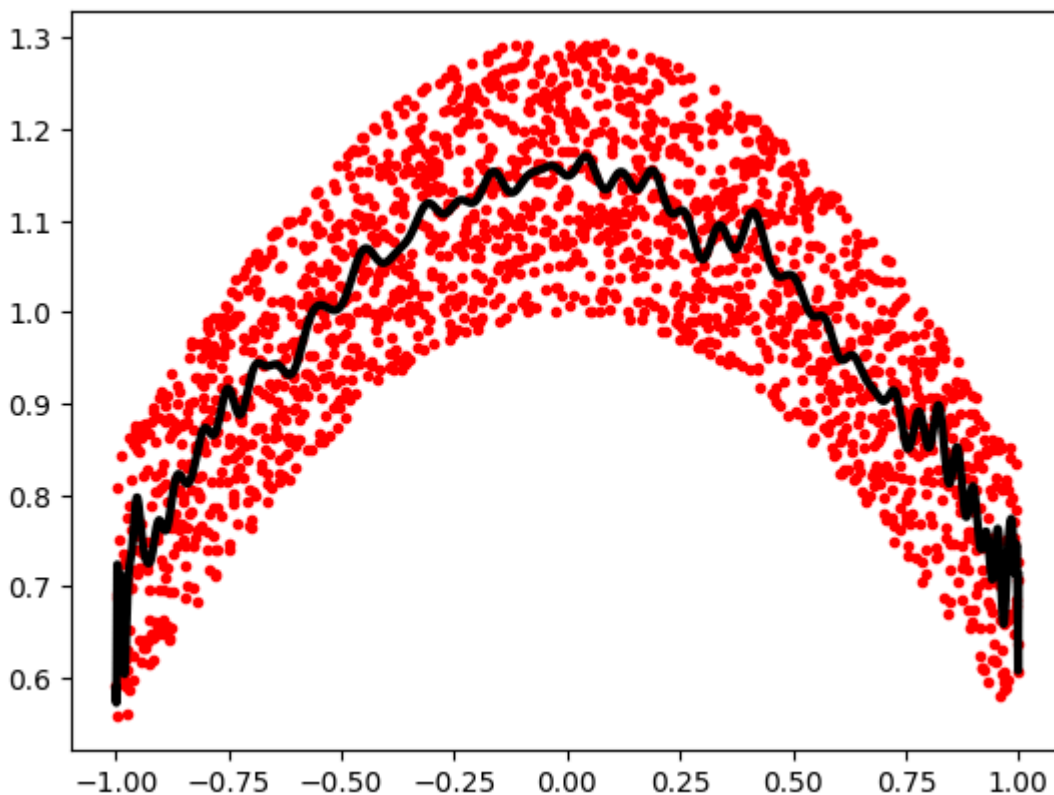
2

Example using polynomials in Chebyshev basis, for polynomials in range `[-1, 1]`:

```
x = np.linspace(-1, 1, 2000)
rng = np.random.default_rng()
y = np.cos(x) + 0.3*rng.random(2000)
p = np.polynomial.Chebyshev.fit(x, y, 90)
```

```
plt.plot(x, y, 'r.')
plt.plot(x, p(x), 'k-', lw=3)
```

```
[<matplotlib.lines.Line2D at 0x7f000ce8f3b0>]
```



The Chebyshev polynomials have some advantages in interpolation.

Loading data files

Text files

Example:  `populations.txt`.

```
data = np.loadtxt('data/populations.txt')
data
```

```
array([[ 1900., 30000.,  4000., 48300.],
       [ 1901., 47200.,  6100., 48200.],
       [ 1902., 70200.,  9800., 41500.],
       [ 1903., 77400., 35200., 38200.],
       [ 1904., 36300., 59400., 40600.],
       [ 1905., 20600., 41700., 39800.],
       [ 1906., 18100., 19000., 38600.],
       [ 1907., 21400., 13000., 42300.],
       [ 1908., 22000.,  8300., 44500.],
       [ 1909., 25400.,  9100., 42100.],
       [ 1910., 27100.,  7400., 46000.],
       [ 1911., 40300.,  8000., 46800.],
       [ 1912., 57000., 12300., 43800.],
       [ 1913., 76600., 19500., 40900.],
       [ 1914., 52300., 45700., 39400.],
       [ 1915., 19500., 51100., 39000.],
       [ 1916., 11200., 29700., 36700.],
       [ 1917.,  7600., 15800., 41800.],
       [ 1918., 14600.,  9700., 43300.],
       [ 1919., 16200., 10100., 41300.],
       [ 1920., 24700.,  8600., 47300.]])
```

```
np.savetxt('pop2.txt', data)
data2 = np.loadtxt('pop2.txt')
```

Note

If you have a complicated text file, what you can try are:

- `np.genfromtxt`
- Using Python's I/O functions and e.g. regexps for parsing (Python is quite well suited for this)

Reminder: Navigating the filesystem with Jupyter and IPython

Show current directory:

```
pwd
```

```
'/home/runner/work/scientific-python-lectures/scientific-python-lectures/intro/numpy'
```

Change to `data` subdirectory:

```
cd data
```

```
/home/runner/work/scientific-python-lectures/scientific-python-lectures/data
```

Show filesystem listing for current directory:

```
ls
```

```
LICENCE.txt      moonlanding.png  test.png         women_percentage.txt
MV_HFV_012.jpg*  organisms.txt    test.wav
an_array.txt     populations.txt  waveform_1.npy
elephant.png     species.txt      waveform_2.npy
```

Change back to containing directory.

```
cd ..
```

```
/home/runner/work/scientific-python-lectures/scientific-python-lectures
```

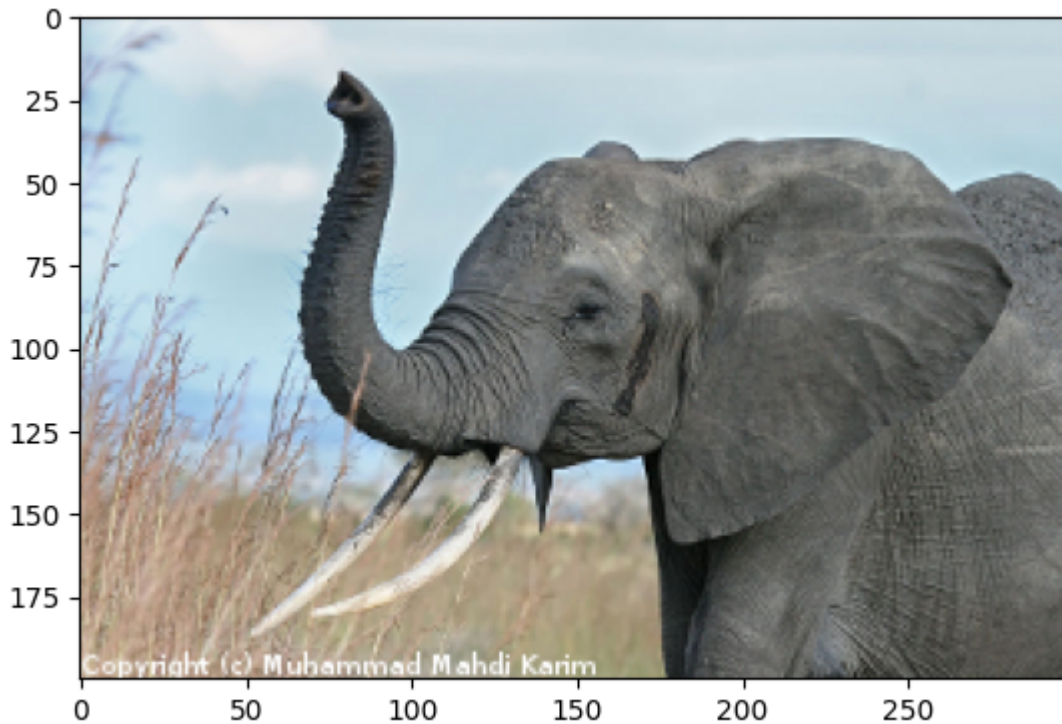
Images

Using Matplotlib:

```
img = plt.imread('data/elephant.png')
img.shape, img.dtype
```

```
((200, 300, 3), dtype('float32'))
```

```
# Plot and save the original figure  
plt.imshow(img)  
plt.savefig('plot.png')
```

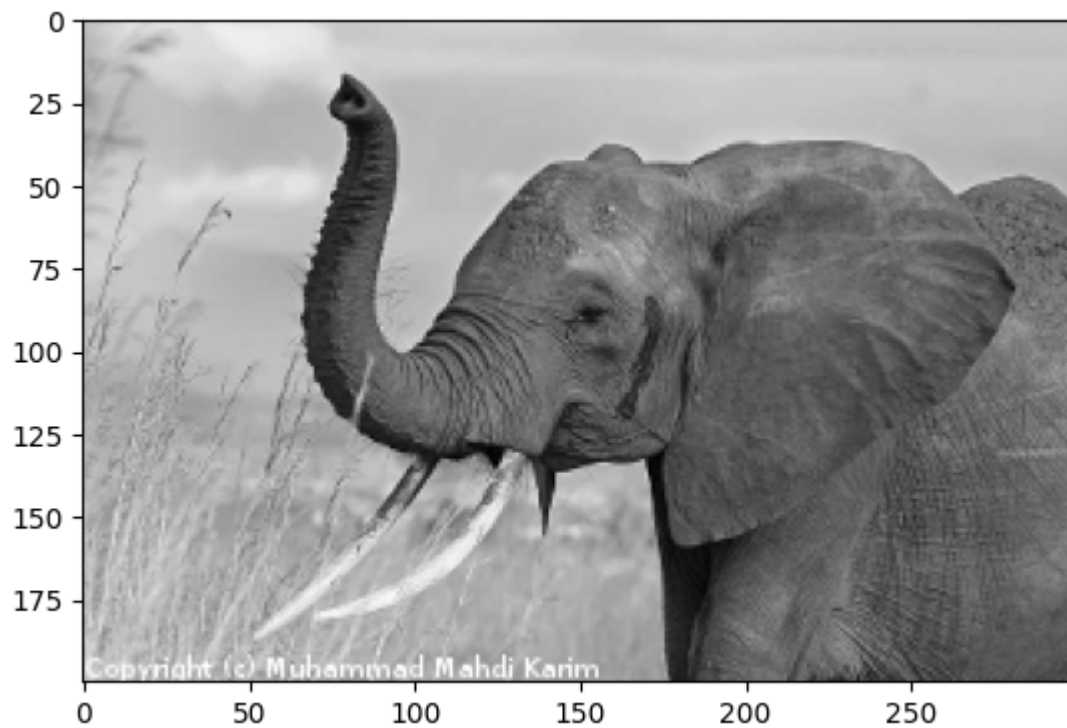


```
# Plot and save the red channel of the image.  
plt.imshow(img[:, :, 0], cmap=plt.cm.gray)
```

This saved only one channel (of RGB):

```
plt.imshow(plt.imread('red_elephant.png'))
```

```
<matplotlib.image.AxesImage at 0x7f0012714cb0>
```

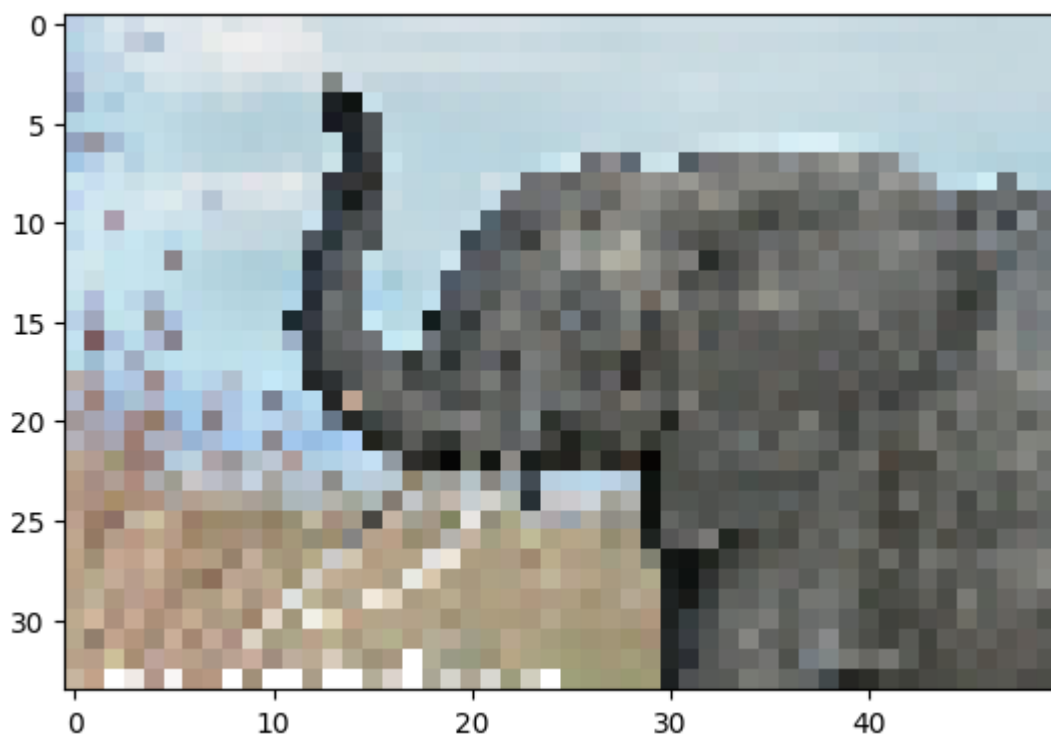


Other libraries:

```
import imageio.v3 as iio

# Lower resolution (every sixth pixel in each dimension).
iio.imwrite('tiny_elephant.png', (img[:,::6,::6] * 255).astype(np.uint8))
plt.imshow(plt.imread('tiny_elephant.png'), interpolation='nearest')
```

<matplotlib.image.AxesImage at 0x7f000ceb9910>



NumPy's own format

NumPy has its own binary format, not portable but with efficient I/O:

```
data = np.ones((3, 3))
np.save('pop.npy', data)
data3 = np.load('pop.npy')
```

Well-known (& more obscure) file formats

- HDF5: [h5py](#), [PyTables](#)
- NetCDF: `scipy.io.netcdf_file`, [netcdf4-python](#), ...
- Matlab: `scipy.io.loadmat`, `scipy.io.savemat`
- MatrixMarket: `scipy.io.mmread`, `scipy.io.mmwrite`
- IDL: `scipy.io.readsav`

... if somebody uses it, there's probably also a Python library for it.

Exercise 21



Write code that loads data from  `populations.txt` : and drops the last column and the first 5 rows. Save the smaller dataset to `pop2.txt` .

Solution to [Exercise 21](#)



```
data = np.loadtxt("data/populations.txt")
reduced_data = data[5:, :-1]
np.savetxt("pop2.txt", reduced_data)
```

NumPy internals

If you are interested in the NumPy internals, there is a good discussion in [Advanced NumPy](#).

Some exercises

Contents

- Array manipulations
- Picture manipulation: Framing a Face
- Data statistics
- Crude integral approximations
- Mandelbrot set
- Markov chain

```
import numpy as np
import matplotlib.pyplot as plt
```


Array manipulations

Exercise 22



Form the 2-D array (without typing it in explicitly)

```
[[1, 6, 11],  
 [2, 7, 12],  
 [3, 8, 13],  
 [4, 9, 14],  
 [5, 10, 15]]
```

and generate a new array containing its 2nd and 4th rows.

Divide each column of the array

```
import numpy as np  
a = np.arange(25).reshape(5, 5)
```

elementwise with the array `b = np.array([1., 5, 10, 15, 20])`. (Hint: `np.newaxis`).

Harder one, random numbers

Generate a 10 x 3 array of random numbers (in range [0,1]). For each row, pick the number closest to 0.5.

- Use `abs` and `argmin` to find the column `j` closest for each row.
- Use fancy indexing to extract the numbers. (Hint: `a[i, j]` – the array `i` must contain the row numbers corresponding to stuff in `j`.)

Solution to [Exercise 22](#)



```
import numpy as np
from numpy import newaxis

# Part 1.

a = np.arange(1, 16).reshape(3, -1).T
print(a)
```

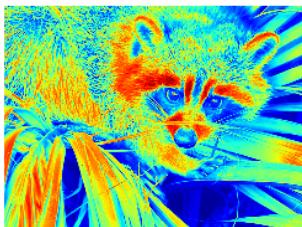
```
[[ 1  6 11]
 [ 2  7 12]
 [ 3  8 13]
 [ 4  9 14]
 [ 5 10 15]]
```

Picture manipulation: Framing a Face

Let's do some manipulations on NumPy arrays by starting with an image of a raccoon. `scipy` provides a 2D array of this image with the `scipy.datasets.face` function:

```
import scipy as sp
face = sp.datasets.face(gray=True) # 2D grayscale image
```

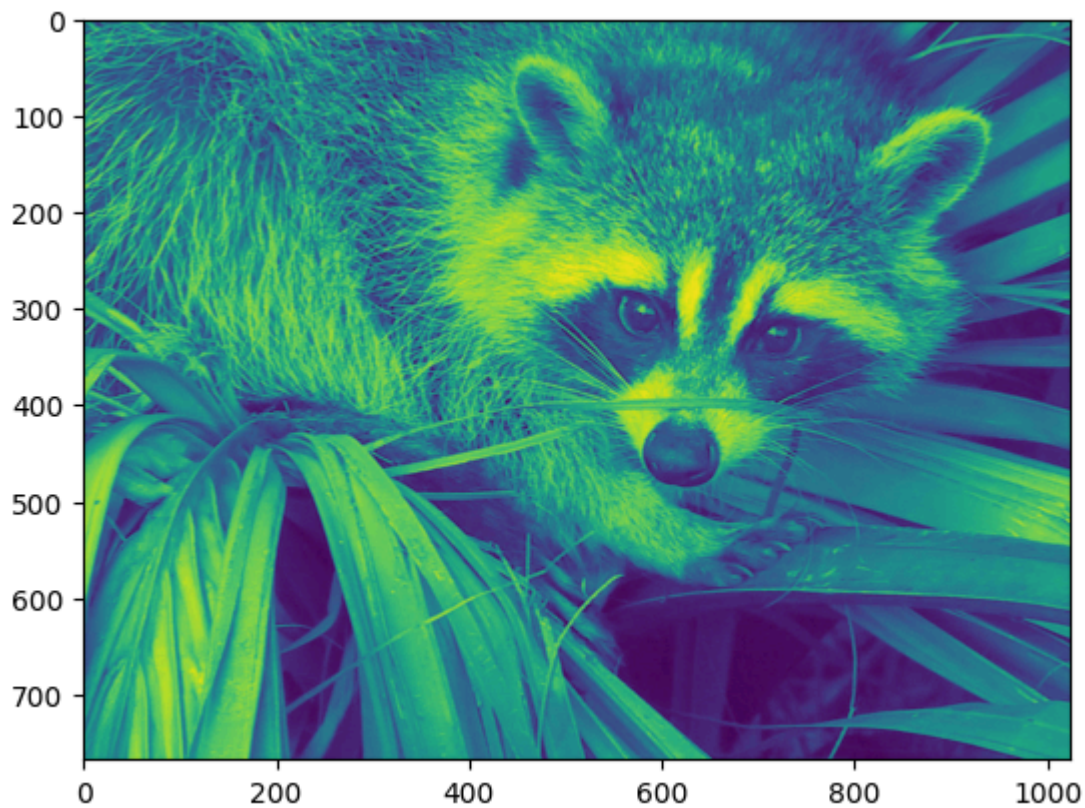
Here are a few images we will be able to obtain with our manipulations: use different colormaps, crop the image, change some parts of the image.



Let's use the `imshow` function of matplotlib to display the image.

```
import matplotlib.pyplot as plt
face = sp.datasets.face(gray=True)
plt.imshow(face)
```

```
<matplotlib.image.AxesImage at 0x7f20d14e3ce0>
```



The face is displayed in false colors. A colormap must be specified for it to be displayed in grey.

```
plt.imshow(face, cmap=plt.cm.gray)
```

```
<matplotlib.image.AxesImage at 0x7f20d0edad50>
```



Narrow centering

Create an array of the image with a narrower centering; remove 100 pixels from all the borders of the image. To check the result, display this new array with `imshow`.

```
crop_face = face[100:-100, 100:-100]
```

Frame face

We will now frame the face with a black locket. For this, we need to create a mask corresponding to the pixels we want to be black. The center of the face is around (660, 330), so we defined the mask by this condition $(y-300)^2$

- $(x-660)^2$

```
sy, sx = face.shape
y, x = np.ogrid[0:sy, 0:sx] # x and y indices of pixels
y.shape, x.shape
```

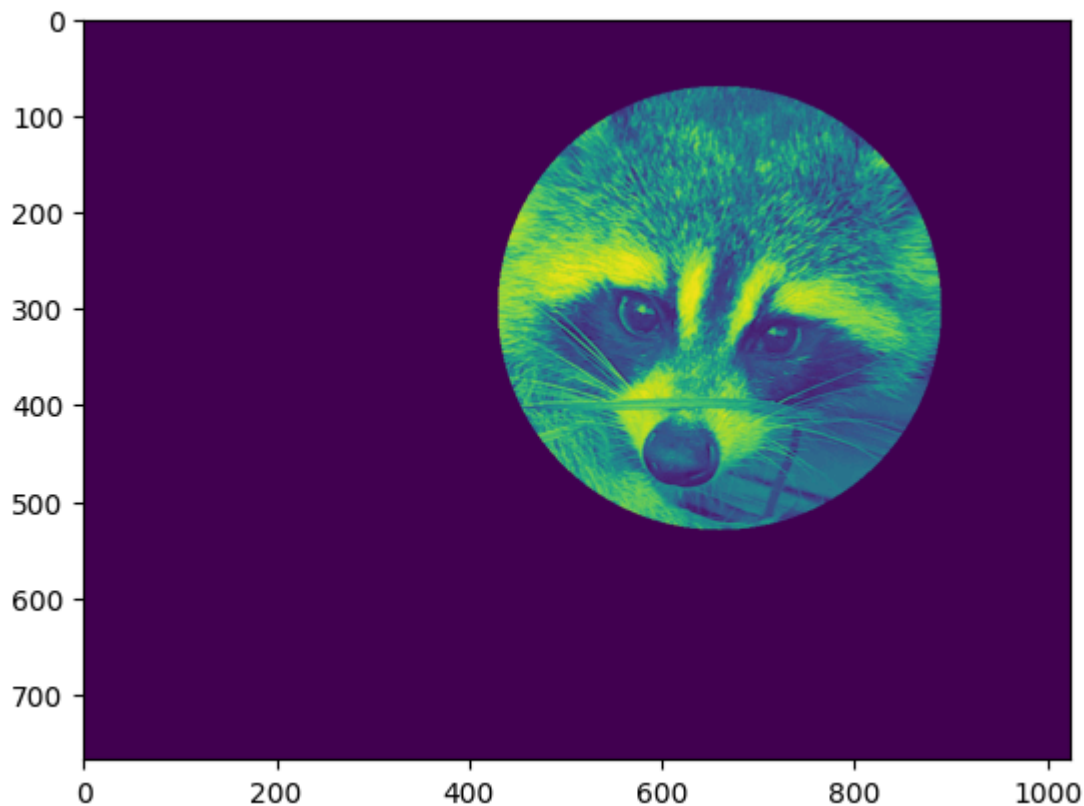
```
((768, 1), (1, 1024))
```

```
centerx, centery = (660, 300) # center of the image
mask = ((y - centery)**2 + (x - centerx)**2) > 230**2 # circle
```

then we assign the value 0 to the pixels of the image corresponding to the mask. The syntax is extremely simple and intuitive:

```
face[mask] = 0
plt.imshow(face)
```

```
<matplotlib.image.AxesImage at 0x7f20ccd30380>
```



Follow-up:

- copy all instructions of this exercise in a script called : `face_locket.py` then execute this script in IPython with `%run face_locket.py`.
- Change the circle to an ellipsoid.

Data statistics

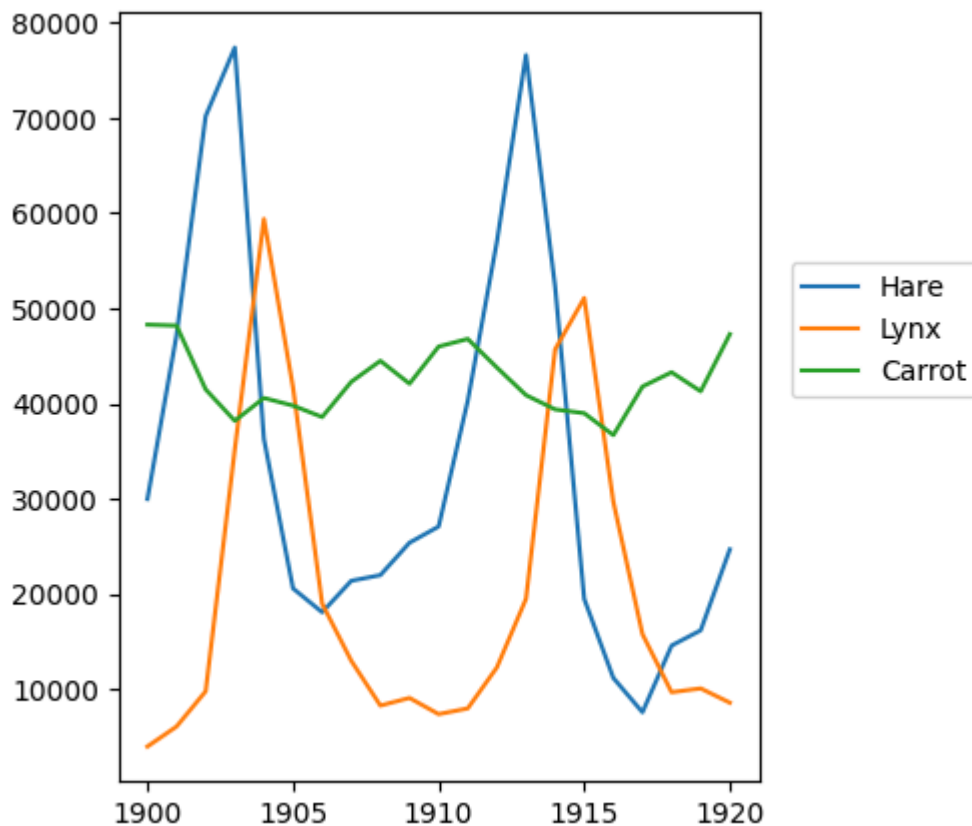
The data in [↓ populations.txt](#) describes the populations of hares and lynxes (and carrots) in northern Canada during 20 years:

```
data = np.loadtxt('data/populations.txt')
year, hares, lynxes, carrots = data.T # trick: columns to variables
```

```
import matplotlib.pyplot as plt

plt.axes([0.2, 0.1, 0.5, 0.8])
plt.plot(year, hares, year, lynxes, year, carrots)
plt.legend(['Hare', 'Lynx', 'Carrot'], loc=(1.05, 0.5))
```

```
<matplotlib.legend.Legend at 0x7f20d0f40a10>
```



Exercise 23

Compute and print, based on the data in `populations.txt` ...

1. The mean and std of the populations of each species for the years in the period.
2. Which year each species had the largest population.
3. Which species has the largest population for each year. (Hint: `argsort` & fancy indexing of `np.array(['H', 'L', 'C'])`)
4. Which years any of the populations is above 50000. (Hint: comparisons and `np.any`)
5. The top 2 years for each species when they had the lowest populations. (Hint: `argsort`, fancy indexing)
6. Compare (plot) the change in hare population (see `help(np.gradient)`) and the number of lynxes. Check correlation (see `help(np.corrcoef)`).

... all without for-loops.

Solution to [Exercise 23](#)

```
import numpy as np

data = np.loadtxt("data/populations.txt")
year, hares, lynxes, carrots = data.T
populations = data[:, 1:]

print("      Hares, Lynxes, Carrots")
print("Mean:", populations.mean(axis=0))
print("Std:", populations.std(axis=0))

j_max_years = np.argmax(populations, axis=0)
print("Max. year:", year[j_max_years])

max_species = np.argmax(populations, axis=1)
species = np.array(["Hare", "Lynx", "Carrot"])
print("Max species:")
print(year)
print(species[max_species])

above_50000 = np.any(populations > 50000, axis=1)
print("Any above 50000:", year[above_50000])

j_top_2 = np.argsort(populations, axis=0)[:2]
print("Top 2 years with lowest populations for each:")
print(year[j_top_2])

hare_grad = np.gradient(hares, 1.0)
print("diff(Hares) vs. Lynxes correlation", np.corrcoef(hare_grad,
lynxes)[0, 1])

import matplotlib.pyplot as plt

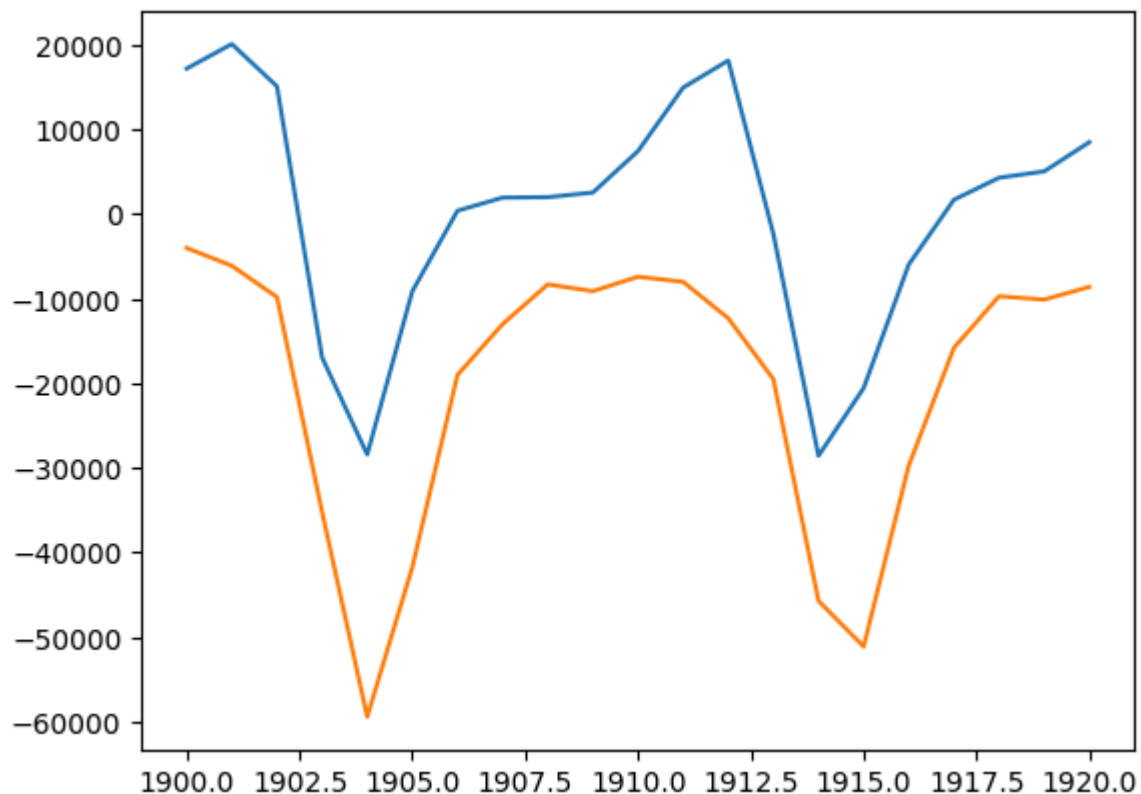
plt.plot(year, hare_grad, year, -lynxes)
plt.savefig("plot.png")
```



```

Hares, Lynxes, Carrots
Mean: [34080.95238095 20166.66666667 42400.          ]
Std: [20897.90645809 16254.59153691 3322.50622558]
Max. year: [1903. 1904. 1900.]
Max species:
[1900. 1901. 1902. 1903. 1904. 1905. 1906. 1907. 1908. 1909. 1910. 1911.
 1912. 1913. 1914. 1915. 1916. 1917. 1918. 1919. 1920.]
['Carrot' 'Carrot' 'Hare' 'Hare' 'Lynx' 'Lynx' 'Carrot' 'Carrot'
'Carrot'
 'Carrot' 'Carrot' 'Carrot' 'Hare' 'Hare' 'Hare' 'Lynx' 'Carrot'
'Carrot'
 'Carrot' 'Carrot' 'Carrot']
Any above 50000: [1902. 1903. 1904. 1912. 1913. 1914. 1915.]
Top 2 years with lowest populations for each:
[[1917. 1900. 1916.]
 [1916. 1901. 1903.]]
diff(Hares) vs. Lynxes correlation -0.9179248480315341

```



Crude integral approximations

Exercise 24

Write a function `f(a, b, c)` that returns $a^b - c$. Form a 24x12x6 array containing its values in parameter ranges `[0,1] × [0,1] × [0,1]`.

Approximate the 3-d integral

$$\int_0^1 \int_0^1 \int_0^1 (a^b - c) da db dc$$

over this volume with the mean. The exact result is: $\ln 2 - \frac{1}{2} \approx 0.1931 \dots$ — what is your relative error?

(Hints: use elementwise operations and broadcasting. You can make `np.ogrid` give a number of points in given range with `np.ogrid[0:1:20j]`.)

Reminder Python functions:

```
def f(a, b, c):  
    return some_result
```

Solution to [Exercise 24](#)

```
import numpy as np
from numpy import newaxis

def f(a, b, c):
    return a**b - c

a = np.linspace(0, 1, 24)
b = np.linspace(0, 1, 12)
c = np.linspace(0, 1, 6)

samples = f(a[:, newaxis, newaxis], b[newaxis, :, newaxis], c[newaxis,
newaxis, :])

# or,
#
# a, b, c = np.ogrid[0:1:24j, 0:1:12j, 0:1:6j]
# samples = f(a, b, c)

integral = samples.mean()

print("Approximation:", integral)
print("Exact:", np.log(2) - 0.5)
```

```
Approximation: 0.1888423460296792
Exact: 0.1931471805599453
```

Mandelbrot set

Exercise 25

Write a script that computes the Mandelbrot fractal. The Mandelbrot iteration:

```
N_max = 50
some_threshold = 50

c = x + 1j*y

z = 0
for j in range(N_max):
    z = z**2 + c
```

```
/tmp/ipykernel_7280/585550489.py:8: RuntimeWarning: overflow
encountered in square
  z = z**2 + c
/tmp/ipykernel_7280/585550489.py:8: RuntimeWarning: invalid value
encountered in square
  z = z**2 + c
```

Point (x, y) belongs to the Mandelbrot set if $|z| < \text{some_threshold}$.

Do this computation by:

► Show code cell source

1. Construct a grid of $c = x + 1j*y$ values in range $[-2, 1] \times [-1.5, 1.5]$
2. Do the iteration
3. Form the 2-d boolean mask indicating which points are in the set
4. Save the result to an image with:

```
import matplotlib.pyplot as plt
plt.imshow(mask.T, extent=[-2, 1, -1.5, 1.5])
plt.gray()
plt.savefig('mandelbrot.png')
```

Solution to [Exercise 25](#)

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import newaxis

def compute_mandelbrot(N_max, some_threshold, nx, ny):
    # A grid of c-values
    x = np.linspace(-2, 1, nx)
    y = np.linspace(-1.5, 1.5, ny)

    c = x[:, newaxis] + 1j * y[newaxis, :]

    # Mandelbrot iteration

    z = c
    for j in range(N_max):
        z = z**2 + c

    mandelbrot_set = abs(z) < some_threshold

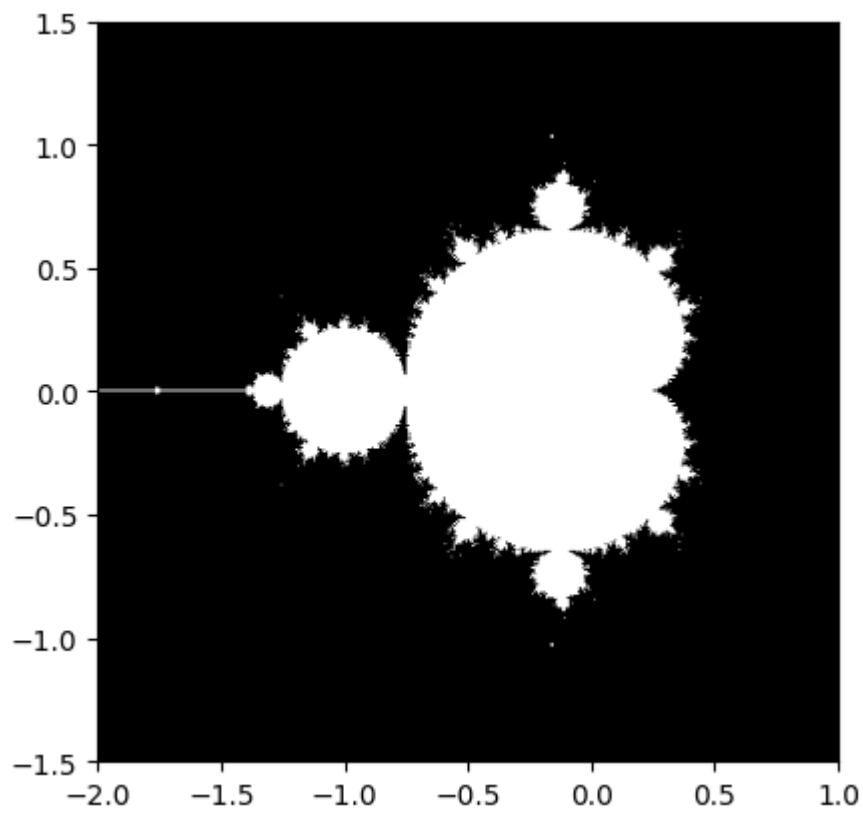
    return mandelbrot_set

# Save

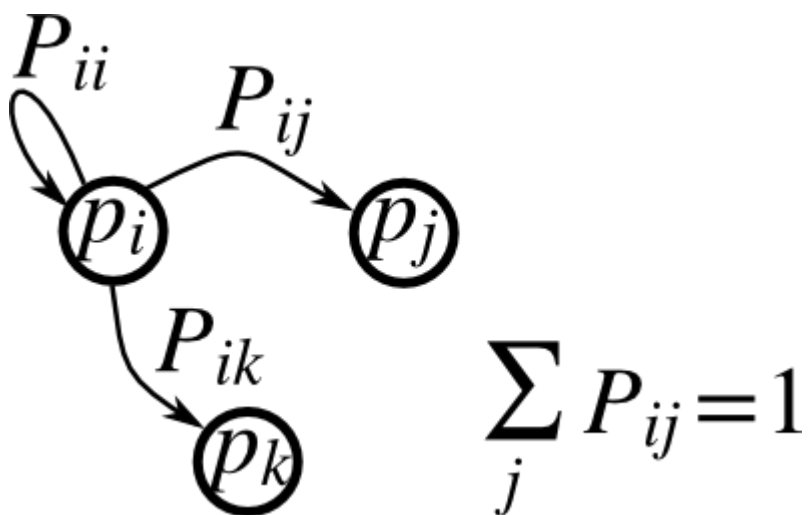
mandelbrot_set = compute_mandelbrot(50, 50.0, 601, 401)

plt.imshow(mandelbrot_set.T, extent=[-2, 1, -1.5, 1.5]) # type:
ignore[arg-type]
plt.gray()
plt.savefig("mandelbrot.png")
```

```
/tmp/ipykernel_7280/2370088976.py:17: RuntimeWarning: overflow
encountered in square
  z = z**2 + c
/tmp/ipykernel_7280/2370088976.py:17: RuntimeWarning: invalid value
encountered in square
  z = z**2 + c
```



Markov chain



Exercise 26

Markov chain transition matrix `P`, and probability distribution on the states `p`:

1. `0 <= P[i,j] <= 1`: probability to go from state `i` to state `j`
2. Transition rule: $p_{new} = P^T p_{old}$
3. `all(sum(P, axis=1) == 1)`, `p.sum() == 1`: normalization

Write a script that works with 5 states, and:

- Constructs a random matrix, and normalizes each row so that it is a transition matrix.
- Starts from a random (normalized) probability distribution `p` and takes 50 steps => `p_50`
- Computes the stationary distribution: the eigenvector of `P.T` with eigenvalue 1 (numerically: closest to 1) => `p_stationary`

Remember to normalize the eigenvector — I didn't...

- Checks if `p_50` and `p_stationary` are equal to tolerance $1e-5$

Toolbox: `np.random`, `@`, `np.linalg.eig`, reductions, `abs()`, `argmin`, comparisons, `all`, `np.linalg.norm`, etc.

Solution to [Exercise 26](#)

Solution to Markov chain exercise.

```
import numpy as np

rng = np.random.default_rng(27446968)

n_states = 5
n_steps = 50
tolerance = 1e-5

# Random transition matrix and state vector
P = rng.random(size=(n_states, n_states))
p = rng.random(n_states)

# Normalize rows in P
P /= P.sum(axis=1)[:, np.newaxis]

# Normalize p
p /= p.sum()

# Take steps
for k in range(n_steps):
    p = P.T @ p

p_50 = p
print(p_50)

# Compute stationary state
w, v = np.linalg.eig(P.T)

j_stationary = np.argmin(abs(w - 1.0))
p_stationary = v[:, j_stationary].real
p_stationary /= p_stationary.sum()
print(p_stationary)

# Compare
if all(abs(p_50 - p_stationary) < tolerance):
    print("Tolerance satisfied in infty-norm")

if np.linalg.norm(p_50 - p_stationary) < tolerance:
    print("Tolerance satisfied in 2-norm")
```

```
[0.15219309 0.25577972 0.22811916 0.14261754 0.22129049]
[0.15219309 0.25577972 0.22811916 0.14261754 0.22129049]
Tolerance satisfied in infty-norm
Tolerance satisfied in 2-norm
```

Matplotlib: plotting

Contents

- Introduction
- Simple plot
- Figures, Subplots, Axes and Ticks
- Other Types of Plots: examples and exercises
- Beyond this tutorial
- Quick reference

Authors: *Nicolas Rougier, Mike Müller, Gaël Varoquaux*

Thanks

Many thanks to **Bill Wing** and **Christoph Deil** for review and corrections.

Introduction

[Matplotlib](#) is probably the most used Python package for 2D-graphics. It provides both a quick way to visualize data from Python and publication-quality figures in many formats. We are going to explore matplotlib in interactive mode covering most common cases.

IPython, Jupyter, and matplotlib modes

The [Jupyter](#) notebook and the [IPython](#) enhanced interactive Python, are tuned for the scientific-computing workflow in Python, in combination with Matplotlib:

For interactive matplotlib sessions, turn on the **matplotlib mode**.

IPython sessions

To make plots open interactively in an IPython console session use the following [magic command](#):

```
%matplotlib
```



```
Using matplotlib backend: module://matplotlib_inline.backend_inline
```

Jupyter notebook

The Jupyter Notebook uses Matplotlib mode by default; that is, it inserts the figures into the notebook, as you run Matplotlib commands.

pyplot

pyplot provides a procedural interface to the matplotlib object-oriented plotting library. It is modeled closely after Matlab™. Therefore, the majority of plotting commands in pyplot have Matlab™ analogs with similar arguments. Important commands are explained with interactive examples.

```
import matplotlib.pyplot as plt
```

Simple plot

In this section, we want to draw the cosine and sine functions on the same plot. Starting from the default settings, we'll enrich the figure step by step to make it nicer.

First step is to get the data for the sine and cosine functions:

```
import numpy as np

X = np.linspace(-np.pi, np.pi, 256)
C, S = np.cos(X), np.sin(X)
```

X is now a numpy array with 256 values ranging from $-\pi$ to $+\pi$ (included). **C** is the cosine (256 values) and **S** is the sine (256 values).

To run the code, you can execute it in a Jupyter notebook or type it in an IPython interactive session:

```
$ ipython --matplotlib
```

This brings us to the IPython prompt:

```
IPython 0.13 -- An enhanced Interactive Python.  
?      -> Introduction to IPython's features.  
%magic -> Information about IPython's 'magic' % functions.  
help   -> Python's own help system.  
object? -> Details about 'object'. ?object also works, ?? prints more.
```

Plotting with default settings

Hint

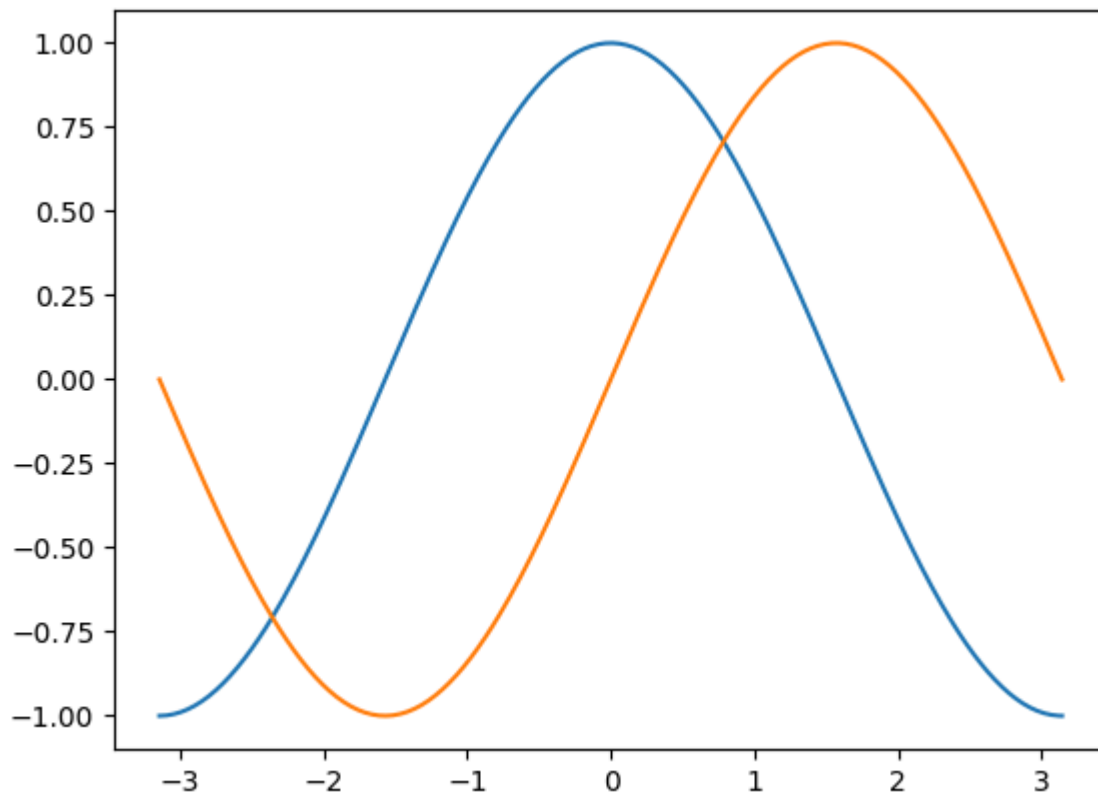
Documentation

- [plot tutorial](#)
- `plot()` command

Note

Matplotlib comes with a set of default settings that allow customizing all kinds of properties. You can control the defaults of almost every property in matplotlib: figure size and dpi, line width, color and style, axes, axis and grid properties, text and font properties and so on.

```
import numpy as np  
import matplotlib.pyplot as plt  
  
X = np.linspace(-np.pi, np.pi, 256)  
C, S = np.cos(X), np.sin(X)  
  
plt.plot(X, C)  
plt.plot(X, S);
```



Note

You will notice that we used a semicolon (;) to end the last line in the cell above. This is to prevent Jupyter or IPython echoing the return value of this final expression back to us in the notebook or console session. It has no other effect; it does not affect the execution of the code.

Instantiating defaults

Hint

Documentation

- [Customizing matplotlib](#)

In the plotting code below, you will see that we've instantiated (and commented) all the figure settings that influence the appearance of the plot.

Note



The settings have been explicitly set to their default values, but now you can interactively play with the values to explore their affect (see [Line properties](#) and [Line styles](#) below).

```
import numpy as np
import matplotlib.pyplot as plt

# Create a figure of size 8x6 inches, 80 dots per inch
plt.figure(figsize=(8, 6), dpi=80)

# Create a new subplot from a grid of 1x1
plt.subplot(1, 1, 1)

X = np.linspace(-np.pi, np.pi, 256)
C, S = np.cos(X), np.sin(X)

# Plot cosine with a blue continuous line of width 1 (pixels)
plt.plot(X, C, color="blue", linewidth=1.0, linestyle="-")

# Plot sine with a green continuous line of width 1 (pixels)
plt.plot(X, S, color="green", linewidth=1.0, linestyle="-")

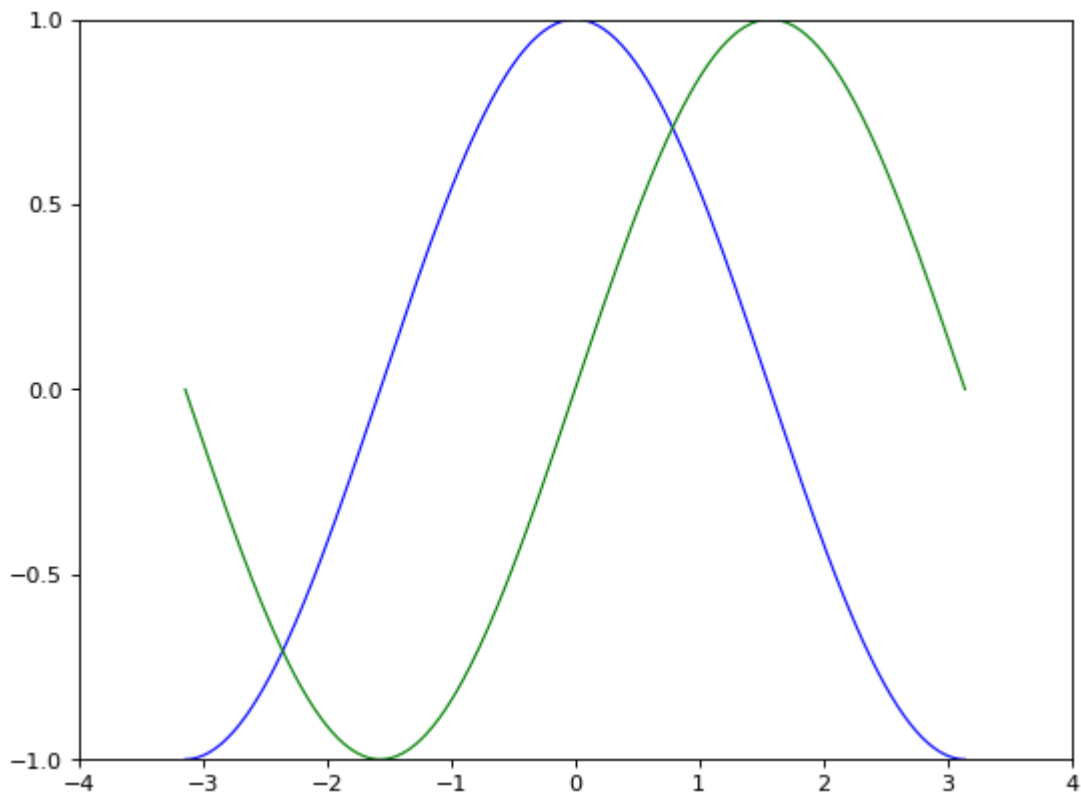
# Set x limits
plt.xlim(-4.0, 4.0)

# Set x ticks
plt.xticks(np.linspace(-4, 4, 9))

# Set y limits
plt.ylim(-1.0, 1.0)

# Set y ticks
plt.yticks(np.linspace(-1, 1, 5));

# You could also save this figure using 72 dots per inch with:
# plt.savefig("exercise_2.png", dpi=72)
```



Changing colors and line widths

Hint

Documentation

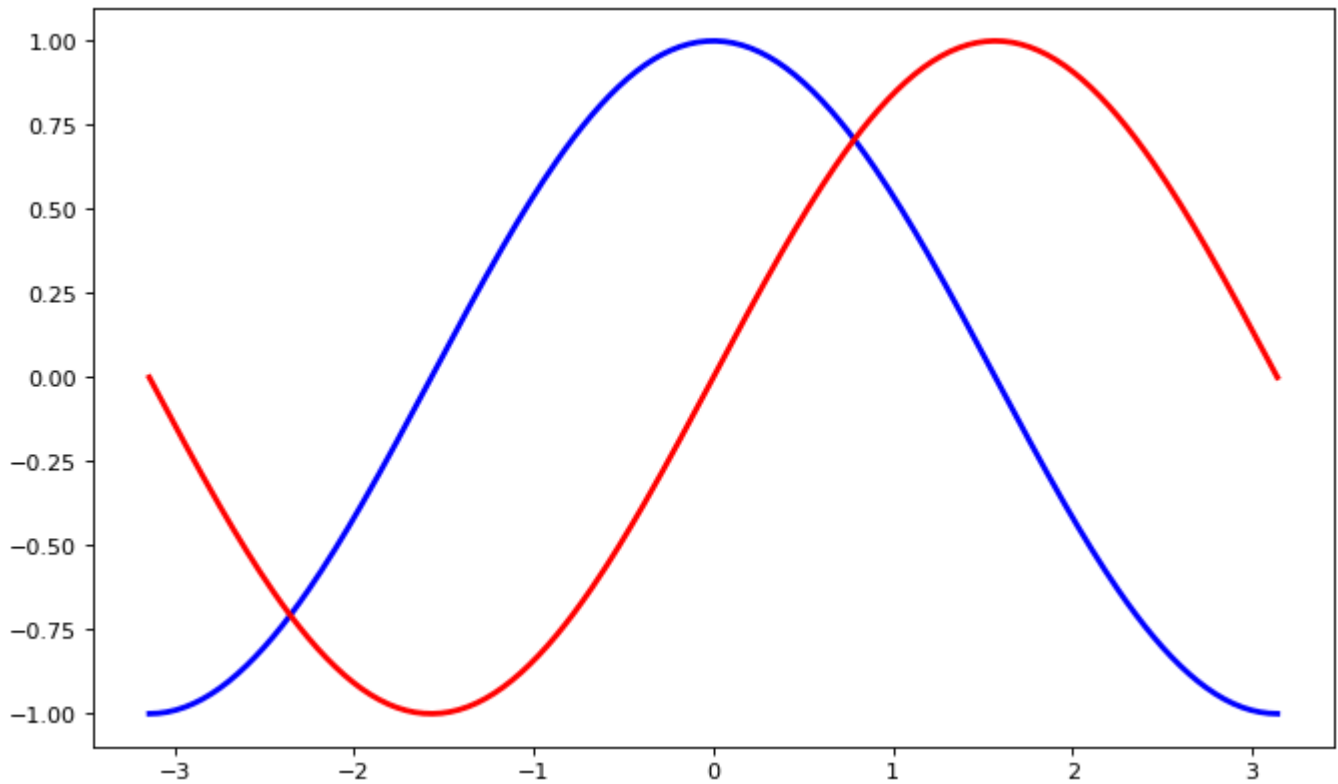
- [Controlling line properties](#)
- [Line2D](#) API

Note

First step, we want to have the cosine in blue and the sine in red and a slightly thicker line for both of them. We'll also slightly alter the figure size to make it more horizontal.

```
# Generate the plot.
plt.figure(figsize=(10, 6), dpi=80)
plt.plot(X, C, color="blue", linewidth=2.5, linestyle="-")
plt.plot(X, S, color="red", linewidth=2.5, linestyle="-");

# Get the current figure (gcf) into a variable for later use.
fig_to_update = plt.gcf()
```



Note

The final line `fig_to_update = plt.gcf()` uses `plt.gcf()` to Get the Current Figure — the figure we’ve just built in the cell. We then store that figure in the `fig_to_update` variable, so we can restore it, and update it, in the cells below. This is not a very common pattern in general, we are using it here to show you how to build up a figure in steps.

Setting limits

Hint

Documentation

- `xlim()` command
- `ylim()` command

Note

Current limits of the figure are a bit too tight and we want to make some space in order to clearly see all data points.

Note



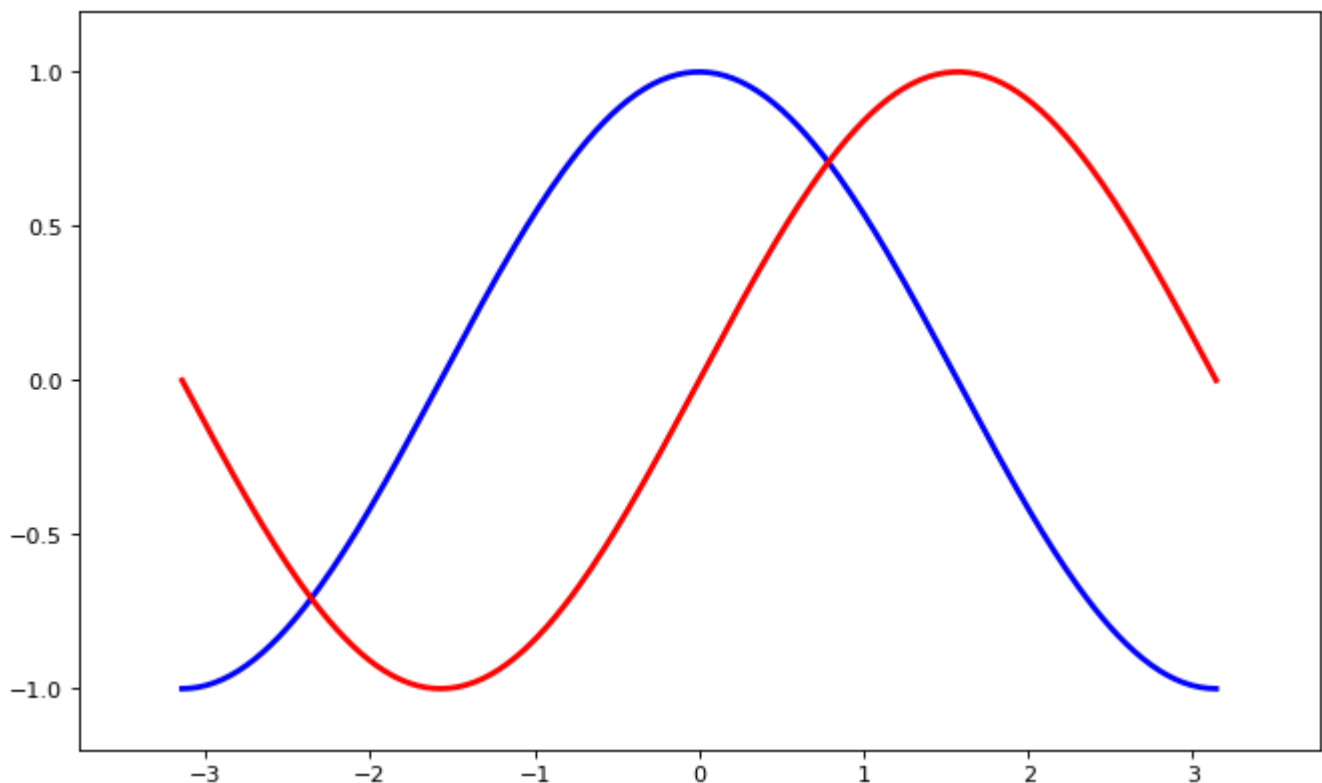
Following on from the note above, for the purposes of the tutorial, we first restore the figure we stored above (with `plt.figure(fig_to_update)`), then we add the limits to the figure, and finally, we press Jupyter to display the figure by putting the figure variable as an expression in the last line of the cell.

Again, this pattern of restore, update, redisplay is not a very common one in ordinary use of Matplotlib; we use it here to allow us to separate the various steps in the process of updating the figure.

```
# Restore previous figure, ready to update below.
plt.figure(fig_to_update)

# Setting the axis limits.
plt.xlim(X.min() * 1.2, X.max() * 1.2)
plt.ylim(C.min() * 1.2, C.max() * 1.2)

# Make Jupyter display updated figure.
fig_to_update
```



Setting ticks

Hint

Documentation

- `xticks()` command
- `yticks()` command
- [Tick container](#)
- [Tick locating and formatting](#)

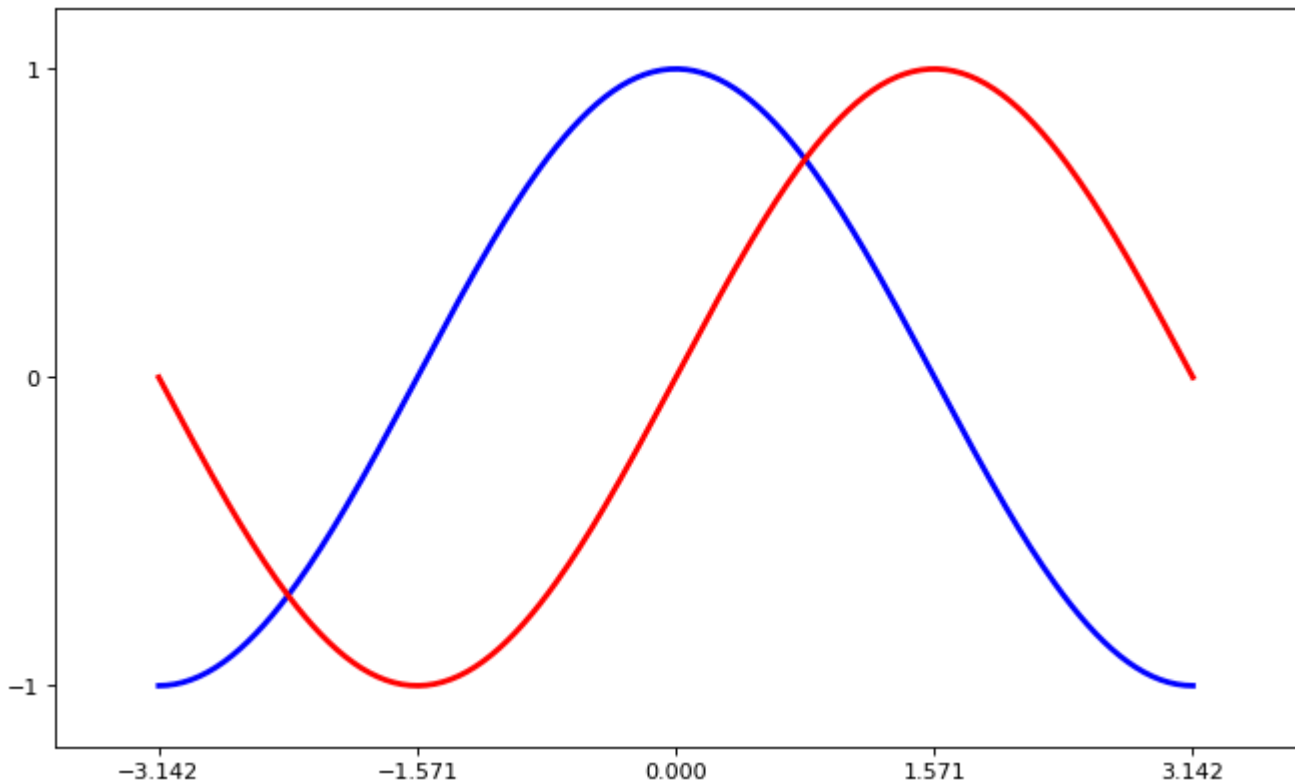
Note

Current ticks are not ideal because they do not show the interesting values ($\pm\pi$, $\pm\frac{\pi}{2}$) for sine and cosine. We'll change them such that they show only these values.

```
# Restore figure we are working on.
plt.figure(fig_to_update)

# Set x and y ticks.
plt.xticks([-np.pi, -np.pi / 2, 0, np.pi / 2, np.pi])
plt.yticks([-1, 0, +1])

# Make Jupyter display updated figure.
fig_to_update
```

Setting tick labels

Hint

Documentation

- [Working with text](#)
- `xticks()` command
- `yticks()` command
- `set_xticklabels()`
- `set_yticklabels()`

Note

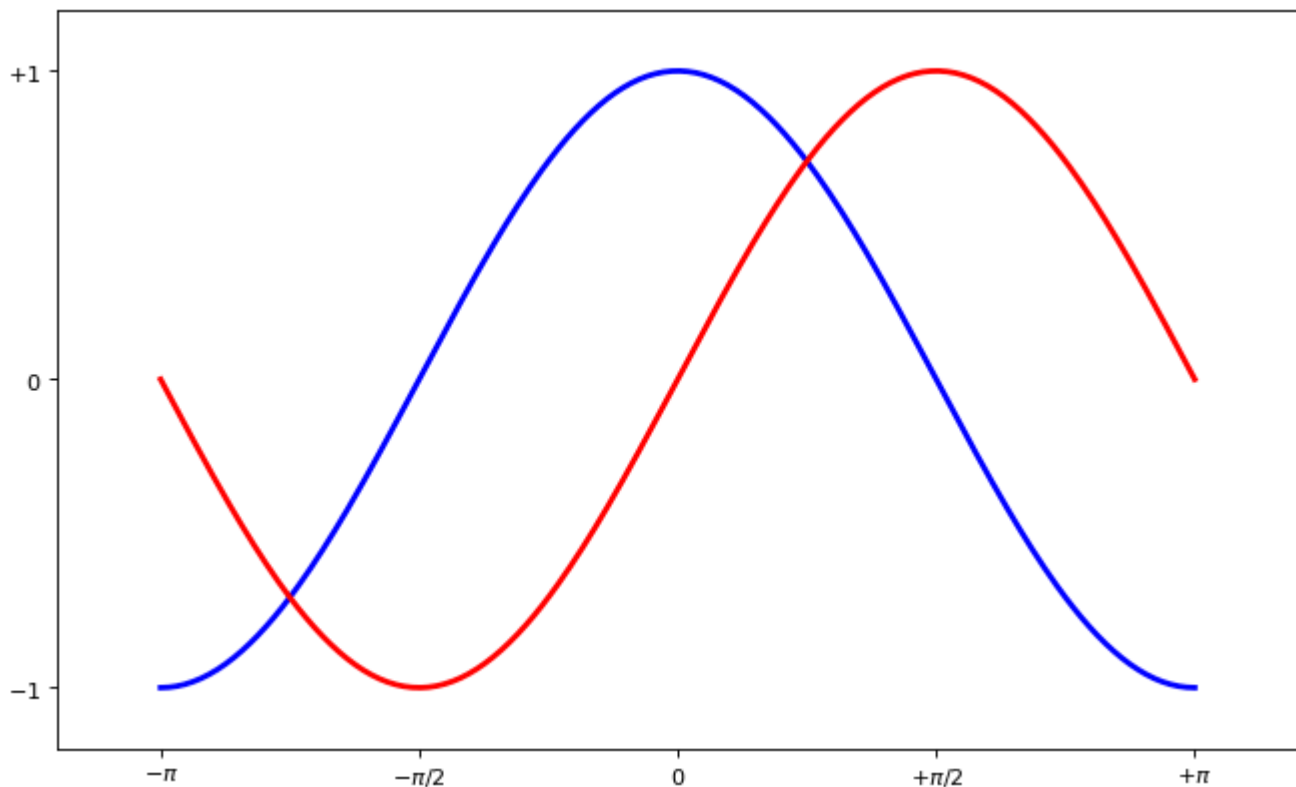
Ticks are now properly placed but their label is not very explicit. We could guess that 3.142 is π but it would be better to make it explicit. When we set tick values, we can also provide a corresponding label in the second argument list. Note that we'll use latex to allow for nice rendering of the label.

```
# Restore figure
plt.figure(fig_to_update)

# Update tick labels.
plt.xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi],
           [r'$-\pi$', r'$-\pi/2$', r'$0$', r'$+\pi/2$', r'$+\pi$'])

plt.yticks([-1, 0, +1],
           [r'$-1$', r'$0$', r'$+1$'])

# Force display of updated figure.
fig_to_update
```



Moving spines

💡 Hint

Documentation

- [spines](#) API
- [Axis container](#)
- [Transformations tutorial](#)

Note

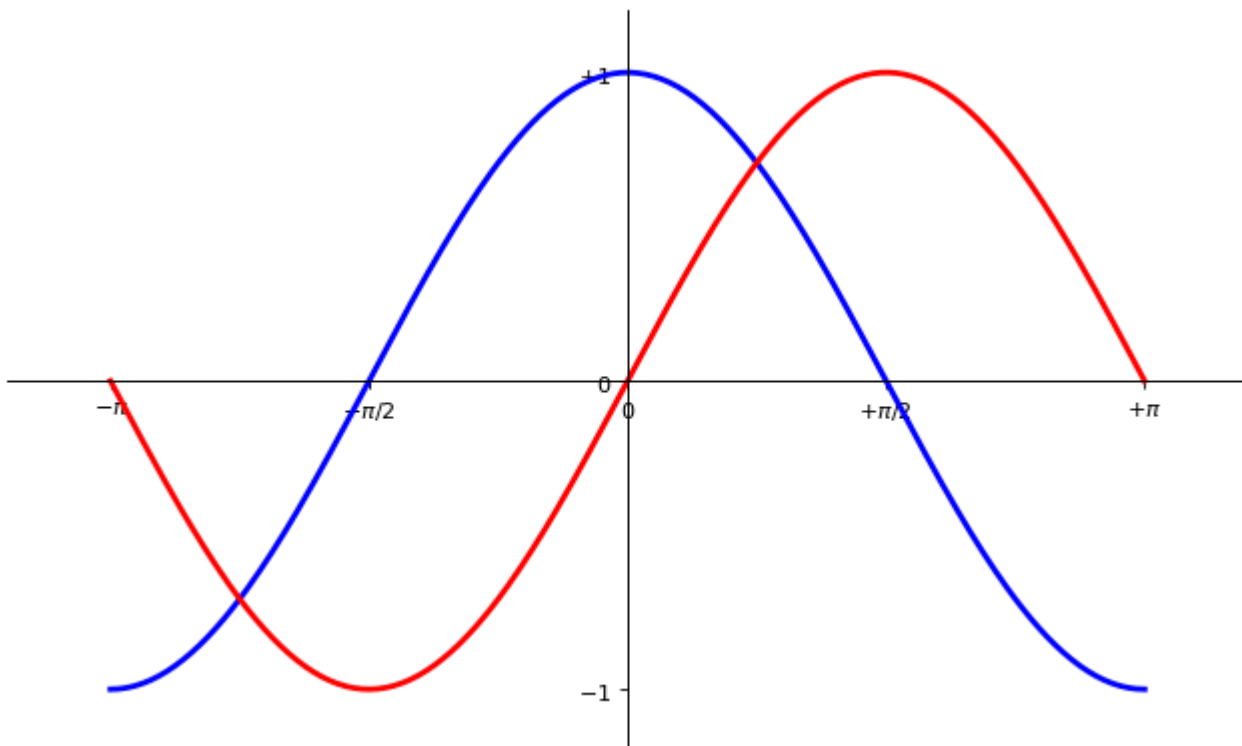


Spines are the lines connecting the axis tick marks and noting the boundaries of the data area. They can be placed at arbitrary positions and until now, they were on the border of the axis. We'll change that since we want to have them in the middle. Since there are four of them (top/bottom/left/right), we'll discard the top and right by setting their color to none and we'll move the bottom and left ones to coordinate 0 in data space coordinates.

```
# Restore figure
plt.figure(fig_to_update)

# Update spines.
ax = plt.gca() # gca stands for 'get current axis'
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',0))

# Force display of updated figure.
fig_to_update
```



Adding a legend

Hint

Documentation

- [Legend guide](#)
- `legend()` command
- [Legend](#) API

Note

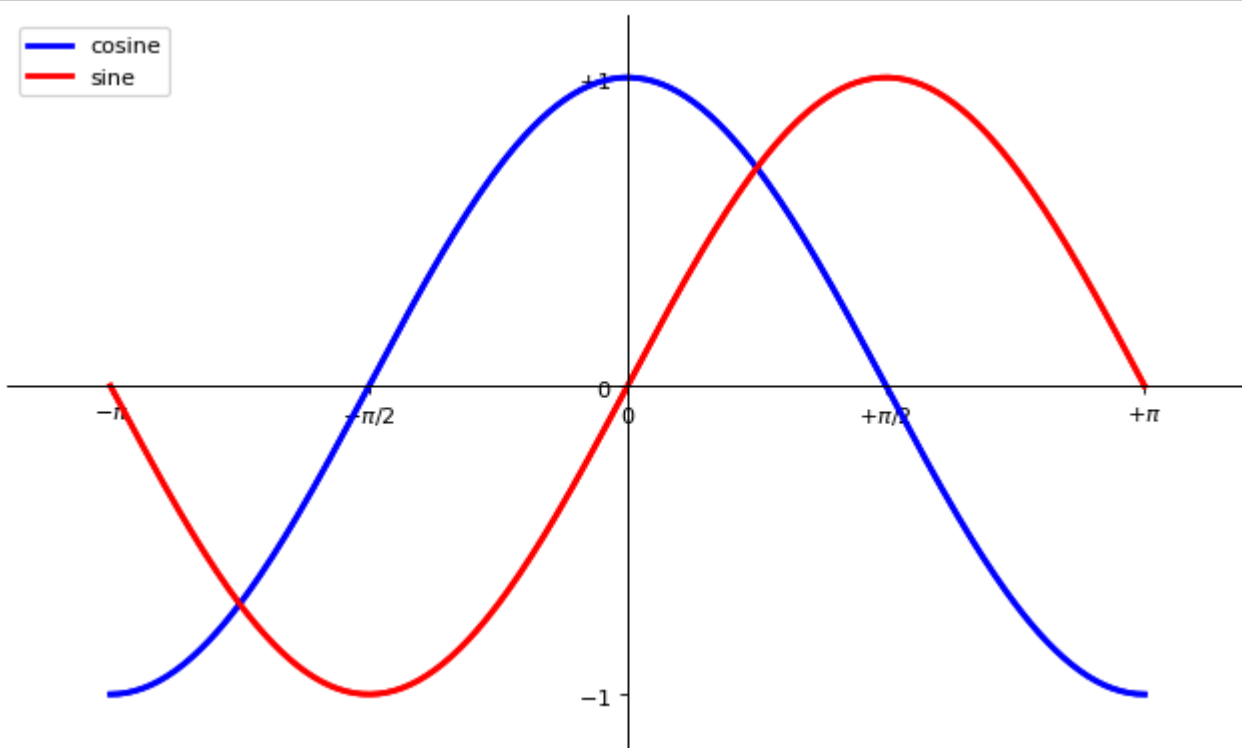
Let's add a legend in the upper left corner. This only requires adding the keyword argument `label` (that will be used in the legend box) to the plot commands.

```
# Restore figure
plt.figure(fig_to_update)

# Add legend.
plt.plot(X, C, color="blue", linewidth=2.5, linestyle="-", label="cosine")
plt.plot(X, S, color="red", linewidth=2.5, linestyle="-", label="sine")

plt.legend(loc='upper left')

# Force display of updated figure.
fig_to_update
```



Annotate some points

Hint

Documentation

- [Annotating axis](#)
- `annotate()` command

Note

Let's annotate some interesting points using the `annotate` command. We chose the $2\pi/3$ value and we want to annotate both the sine and the cosine. We'll first draw a marker on the curve as well as a straight dotted line. Then, we'll use the `annotate` command to display some text with an arrow.

```
# Restore figure
plt.figure(fig_to_update)

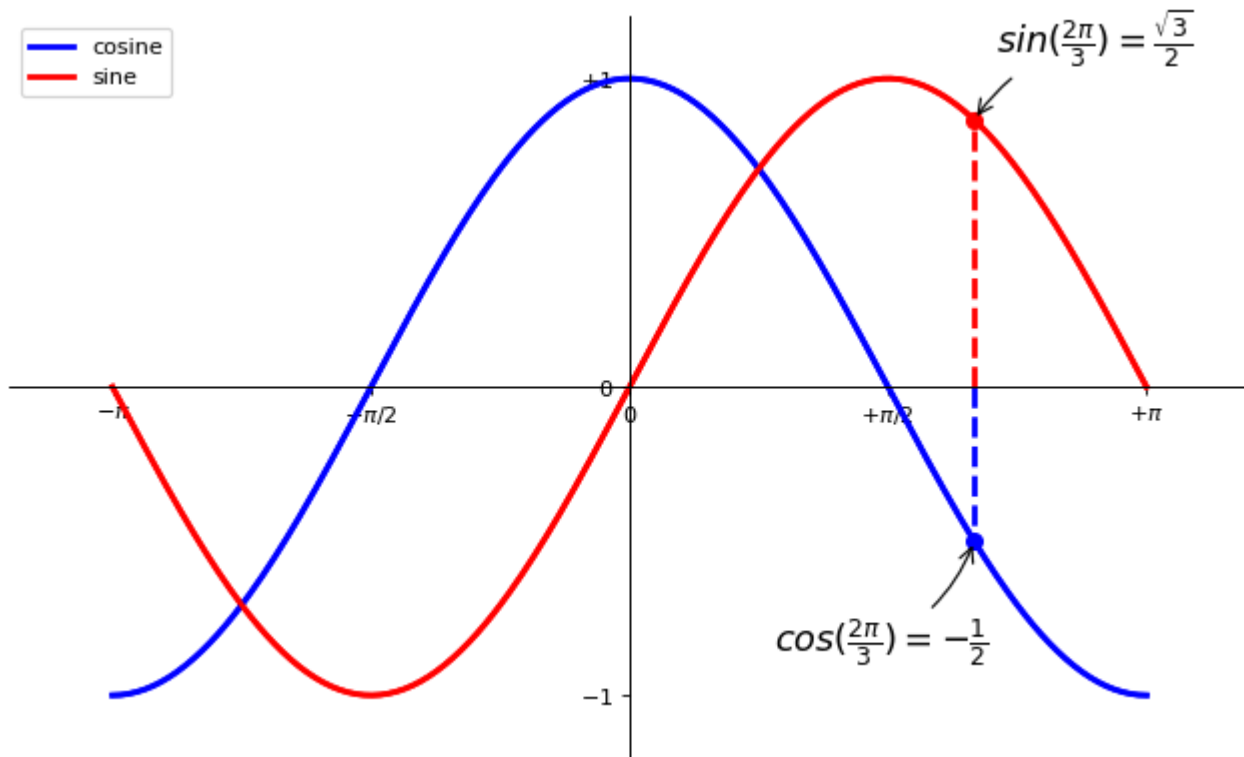
# Annotate points.
t = 2 * np.pi / 3
plt.plot([t, t], [0, np.cos(t)], color='blue', linewidth=2.5, linestyle="--")
plt.scatter([t, ], [np.cos(t), ], 50, color='blue')

plt.annotate(r'$\cos(\frac{2\pi}{3})=-\frac{1}{2}$',
             xy=(t, np.cos(t)), xycoords='data',
             xytext=(-90, -50), textcoords='offset points', fontsize=16,
             arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.2"))

plt.plot([t, t],[0, np.sin(t)], color='red', linewidth=2.5, linestyle="--")
plt.scatter([t, ],[np.sin(t), ], 50, color='red')

plt.annotate(r'$\sin(\frac{2\pi}{3})=\frac{\sqrt{3}}{2}$',
             xy=(t, np.sin(t)), xycoords='data',
             xytext=(+10, +30), textcoords='offset points', fontsize=16,
             arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.2"))

# Force display of updated figure.
fig_to_update
```



Devil is in the details

Hint

Documentation

- `artist` API
- `set_bbox()` method

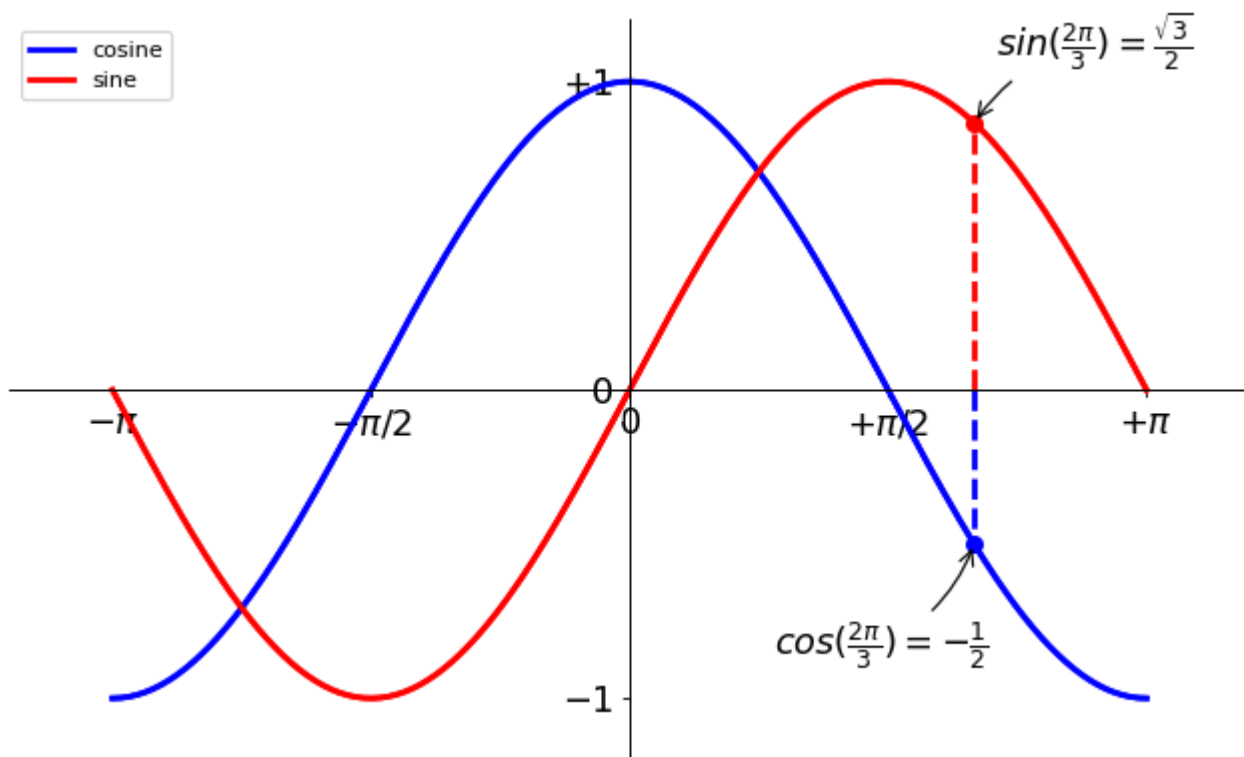
Note

The tick labels are now hardly visible because of the blue and red lines. We can make them bigger and we can also adjust their properties such that they'll be rendered on a semi-transparent white background. This will allow us to see both the data and the labels.

```
# Restore figure
plt.figure(fig_to_update)

# Set properties of tick labels.
for label in ax.get_xticklabels() + ax.get_yticklabels():
    label.set_fontsize(16)
    label.set_bbox(dict(facecolor='white', edgecolor='None', alpha=0.65))

# Force display of updated figure.
fig_to_update
```



Figures, Subplots, Axes and Ticks

A “**figure**” in matplotlib means the whole window in the user interface. Within this figure there can be “**subplots**”.

Note



So far we have used implicit figure and axes creation. This is handy for fast plots. We can have more control over the display using figure, subplot, and axes explicitly. While subplot positions the plots in a regular grid, axes allows free placement within the figure. Both can be useful depending on your intention. We've already worked with figures and subplots without explicitly calling them. When we call plot, matplotlib calls `gca()` to get the current axes and gca in turn calls `gcf()` to get the current figure. If there is none it calls `figure()` to make one, strictly speaking, to make a `subplot(111)`. Let's look at the details.

Figures

Note



A figure is a window in the GUI that has "Figure #" as title. Figures are numbered starting from 1 as opposed to the normal Python way starting from 0. This is clearly MATLAB-style. There are several parameters that determine what the figure looks like:

Argument	Default	Description
<code>num</code>	<code>1</code>	number of figure
<code>figsize</code>	<code>figure(figsize)</code>	figure size in inches (width, height)
<code>dpi</code>	<code>figure.dpi</code>	resolution in dots per inch
<code>facecolor</code>	<code>figure.facecolor</code>	color of the drawing background
<code>edgecolor</code>	<code>figure.edgecolor</code>	color of edge around the drawing background
<code>frameon</code>	<code>True</code>	draw figure frame or not

Note



The defaults can be specified in the resource file and will be used most of the time. Only the number of the figure is frequently changed.

As with other objects, you can set figure properties with the `plt.setp` function, or with the `set_`(something) methods.

When you work with the GUI, rather than in a notebook, you can close a figure by clicking on the x in the upper right corner. But you can close a figure programmatically by calling `close`. Depending on the argument it closes (1) the current figure (no argument), (2) a specific figure (figure number or figure instance as argument), or (3) all figures (`"all"` as argument).

```
# Useful working in a GUI outside the notebook.  
plt.close(1)      # Closes figure 1
```

Subplots

Note



With subplot you can arrange plots in a regular grid. You need to specify the number of rows and columns and the number of the plot. Note that the [gridspec](#) command is a more powerful alternative.

► Show code cell source

Horizontal subplots

```
subplot(2,1,1)
```

```
subplot(2,1,2)
```

► [Show code cell source](#)

Vertical subplots

```
subplot(1,2,1)
```

```
subplot(1,2,2)
```

► [Show code cell source](#)

Subplot grid



► [Show code cell source](#)

Subplot with gridspec



Axes

Axes are very similar to subplots but allow placement of plots at any location in the figure. So if we want to put a smaller plot inside a bigger one we do so with axes.

► Show code cell source

```
axes([0.1, 0.1, 0.8, 0.8])
```

► Show code cell source

```
axes([0.2, 0.2, 0.3, 0.3])
```

Ticks

Well formatted ticks are an important part of publishing-ready figures. Matplotlib provides a totally configurable system for ticks. There are tick locators to specify where ticks should appear and tick formatters to give ticks the appearance you want. Major and minor ticks can be located and formatted independently from each other. Per default minor ticks are not shown, i.e. there is only an empty list for them because it is as `NullLocator` (see below).

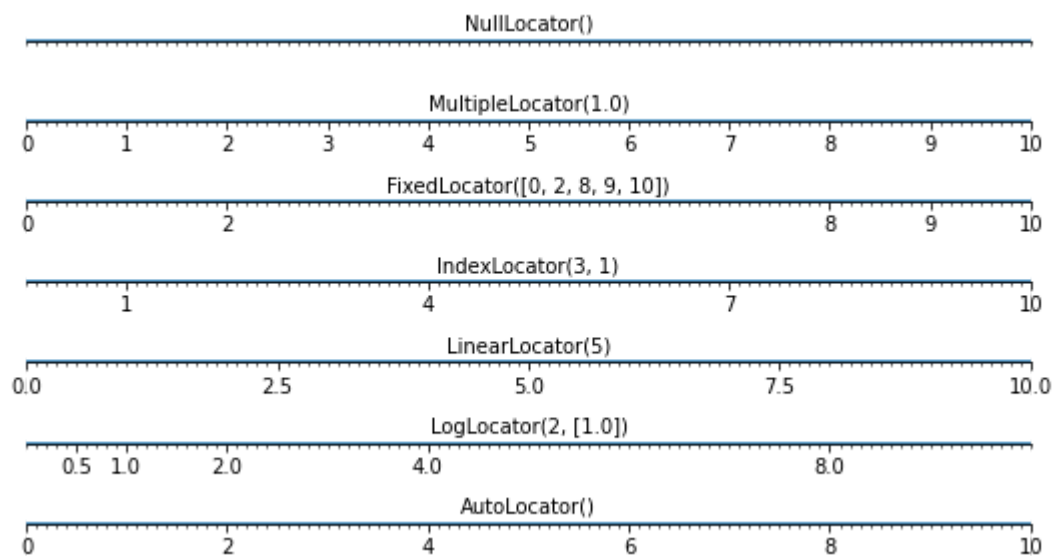
Tick Locators

Tick locators control the positions of the ticks. They are set as follows:

```
ax = plt.gca()
ax.xaxis.set_major_locator(eval(locator))
```

There are several locators for different kind of requirements:

► Show code cell source

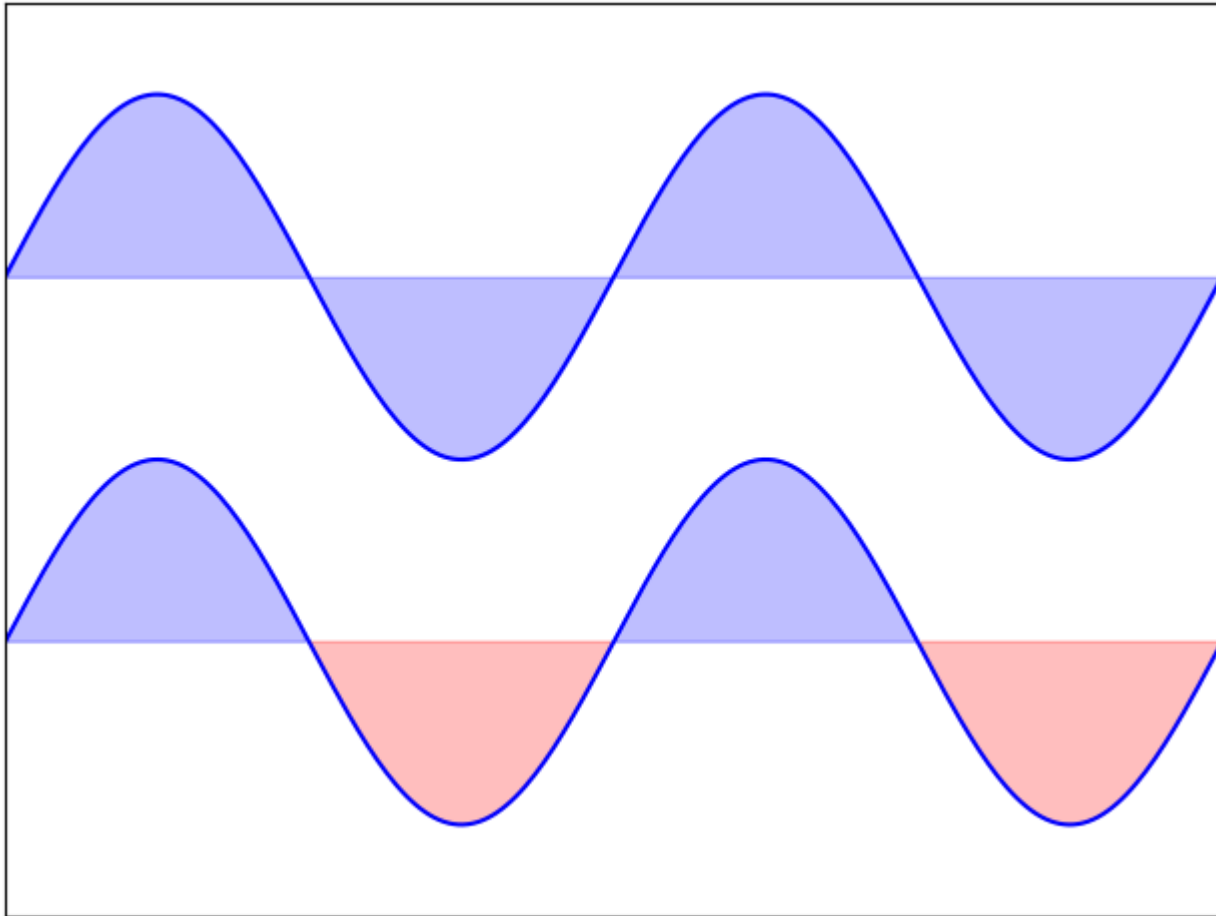


All of these “locators” (see code above) derive from the base class `matplotlib.ticker.Locator`. You can make your own locator deriving from it. Handling dates as ticks can be especially tricky. Therefore, matplotlib provides special locators in `matplotlib.dates`.

Other Types of Plots: examples and exercises

Regular Plots

► Show code cell source



Exercise 27



Starting from the code below, try to reproduce the graphic taking care of filled areas:

Hint

You need to use the `fill_between()` command.

```
n = 256
X = np.linspace(-np.pi, np.pi, n)
Y = np.sin(2 * X)

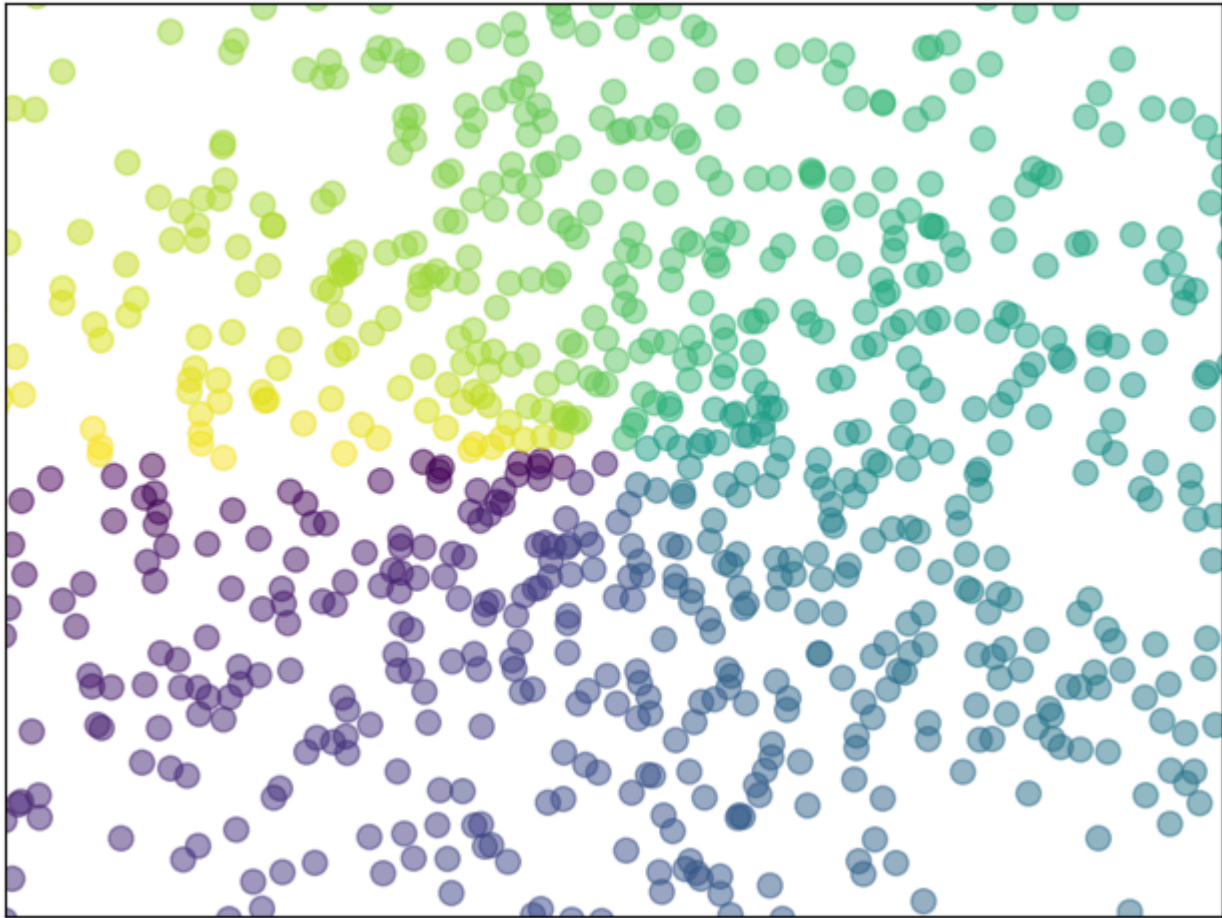
plt.plot(X, Y + 1, color='blue', alpha=1.00)
plt.plot(X, Y - 1, color='blue', alpha=1.00)
```

► Show code cell output ☐

Click on the hidden code for the figure above for solution.

Scatter Plots

► [Show code cell source](#)



Exercise 28



Starting from the code below, try to reproduce the graphic taking care of marker size, color and transparency.



Hint

Color is given by angle of (X,Y).

```
n = 1024
rng = np.random.default_rng()
X = rng.normal(0,1,n)
Y = rng.normal(0,1,n)

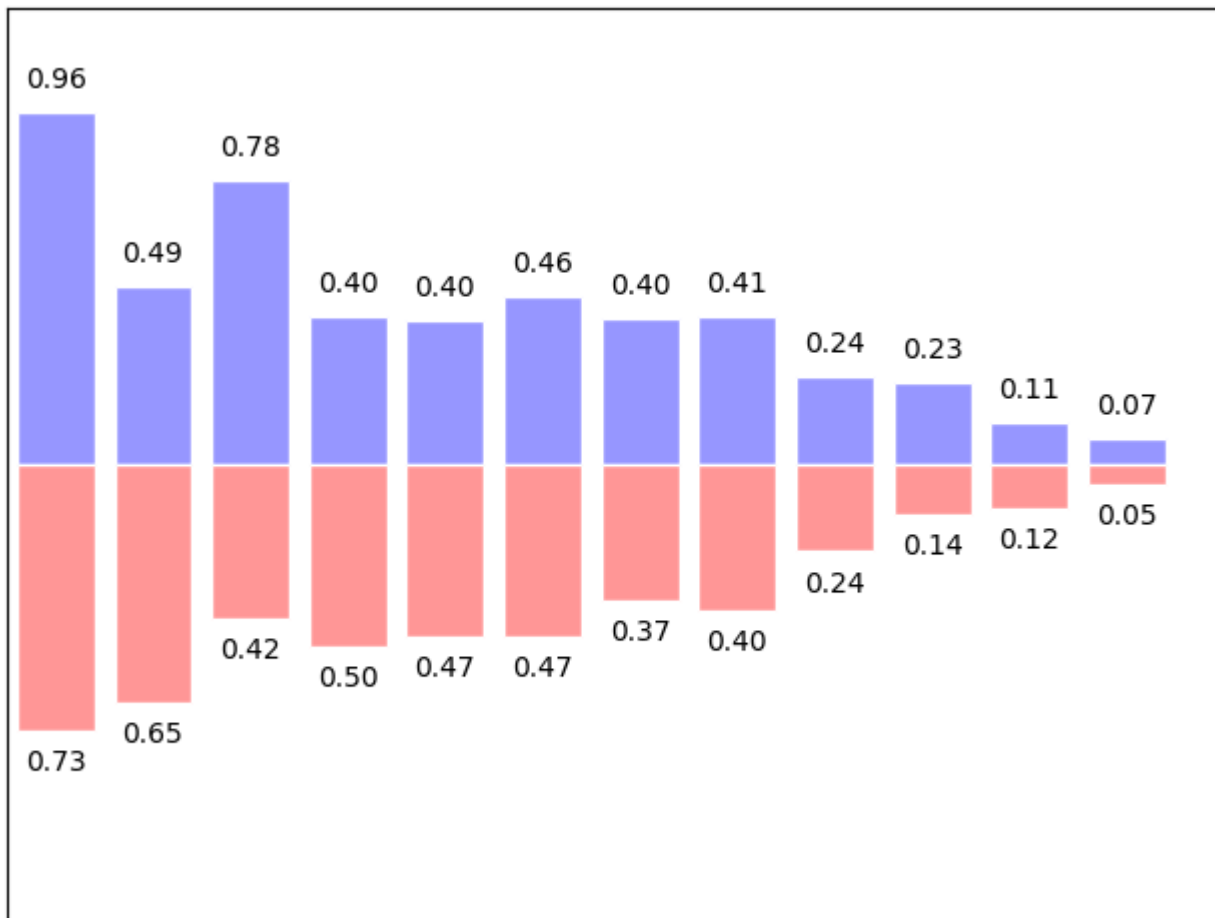
plt.scatter(X,Y)
```

► Show code cell output ☐

Click on the hidden code for the figure above for solution.

Bar Plots

► Show code cell source



Exercise 29

Starting from the code below, try to reproduce the graphic by adding labels for red bars.

Hint

You need to take care of text alignment.

```
n = 12
X = np.arange(n)
rng = np.random.default_rng()
Y1 = (1 - X / float(n)) * rng.uniform(0.5, 1.0, n)
Y2 = (1 - X / float(n)) * rng.uniform(0.5, 1.0, n)

plt.bar(X, +Y1, facecolor='#9999ff', edgecolor='white')
plt.bar(X, -Y2, facecolor='#ff9999', edgecolor='white')

for x, y in zip(X, Y1):
    plt.text(x + 0.4, y + 0.05, '%.2f' % y, ha='center', va='bottom')

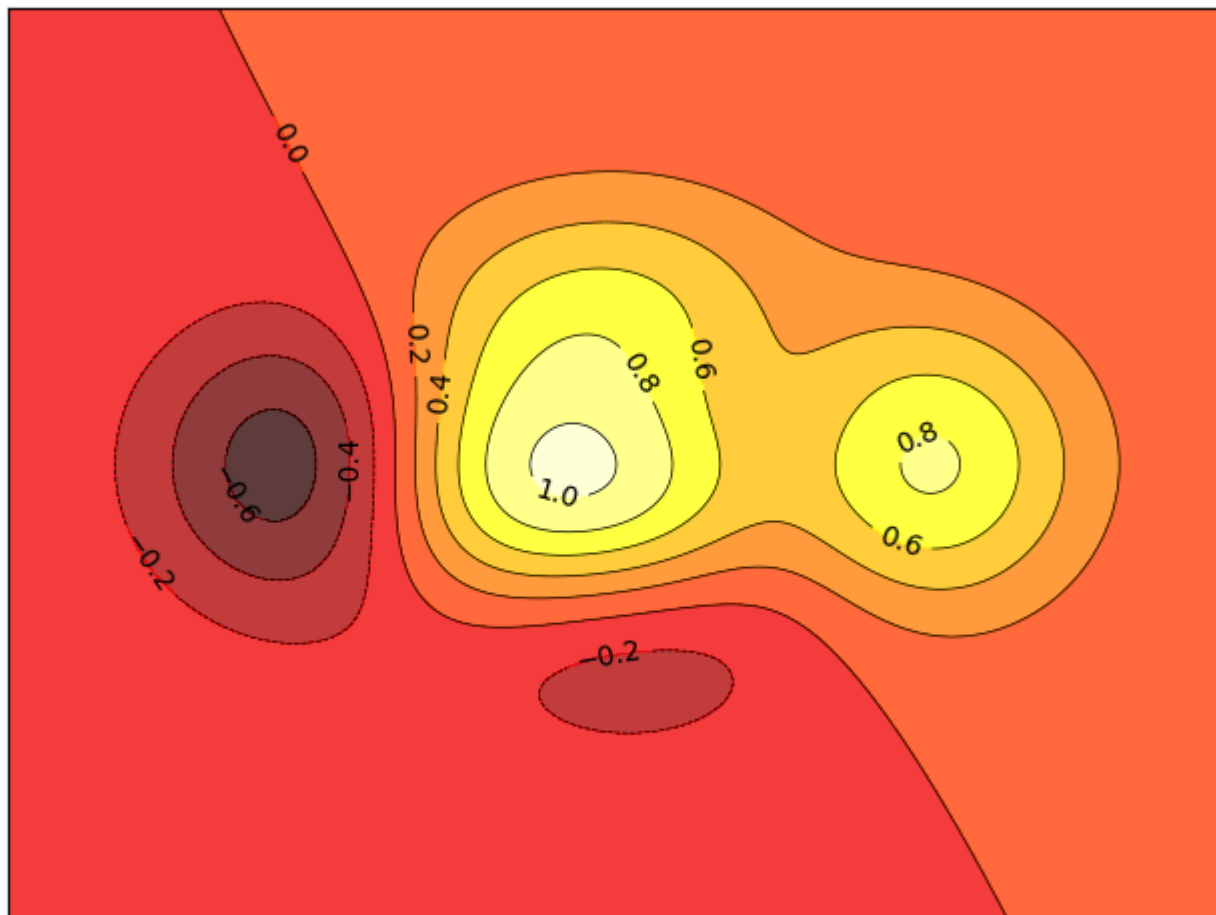
plt.ylim(-1.25, +1.25)
```

► Show code cell output

Click on the hidden code for the figure above for solution.

Contour Plots

► Show code cell source



Exercise 30



Starting from the code below, try to reproduce the graphic taking care of the colormap (see [Colormaps] below).

Hint

You need to use the `clabel()` command.

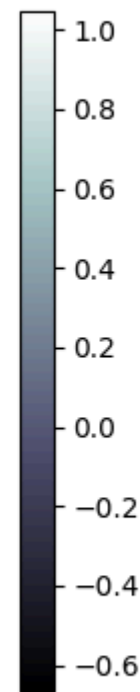
```
def f(x, y):  
    return (1 - x / 2 + x ** 5 + y ** 3) * np.exp(-x ** 2 - y ** 2)  
  
n = 256  
x = np.linspace(-3, 3, n)  
y = np.linspace(-3, 3, n)  
X, Y = np.meshgrid(x, y)  
  
plt.contourf(X, Y, f(X, Y), 8, alpha=.75, cmap='jet')  
C = plt.contour(X, Y, f(X, Y), 8, colors='black', linewidth=.5)
```

► Show code cell output

Click on the hidden code for the figure above for solution.

Imshow

► Show code cell source



Exercise 31



Starting from the code below, try to reproduce the graphic taking care of colormap, image interpolation and origin.

Hint

You need to take care of the `origin` of the image in the `imshow` command and use a `colorbar()`

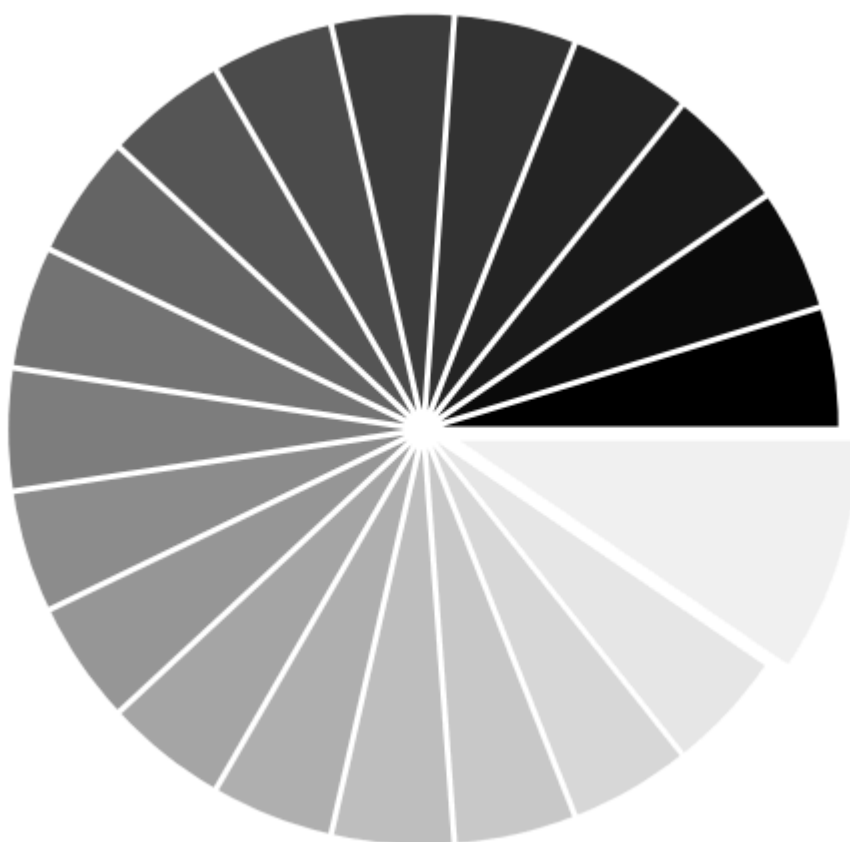
```
def f(x, y):  
    return (1 - x / 2 + x ** 5 + y ** 3) * np.exp(-x ** 2 - y ** 2)  
  
n = 10  
x = np.linspace(-3, 3, 4 * n)  
y = np.linspace(-3, 3, 3 * n)  
X, Y = np.meshgrid(x, y)  
plt.imshow(f(X, Y))
```

► Show code cell output ☐

Click on the hidden code for the figure above for solution.

Pie Charts

► Show code cell source



Exercise 32



Starting from the code below, try to reproduce the graphic taking care of colors and slices size.

Hint

You need to modify `Z`.

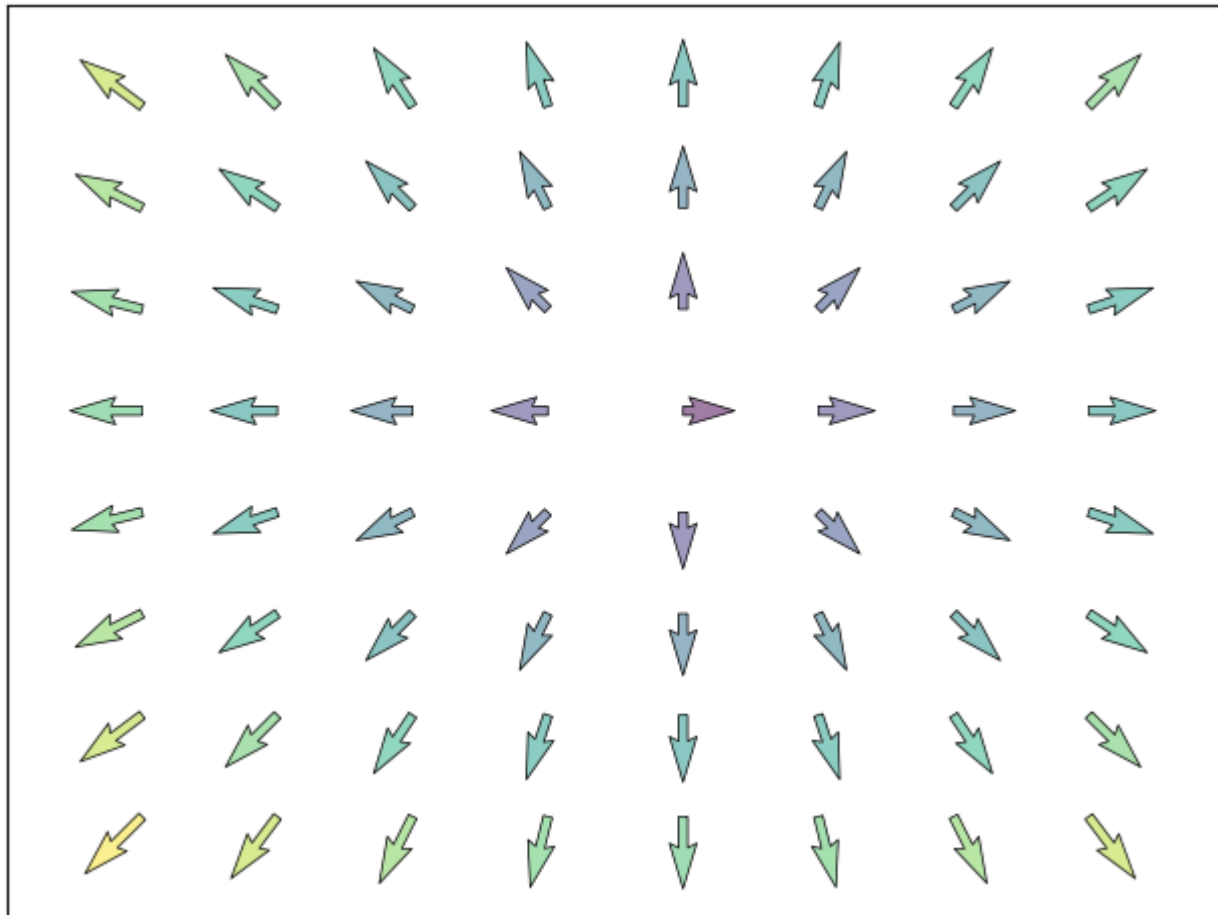
```
rng = np.random.default_rng()  
Z = rng.uniform(0, 1, 20)  
plt.pie(Z);
```

► Show code cell output

Click on the hidden code for the figure above for solution.

Quiver Plots

► Show code cell source



Exercise 33



Starting from the code below, try to reproduce the graphic taking care of colors and orientations.

Hint

You need to draw arrows twice.

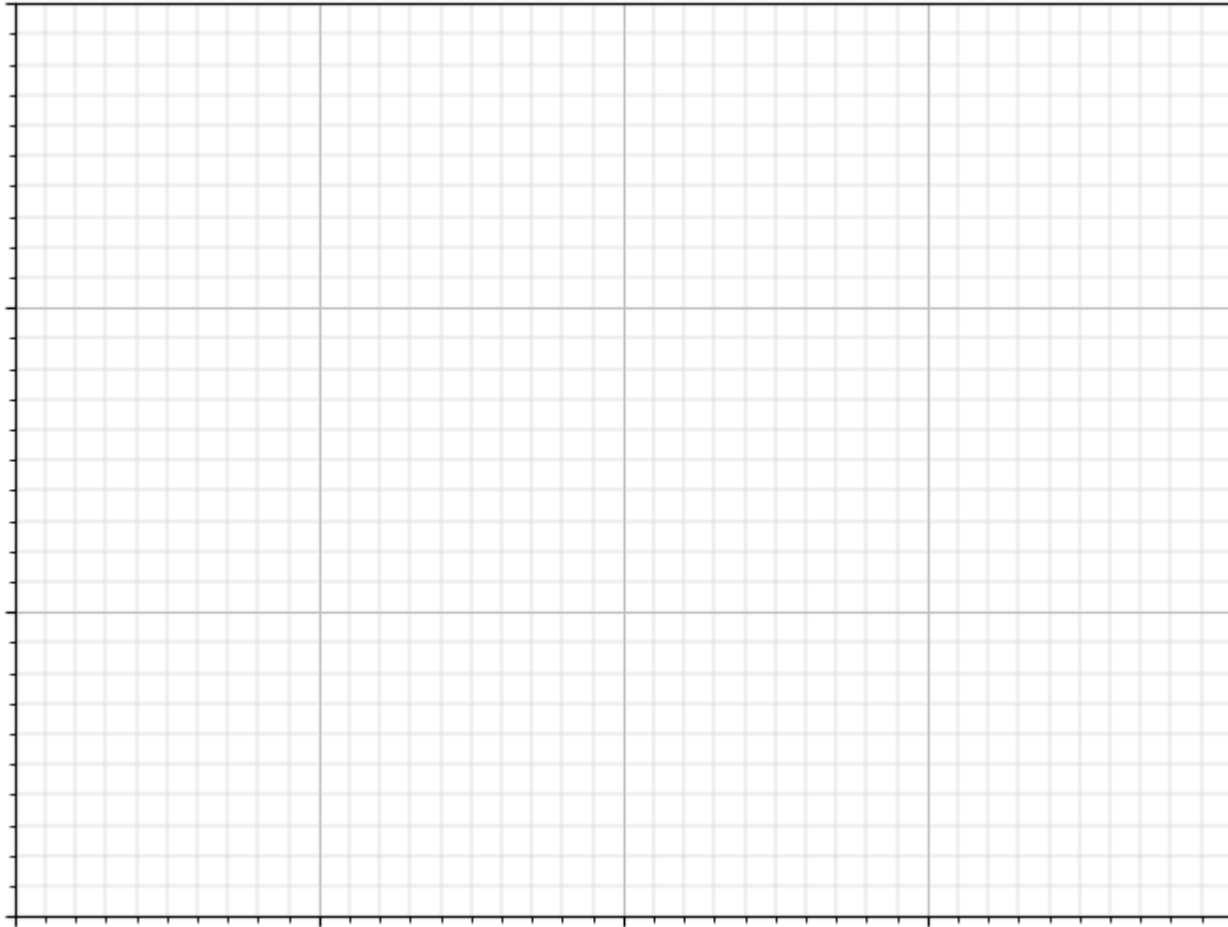
```
n = 8
X, Y = np.mgrid[0:n, 0:n]
plt.quiver(X, Y)
```

► Show code cell output

Click on the hidden code for the figure above for solution.

Grids

► Show code cell source



Exercise 34



Starting from the code below, try to reproduce the graphic taking care of line styles.

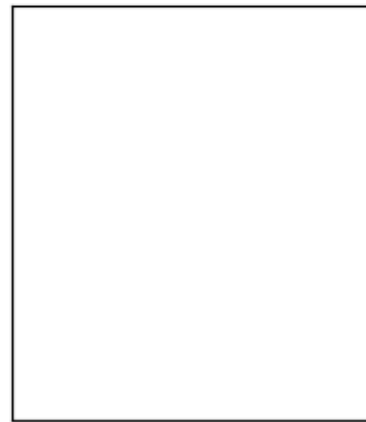
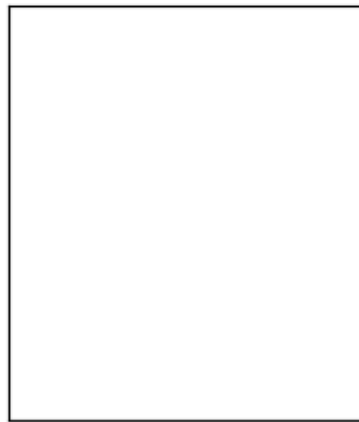
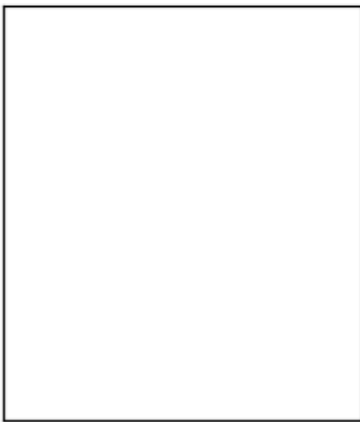
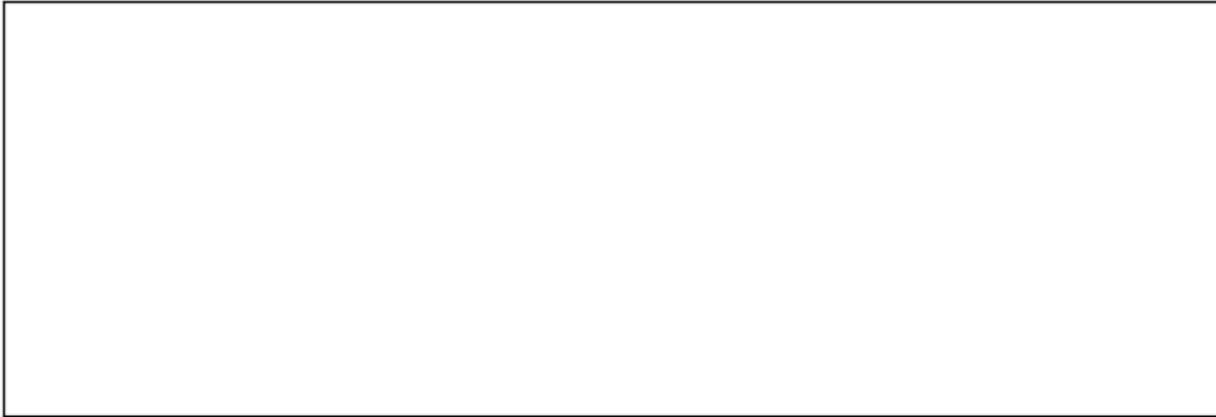
```
axes = plt.gca()
axes.set_xlim(0, 4)
axes.set_ylim(0, 3)
axes.set_xticklabels([])
axes.set_yticklabels([])
```

► Show code cell output

Click on the hidden code for the figure above for solution.

Multi Plots

► Show code cell source



Exercise 35



Starting from the code below, try to reproduce the graphic.

Hint

You can use several subplots with different partition.

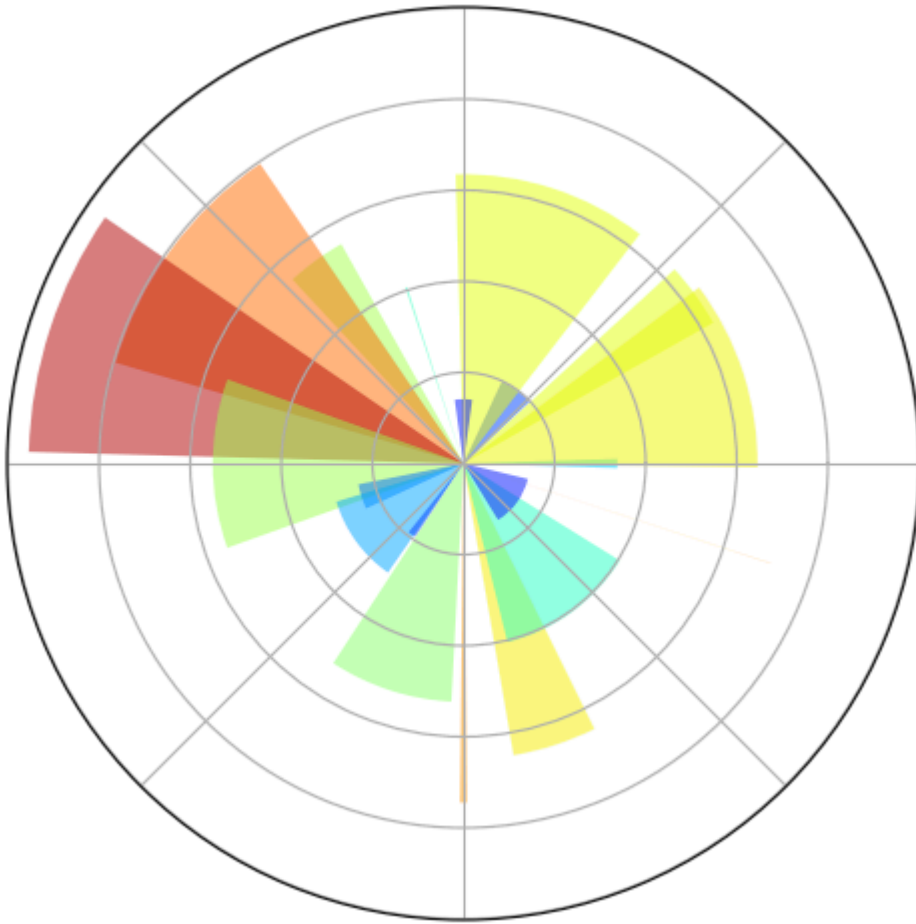
```
plt.subplot(2, 2, 1)
plt.subplot(2, 2, 3)
plt.subplot(2, 2, 4)
```

► Show code cell output ☐

Click on the hidden code for the figure above for solution.

Polar Axis

► [Show code cell source](#)



 Hint

You only need to modify the `axes` line

Starting from the code below, try to reproduce the graphic.

```
plt.axes([0, 0, 1, 1])

N = 20
theta = np.arange(0., 2 * np.pi, 2 * np.pi / N)
rng = np.random.default_rng()
radii = 10 * rng.random(N)
width = np.pi / 4 * rng.random(N)
bars = plt.bar(theta, radii, width=width, bottom=0.0)

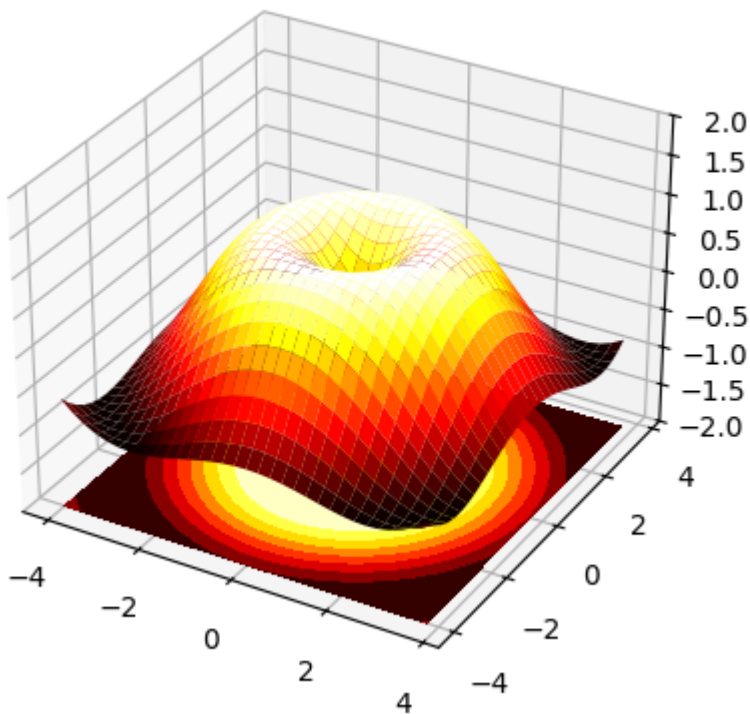
for r, bar in zip(radii, bars):
    bar.set_facecolor(plt.cm.jet(r / 10.))
    bar.set_alpha(0.5)
```

► Show code cell output

Click on the hidden code for the figure above for solution.

3D Plots

► Show code cell source



Exercise 37



Starting from the code below, try to reproduce the graphic.

Hint

You need to use `contourf()`

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = Axes3D(fig)
X = np.arange(-4, 4, 0.25)
Y = np.arange(-4, 4, 0.25)
X, Y = np.meshgrid(X, Y)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)

ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='hot')
```

► Show code cell output ☐

Click on the hidden code for the figure above for solution.

Text

► Show code cell source

The image displays a collage of various mathematical formulas. The most prominent formula at the top is the relativistic energy-momentum relation: $E = mc^2 = \sqrt{m_0^2 c^4 + p^2 c^2}$. Other visible formulas include the Gaussian integral $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$, the definition of the gamma function $\Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$, and the Navier-Stokes equation $\rho \frac{d\vec{v}}{dt} = -\nabla p + \mu \nabla^2 \vec{v} + \rho \vec{g}$. There are also expressions for the gravitational potential $\Phi = -\frac{GM}{r}$ and the Poisson equation $\nabla^2 \Phi = 4\pi G \rho$.

Exercise 38

Try to do the same from scratch !

Hint

Have a look at the [matplotlib logo](#).

Click on the hidden code for the figure above for solution.

Quick read

If you want to do a first quick pass through the Scientific Python Lectures to learn the ecosystem, you can directly skip to the next chapter: [SciPy: high-level scientific computing](#).

The remainder of this chapter is not necessary to follow the rest of the intro part. But be sure to come back and finish this chapter later.

Beyond this tutorial

Matplotlib benefits from extensive documentation as well as a large community of users and developers. Here are some links of interest:

Tutorials

- [Pyplot tutorial](#)
 - Introduction
 - Controlling line properties
 - Working with multiple figures and axes
 - Working with text
- [Image tutorial](#)
 - Startup commands
 - Importing image data into NumPy arrays
 - Plotting NumPy arrays as images
- [Text tutorial](#)
 - Text introduction
 - Basic text commands
 - Text properties and layout
 - Writing mathematical expressions
 - Text rendering With LaTeX
 - Annotating text
- [Artist tutorial](#)
 - Introduction

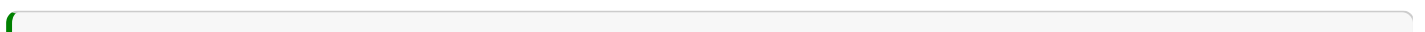
- Customizing your objects
- Object containers
- Figure container
- Axes container
- Axis containers
- Tick containers
- [Path tutorial](#)
 - Introduction
 - Bézier example
 - Compound paths
- [Transforms tutorial](#)
 - Introduction
 - Data coordinates
 - Axes coordinates
 - Blended transformations
 - Using offset transforms to create a shadow effect
 - The transformation pipeline

Matplotlib documentation

- [User guide](#)
- [FAQ](#)
 - Installation
 - Usage
 - How-To
 - Troubleshooting
 - Environment Variables
- [Screenshots](#)

Code documentation

The code is well documented and you can quickly access a specific command from within a python session:



```
import matplotlib.pyplot as plt
help(plt.plot)
```

Help on function plot in module matplotlib.pyplot:

```
plot(*args: 'float | ArrayLike | str', scalex: 'bool' = True, scaley: 'bool' =
True, data=None, **kwargs) -> 'list[Line2D]'
    Plot y versus x as lines and/or markers.
```

Call signatures::

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

The coordinates of the points or line nodes are given by **x**, **y**.

The optional parameter **fmt** is a convenient way for defining basic formatting like color, marker and linestyle. It's a shortcut string notation described in the **Notes** section below.

```
>>> plot(x, y)           # plot x and y using default line style and color
>>> plot(x, y, 'bo')      # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')         # ditto, but with red plusses
```

You can use ``.Line2D`` properties as keyword arguments for more control on the appearance. Line properties and **fmt** can be mixed. The following two calls yield identical results:

```
>>> plot(x, y, 'go--', linewidth=2, markersize=12)
>>> plot(x, y, color='green', marker='o', linestyle='dashed',
...      linewidth=2, markersize=12)
```

When conflicting with **fmt**, keyword arguments take precedence.

****Plotting labelled data****

There's a convenient way for plotting objects with labelled data (i.e. data that can be accessed by index ``obj['y']``). Instead of giving

```
[... salida de 10805 caracteres recortada en el PDF; completa en la versión web
...]
```

Galleries

The [matplotlib gallery](#) is also incredibly useful when you search how to render a given graphic. Each example comes with its source.

Mailing lists

Finally, there is a [user mailing list](#) where you can ask for help and a [developers mailing list](#) that is more technical.

Quick reference

Here is a set of tables that show main properties and styles.

Line properties

Property	Description	Appearance
alpha (or a)	alpha transparency on 0-1 scale	
anti-aliased	True or False - use anti-aliased rendering	Aliased Anti-aliased
color (or c)	matplotlib color arg	
linestyle (or ls)	see Line properties	
linewidth (or lw)	float, the line width in points	
solid_capstyle	Cap style for solid lines	
solid_joinstyle	Join style for solid lines	
dash_capstyle	Cap style for dashes	
dash_joinstyle	Join style for dashes	
marker	see Markers	
markeredgewidth (mew)	line width around the marker symbol	
markeredgecolor (mec)	edge color if a marker is used	
markerfacecolor (mfc)	face color if a marker is used	
markersize (ms)	size of the marker in points	

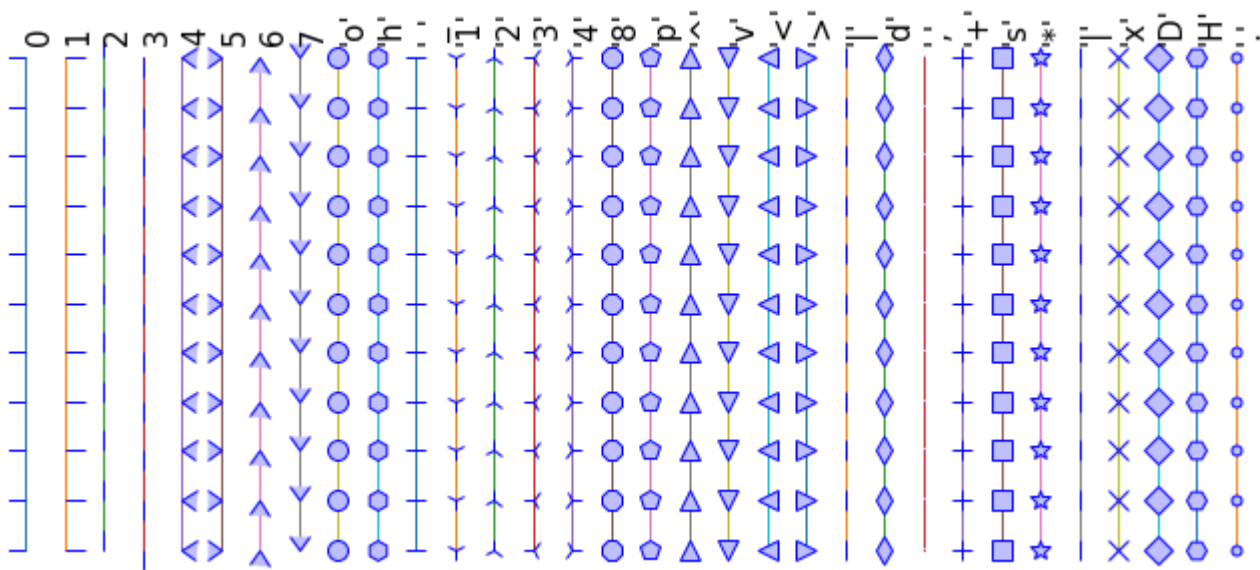
See the [Line property figures](#) for code to generate graphics for the table above.

Line styles



See [Line style figure](#) for code.

Markers

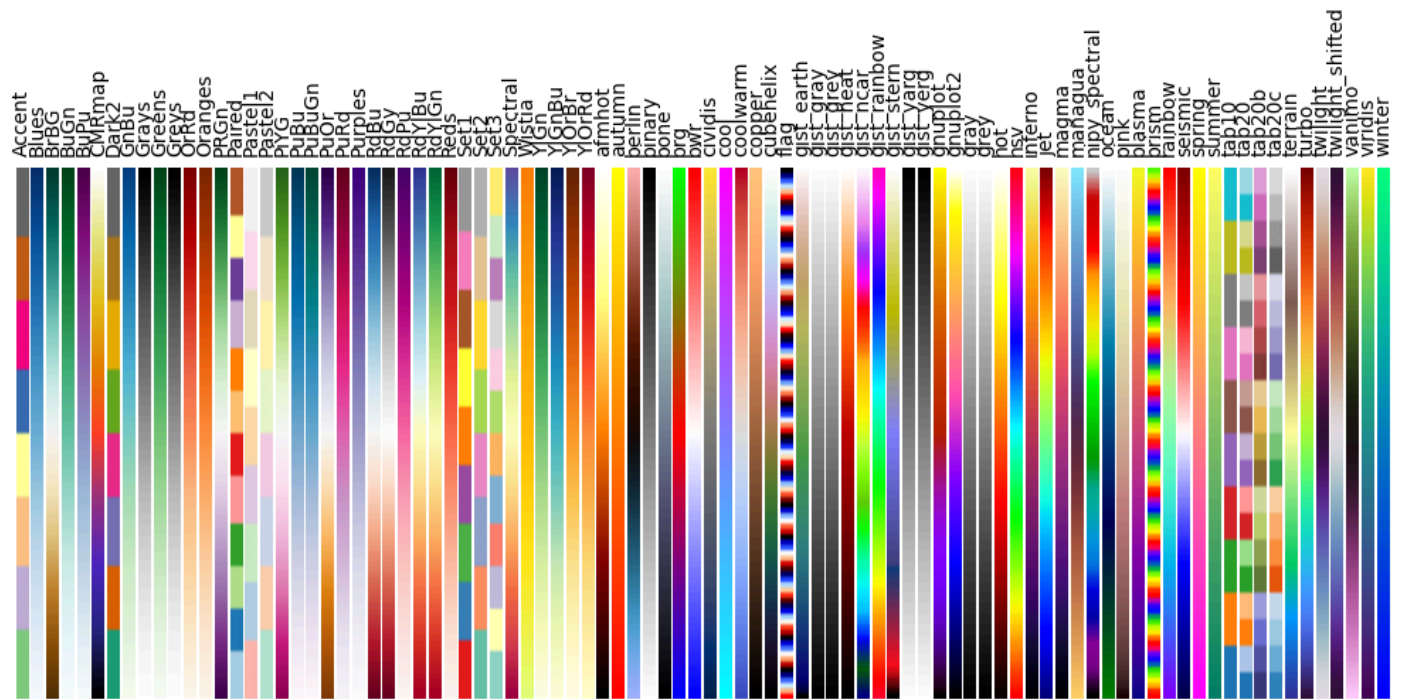


See [Marker style figure](#) for code.

Colormaps

All colormaps can be reversed by appending `_r`. For instance, `gray_r` is the reverse of `gray`.

If you want to know more about colormaps, check the [documentation on Colormaps in matplotlib](#).



See [Colormap figure](#) for code.

SciPy: high-level scientific computing

Contents

- File input/output: `scipy.io`
- Special functions: `scipy.special`
- Linear algebra operations: `scipy.linalg`
- Interpolation: `scipy.interpolate`
- Optimization and fit: `scipy.optimize`
- Statistics and random numbers: `scipy.stats`
- Numerical integration: `scipy.integrate`
- Fast Fourier transforms: `scipy.fft`
- Signal processing: `scipy.signal`
- Image manipulation: `scipy.ndimage`
- Summary exercises on scientific computing
- Other useful links

Authors: *Gaël Varoquaux, Adrien Chauve, Andre Espaze, Emmanuelle Gouillart, Ralf Gommers*

Scipy

The `scipy` package contains various toolboxes dedicated to common issues in scientific computing. Its different submodules correspond to different applications, such as interpolation, integration, optimization, image processing, statistics, special functions, etc.

Note



`scipy` can be compared to other standard scientific-computing libraries, such as the GSL (GNU Scientific Library for C and C++), or Matlab's toolboxes. `scipy` is the core package for scientific routines in Python; it is meant to operate efficiently on `numpy` arrays, so that NumPy and SciPy work hand in hand.

Before implementing a routine, it is worth checking if the desired data processing is not already implemented in SciPy. As non-professional programmers, scientists often tend to **re-invent the wheel**, which leads to buggy, non-optimal, difficult-to-share and unmaintainable code. By contrast, `SciPy`'s routines are optimized and tested, and should therefore be used when possible.

Warning

This tutorial is far from an introduction to numerical computing. As enumerating the different submodules and functions in SciPy would be very boring, we concentrate instead on a few examples to give a general idea of how to use `scipy` for scientific computing.

`scipy` is composed of task-specific sub-modules:

<code>scipy.cluster</code>	Vector quantization / Kmeans
<code>scipy.constants</code>	Physical and mathematical constants
<code>scipy.fft</code>	Fourier transform
<code>scipy.integrate</code>	Integration routines
<code>scipy.interpolate</code>	Interpolation
<code>scipy.io</code>	Data input and output
<code>scipy.linalg</code>	Linear algebra routines
<code>scipy.ndimage</code>	n-dimensional image package
<code>scipy.odr</code>	Orthogonal distance regression
<code>scipy.optimize</code>	Optimization
<code>scipy.signal</code>	Signal processing
<code>scipy.sparse</code>	Sparse matrices
<code>scipy.spatial</code>	Spatial data structures and algorithms
<code>scipy.special</code>	Any special mathematical functions
<code>scipy.stats</code>	Statistics

Scipy modules all depend on `numpy`, but are mostly independent of each other. The standard way of importing NumPy and these SciPy modules is:

```
import numpy as np
import scipy as sp
```

We will also be using plotting for this tutorial.

```
import matplotlib.pyplot as plt
```

File input/output: scipy.io

scipy.io contains functions for loading and saving data in several common formats including Matlab, IDL, Matrix Market, and Harwell-Boeing.

Matlab files: Loading and saving:

```
a = np.ones((3, 3))
sp.io.savemat('file.mat', {'a': a}) # savemat expects a dictionary
data = sp.io.loadmat('file.mat')
data['a']
```

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

Warning — Python / Matlab mismatch

The Matlab file format does not support 1D arrays.

```
a = np.ones(3)
a
```

```
array([1., 1., 1.])
```

```
a.shape
```

```
(3,)
```

```
sp.io.savemat('file.mat', {'a': a})
a2 = sp.io.loadmat('file.mat')['a']
a2
```

```
array([[1., 1., 1.]])
```

```
a2.shape
```

```
(1, 3)
```


Notice that the original array was a one-dimensional array, whereas the saved and reloaded array is a two-dimensional array with a single row.

For other formats, see the [scipy.io](#) documentation.

End of warning

See also

- Load text files: `numpy.loadtxt()` / `numpy.savetxt()`
- Clever loading of text/csv files: `numpy.genfromtxt()`
- Fast and efficient, but NumPy-specific, binary format:
`numpy.save()` / `numpy.load()`
- Basic input/output of images in Matplotlib:
`matplotlib.pyplot.imread()` / `matplotlib.pyplot.imsave()`
- More advanced input/output of images: [imageio](#)

Special functions: [scipy.special](#)

“Special” functions are functions commonly used in science and mathematics that are not considered to be “elementary” functions. Examples include

- the gamma function, `scipy.special.gamma()`,
- the error function, `scipy.special.erf()`,
- Bessel functions, such as `scipy.special.jv()` (Bessel function of the first kind), and
- elliptic functions, such as `scipy.special.ellipj()` (Jacobi elliptic functions).

Other special functions are combinations of familiar elementary functions, but they offer better accuracy or robustness than their naive implementations would.

Most of these function are computed elementwise and follow standard NumPy broadcasting rules when the input arrays have different shapes. For example, `scipy.special.xlog1py()` is mathematically equivalent to $x \log(1 + y)$.

```
x = np.asarray([1, 2])
y = np.asarray([[3], [4], [5]])
res = sp.special.xlog1py(x, y)
res.shape
```

```
(3, 2)
```

```
ref = x * np.log(1 + y)
np.allclose(res, ref)
```

```
True
```

However, `scipy.special.xlog1py()` is numerically favorable for small y , when explicit addition of `1` would lead to loss of precision due to floating point truncation error.

```
x = 2.5
y = 1e-18
x * np.log(1 + y)
```

```
np.float64(0.0)
```

```
sp.special.xlog1py(x, y)
```

```
np.float64(2.5e-18)
```

Many special functions also have “logarithmized” variants. For instance, the gamma function $\Gamma(\cdot)$ is related to the factorial function by $n! = \Gamma(n + 1)$, but it extends the domain from the positive integers to the complex plane.

```
x = np.arange(10)
np.allclose(sp.special.gamma(x + 1), sp.special.factorial(x))
```

```
True
```

```
sp.special.gamma(5) < sp.special.gamma(5.5) < sp.special.gamma(6)
```

```
np.True_
```

The factorial function grows quickly, and so the gamma function overflows for moderate values of the argument. However, sometimes only the logarithm of the gamma function is needed. In such cases, we can compute the logarithm of the gamma function directly using

```
scipy.special.gammaLn().
```

```
x = [5, 50, 500]
np.log(sp.special.gamma(x))
```

```
array([ 3.17805383, 144.56574395,          inf])
```

```
sp.special.gammaln(x)
```

```
array([ 3.17805383, 144.56574395, 2605.11585036])
```

Such functions can often be used when the intermediate components of a calculation would overflow or underflow, but the final result would not. For example, suppose we wish to compute the ratio $\Gamma(500)/\Gamma(499)$.

```
a = sp.special.gamma(500)
b = sp.special.gamma(499)
a, b
```

```
(np.float64(inf), np.float64(inf))
```

Both the numerator and denominator overflow, so performing a/b will not return the result we seek. However, the magnitude of the result should be moderate, so the use of logarithms comes to mind. Combining the identities $\log(a/b) = \log(a) - \log(b)$ and $\exp(\log(x)) = x$, we get:

```
log_a = sp.special.gammaln(500)
log_b = sp.special.gammaln(499)
log_res = log_a - log_b
res = np.exp(log_res)
res
```

```
np.float64(499.000000000006696)
```

Similarly, suppose we wish to compute the difference $\log(\Gamma(500) - \Gamma(499))$. For this, we use [`scipy.special.logsumexp\(\)`](#), which computes $\log(\exp(x) + \exp(y))$ using a numerical trick that avoids overflow.

```
res = sp.special.logsumexp([log_a, log_b],
                           b=[1, -1]) # weights the terms of the sum
res
```

```
np.float64(2605.1138443430073)
```

For more information about these and many other special functions, see the documentation of [scipy.special](#).

Linear algebra operations: [scipy.linalg](#)

[scipy.linalg](#) provides a Python interface to efficient, compiled implementations of standard linear algebra operations: the BLAS (Basic Linear Algebra Subroutines) and LAPACK (Linear Algebra PACKage) libraries.

For example, the [scipy.linalg.det\(\)](#) function computes the determinant of a square matrix:

```
arr = np.array([[1, 2],
                [3, 4]])
sp.linalg.det(arr)
```

```
np.float64(-2.0)
```

Mathematically, the solution of a linear system $Ax = b$ is $x = A^{-1}b$, but explicit inversion of a matrix is numerically unstable and should be avoided. Instead, use [scipy.linalg.solve\(\)](#):

```
A = np.array([[1, 2],
              [2, 3]])
b = np.array([14, 23])
x = sp.linalg.solve(A, b)
x
```

```
array([4., 5.])
```

```
np.allclose(A @ x, b)
```

```
True
```

Linear systems with special structure can often be solved more efficiently than more general systems. For example, systems with triangular matrices can be solved using

[scipy.linalg.solve_triangular\(\)](#):

```
A_upper = np.triu(A)
A_upper
```

```
array([[1, 2],
       [0, 3]])
```

```
np.allclose(sp.linalg.solve_triangular(A_upper, b, lower=False),
            sp.linalg.solve(A_upper, b))
```

True

[scipy.linalg](#) also features matrix factorizations/decompositions such as the singular value decomposition.

```
A = np.array([[1, 2],
              [2, 3]])
U, s, Vh = sp.linalg.svd(A)
s # singular values
```

```
array([4.23606798, 0.23606798])
```

The original matrix can be recovered by matrix multiplication of the factors:

```
S = np.diag(s) # convert to diagonal matrix before matrix multiplication
A2 = U @ S @ Vh
np.allclose(A2, A)
```

True

```
A3 = (U * s) @ Vh # more efficient: use array math broadcasting rules!
np.allclose(A3, A)
```

True

Many other decompositions (e.g. LU, Cholesky, QR), solvers for structured linear systems (e.g. triangular, circulant), eigenvalue problem algorithms, matrix functions (e.g. matrix exponential), and routines for special matrix creation (e.g. block diagonal, toeplitz) are available in [scipy.linalg](#).

Interpolation: `scipy.interpolate`

`scipy.interpolate` is used for fitting a function – an “interpolant” – to experimental or computed data. Once fit, the interpolant can be used to approximate the underlying function at intermediate points; it can also be used to compute the integral, derivative, or inverse of the function.

Some kinds of interpolants, known as “smoothing splines”, are designed to generate smooth curves from noisy data. For example, suppose we have the following data:

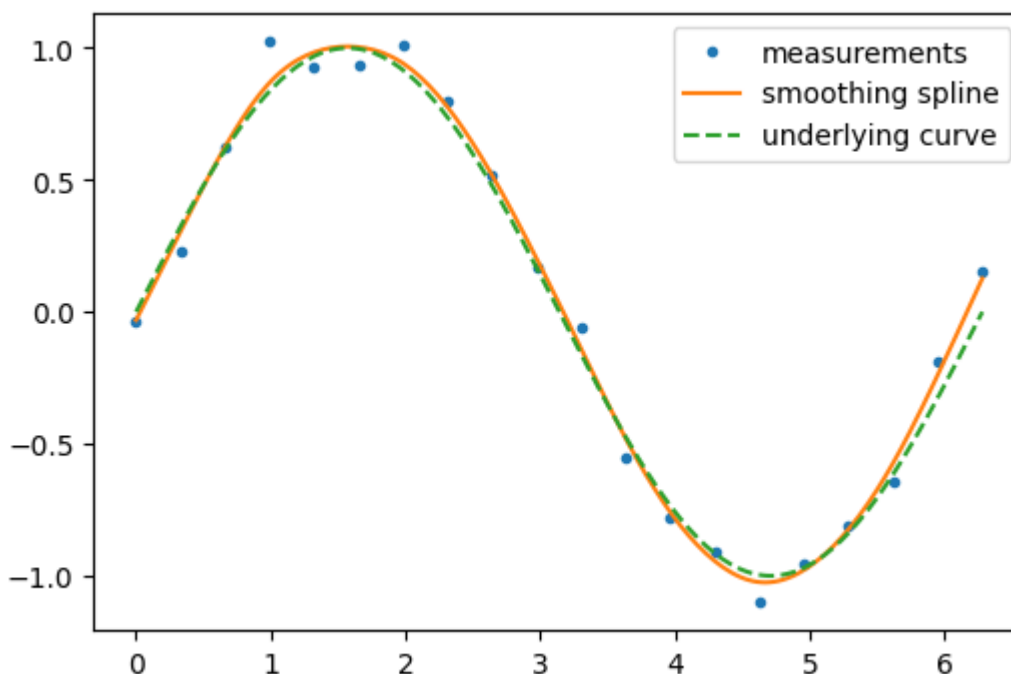
```
rng = np.random.default_rng(27446968)

measured_time = np.linspace(0, 2 * np.pi, 20)
function = np.sin(measured_time)
noise = rng.normal(loc=0, scale=0.1, size=20)
measurements = function + noise
```

`scipy.interpolate.make_smoothing_spline()` can be used to form a curve similar to the underlying sine function.

```
smoothing_spline = sp.interpolate.make_smoothing_spline(measured_time,
measurements)
interpolation_time = np.linspace(0, 2 * np.pi, 200)
smooth_results = smoothing_spline(interpolation_time)
```

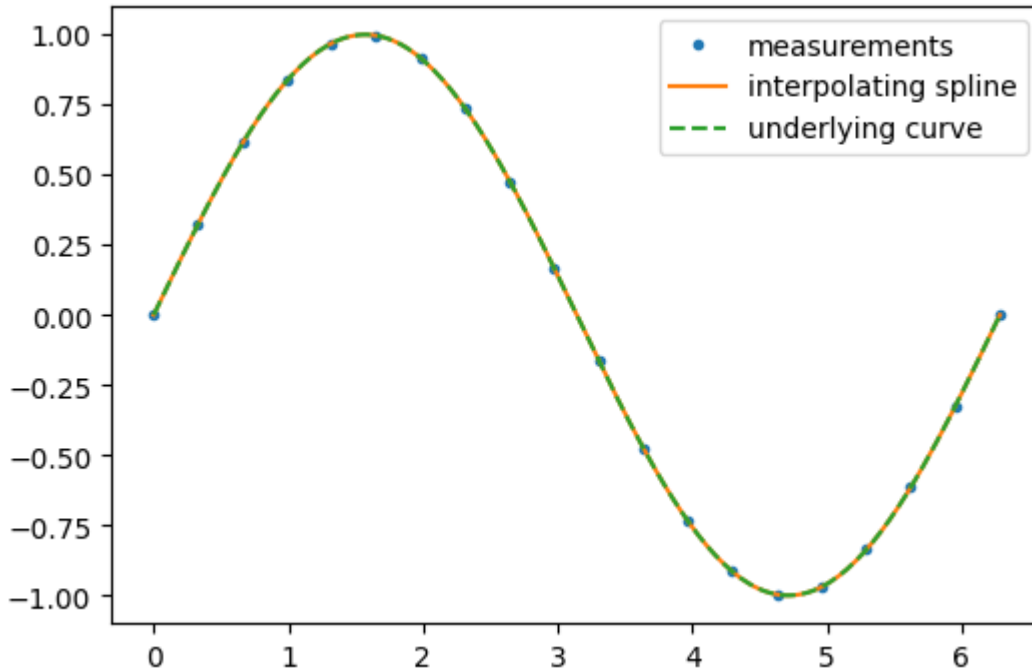
► Show code cell source



On the other hand, if the data are not noisy, it may be desirable to pass exactly through each point.

```
interp_spline = sp.interpolate.make_interp_spline(measured_time, function)
interp_results = interp_spline(interpolation_time)
```

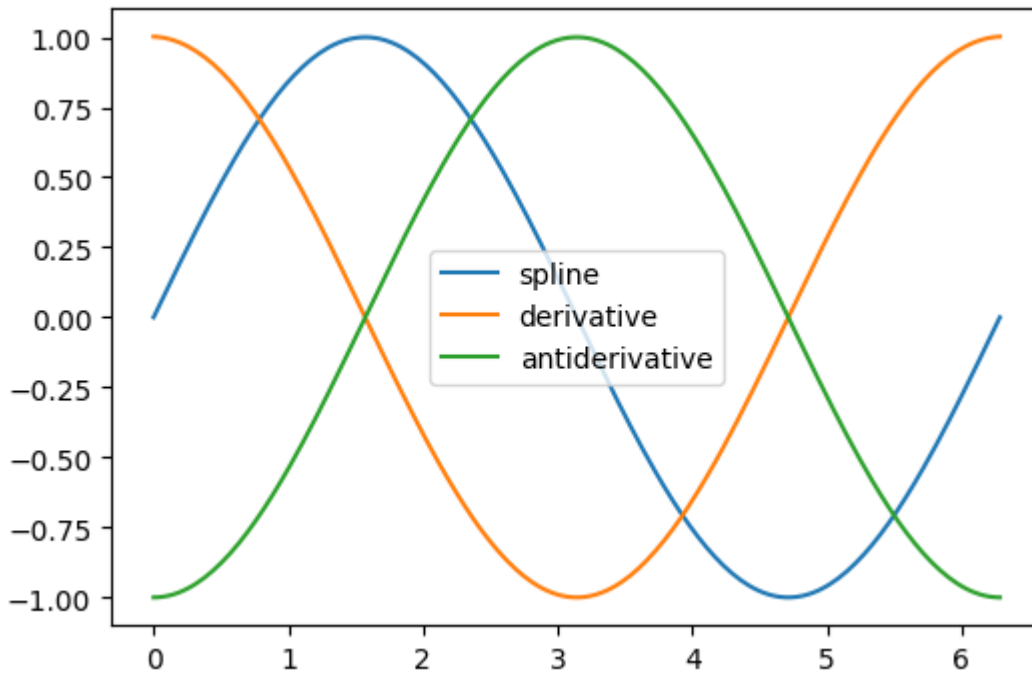
► [Show code cell source](#)



The `derivative` and `antiderivative` methods of the result object can be used for differentiation and integration. For the latter, the constant of integration is assumed to be zero, but we can “wrap” the antiderivative to include a nonzero constant of integration.

```
d_interp_spline = interp_spline.derivative()
d_interp_results = d_interp_spline(interpolation_time)
i_interp_spline = lambda t: interp_spline.antiderivative()(t) - 1
i_interp_results = i_interp_spline(interpolation_time)
```

► [Show code cell source](#)



For functions that are monotonic on an interval (e.g. $\sin$ from $\pi/2$ to $3\pi/2$), we can reverse the arguments of `make_interp_spline` to interpolate the inverse function. Because the first argument is expected to be monotonically *increasing*, we also reverse the order of elements in the arrays with `numpy.flip()`.

```
i = (measured_time > np.pi/2) & (measured_time < 3*np.pi/2)
inverse_spline = sp.interpolate.make_interp_spline(np.flip(function[i]),
                                                  np.flip(measured_time[i]))
inverse_spline(0)
```

```
array(3.14159265)
```

See the summary exercise on [Maximum wind speed prediction at the Sprogø station](#) for a more advanced spline interpolation example, and read the [SciPy interpolation tutorial](#) and the [scipy.interpolate](#) documentation for much more information.

Optimization and fit: `scipy.optimize`

`scipy.optimize` provides algorithms for root finding, curve fitting, and more general optimization.

Root Finding

`scipy.optimize.root_scalar()` attempts to find a root of a specified scalar-valued function (i.e., an argument at which the function value is zero). Like many `scipy.optimize` functions, the

function needs an initial guess of the solution, which the algorithm will refine until it converges or recognizes failure. We also provide the derivative to improve the rate of convergence.

```
def f(x):  
    return (x-1)*(x-2)  
  
def df(x):  
    return 2*x - 3  
  
x0 = 0 # guess  
res = sp.optimize.root_scalar(f, x0=x0, fprime=df)  
res
```

```
converged: True  
    flag: converged  
function_calls: 12  
iterations: 6  
    root: 1.0  
    method: newton
```

⚠ Warning

None of the functions in `scipy.optimize` that accept a guess are guaranteed to converge for all possible guesses! (For example, try `x0=1.5` in the example above, where the derivative of the function is exactly zero.) If this occurs, try a different guess, adjust the options (like providing a `bracket` as shown below), or consider whether SciPy offers a more appropriate method for the problem.

Note that only one the root at `1.0` is found. By inspection, we can tell that there is a second root at `2.0`. We can direct the function toward a particular root by changing the guess or by passing a bracket that contains only the root we seek.

```
res = sp.optimize.root_scalar(f, bracket=(1.5, 10))  
res.root
```

```
2.0
```

For multivariate problems, use `scipy.optimize.root()`.

```
def f(x):
    # intersection of unit circle and line from origin
    return [x[0]**2 + x[1]**2 - 1,
            x[1] - x[0]]

res = sp.optimize.root(f, x0=[0, 0])
np.allclose(f(res.x), 0, atol=1e-10)
```

True

```
np.allclose(res.x, np.sqrt(2)/2)
```

True

Over-constrained problems can be solved in the least-squares sense using

`scipy.optimize.root()` with `method='lm'` (Levenberg-Marquardt).

```
def f(x):
    # intersection of unit circle, line from origin, and parabola
    return [x[0]**2 + x[1]**2 - 1,
            x[1] - x[0],
            x[1] - x[0]**2]

res = sp.optimize.root(f, x0=[1, 1], method='lm')
res.success
```

True

```
res.x
```

```
array([0.76096066, 0.66017736])
```

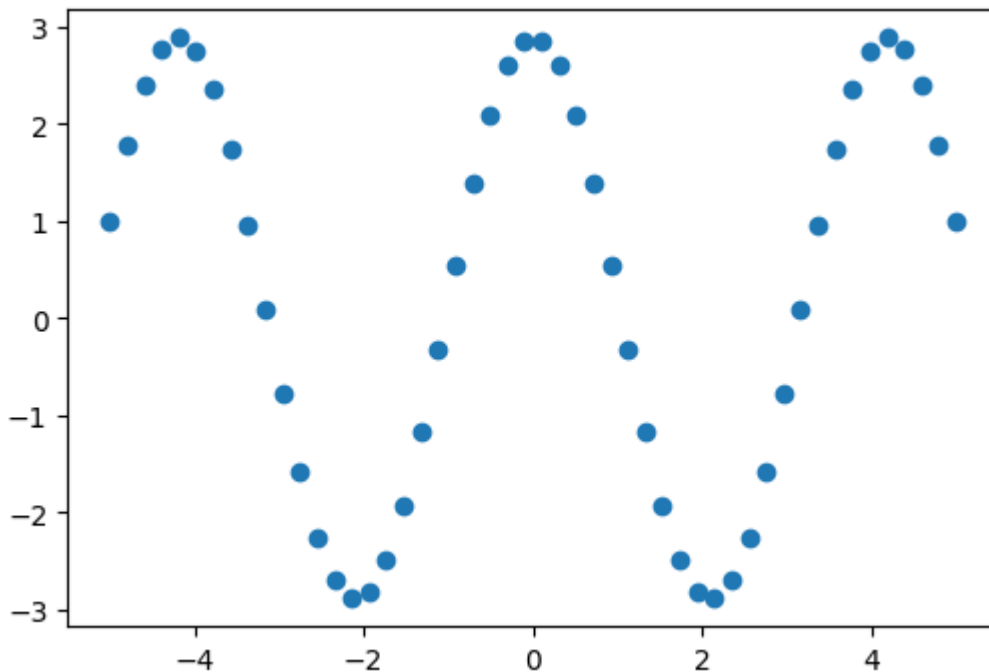
See the documentation of `scipy.optimize.root_scalar()` and `scipy.optimize.root()` for a variety of other solution algorithms and options.

Curve fitting

Suppose we have data that is sinusoidal but noisy:

```
x_data = np.linspace(-5, 5, num=50) # 50 values between -5 and 5
noise = 0.01 * np.cos(100 * x_data)
a, b = 2.9, 1.5
y_data = a * np.cos(b * x_data) + noise
```

► Show code cell source



We can approximate the underlying amplitude, frequency, and phase from the data by least squares curve fitting. To begin, we write a function that accepts the independent variable as the first argument and all parameters to fit as separate arguments:

```
def f(x, a, b, c):
    return a * np.sin(b * x + c)
```

We then use `scipy.optimize.curve_fit()` to find a and b :

```
params, _ = sp.optimize.curve_fit(f, x_data, y_data, p0=[2, 1, 3])
params
```

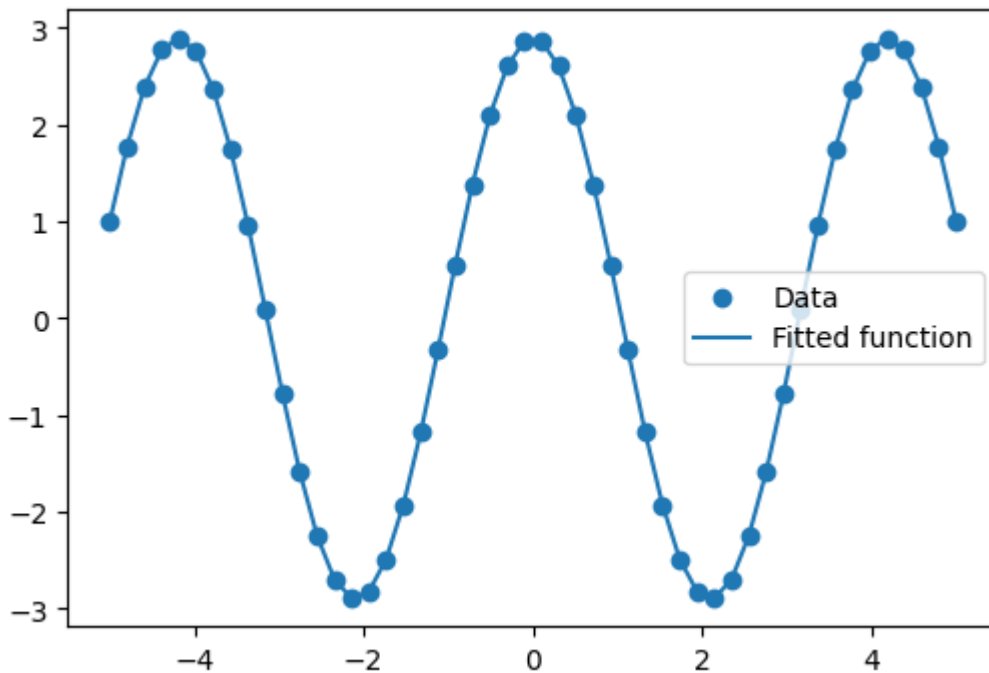
```
array([2.900026 , 1.50012043, 1.57079633])
```

```
ref = [a, b, np.pi/2] # what we'd expect
np.allclose(params, ref, rtol=1e-3)
```

True

We plot the resulting curve on the data:

► Show code cell source



Exercise 39



The temperature extremes in Alaska for each month, starting in January, are given by (in degrees Celsius):

```
max: 17, 19, 21, 28, 33, 38, 37, 37, 31, 23, 19, 18
min: -62, -59, -56, -46, -32, -18, -9, -13, -25, -46, -52, -58
```

1. Plot these temperature extremes.
2. Define a function that can describe min and max temperatures. Hint: this function has to have a period of 1 year. Hint: include a time offset.
3. Fit this function to the data with `scipy.optimize.curve_fit()`.
4. Plot the result. Is the fit reasonable? If not, why?
5. Is the time offset for min and max temperatures the same within the fit accuracy?

Solution to [Exercise 39](#)



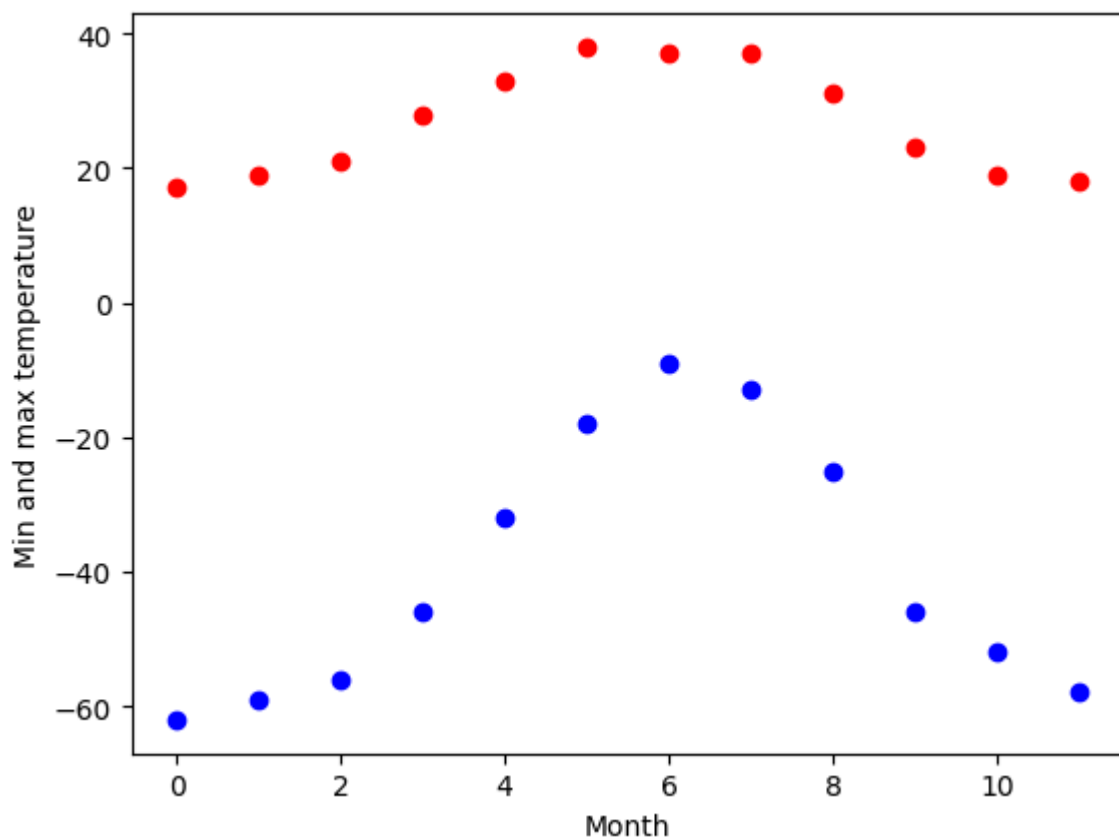
Curve fitting: temperature as a function of month of the year

We have the min and max temperatures in Alaska for each months of the year. We would like to find a function to describe this yearly evolution.

For this, we will fit a periodic function.

```
# The data
temp_max = np.array([17, 19, 21, 28, 33, 38, 37, 37, 31, 23, 19, 18])
temp_min = np.array([-62, -59, -56, -46, -32, -18, -9, -13, -25, -46, -52, -58])

months = np.arange(12)
plt.plot(months, temp_max, "ro")
plt.plot(months, temp_min, "bo")
plt.xlabel("Month")
plt.ylabel("Min and max temperature");
```



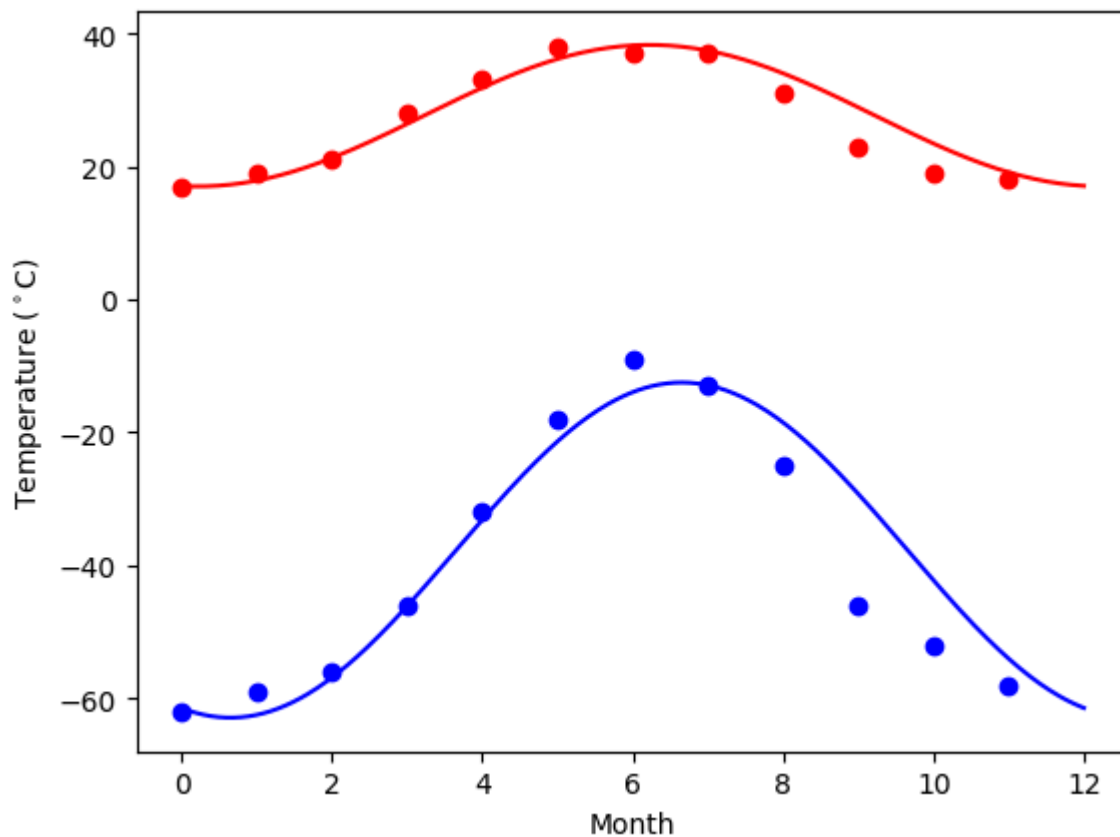
Fitting it to a periodic function:

```
def yearly_temps(times, avg, ampl, time_offset):
    return avg + ampl * np.cos((times + time_offset) * 2 * np.pi /
times.max())

res_max, cov_max = sp.optimize.curve_fit(yearly_temps, months,
temp_max, [20, 10, 0])
res_min, cov_min = sp.optimize.curve_fit(yearly_temps, months,
temp_min, [-40, 20, 0])
```

Plotting the fit

```
days = np.linspace(0, 12, num=365)
plt.figure()
plt.plot(months, temp_max, "ro")
plt.plot(days, yearly_temps(days, *res_max), "r-")
plt.plot(months, temp_min, "bo")
plt.plot(days, yearly_temps(days, *res_min), "b-")
plt.xlabel("Month")
plt.ylabel(r"Temperature ( $^{\circ}\text{C}$ )");
```



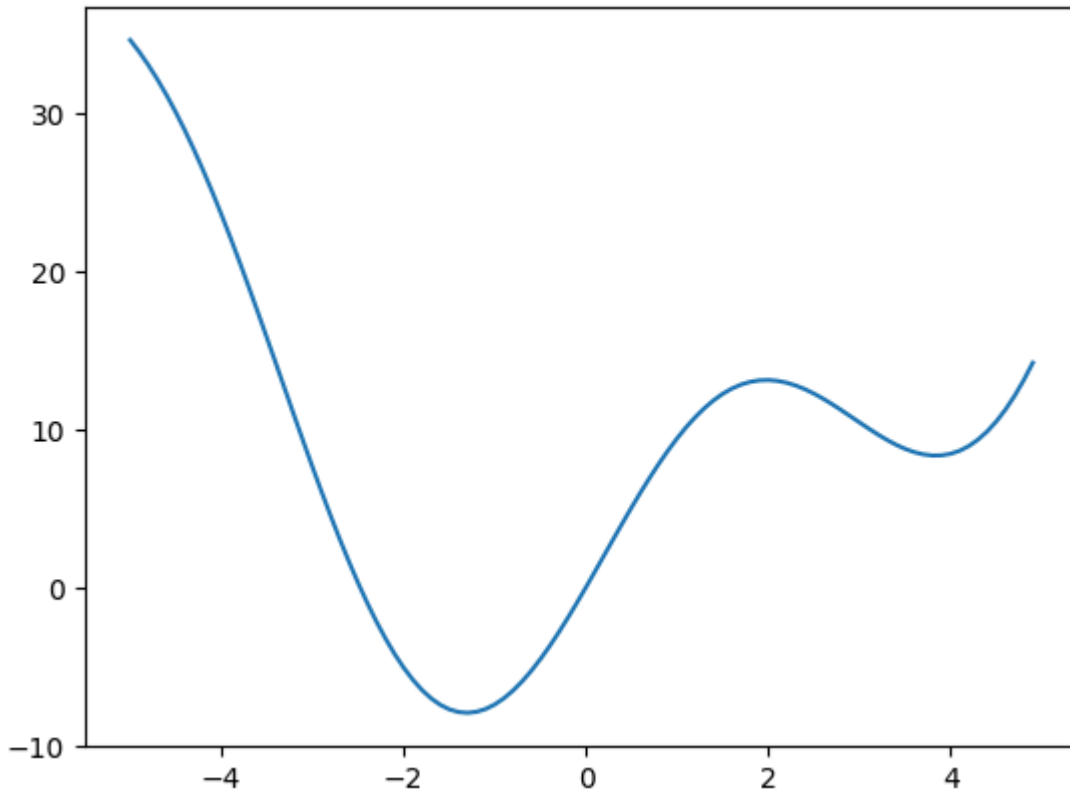
Optimization

Suppose we wish to minimize the scalar-valued function of a single variable

$$f(x) = x^2 + 10 \sin(x):$$

```
def f(x):  
    return x**2 + 10 * np.sin(x)  
  
x = np.arange(-5, 5, 0.1)  
plt.plot(x, f(x))
```

```
[<matplotlib.lines.Line2D at 0x7fb384eecd70>]
```



We can see that the function has a local minimizer near $x = 3.8$ and a global minimizer near $x = -1.3$, but the precise values cannot be determined from the plot.

The most appropriate function for this purpose is `scipy.optimize.minimize_scalar()`. Since we know the approximate locations of the minima, we will provide bounds that restrict the search to the vicinity of the global minimum.

```
res = sp.optimize.minimize_scalar(f, bounds=(-2, -1))  
res
```

```
message: Solution found.  
success: True  
status: 0  
  fun: -7.9458233756095895  
   x: -1.3064409970312618  
  nit: 8  
 nfev: 8
```

```
res.fun == f(res.x)
```

```
np.True_
```

If we did not already know the approximate location of the global minimum, we could use one of SciPy's global minimizers, such as `scipy.optimize.differential_evolution()`. We are required to pass `bounds`, but they do not need to be tight.

```
bounds=[(-5, 5)] # list of lower, upper bound for each variable
res = sp.optimize.differential_evolution(f, bounds=bounds)
res
```

```
message: Optimization terminated successfully.
success: True
      fun: -7.945823375615283
       x: [-1.306e+00]
      nit: 8
     nfev: 143
population: [[-1.305e+00]
             [-1.291e+00]
             ...
             [-1.321e+00]
             [-1.297e+00]]
population_energies: [-7.946e+00 -7.944e+00 ... -7.945e+00 -7.945e+00]
              jac: [-8.882e-08]
```

For multivariate optimization, a good choice for many problems is `scipy.optimize.minimize()`. Suppose we wish to find the minimum of a quadratic function of two variables,

$$f(x_0, x_1) = (x_0 - 1)^2 + (x_1 - 2)^2.$$

```
def f(x):
    return (x[0] - 1)**2 + (x[1] - 2)**2
```

Like `scipy.optimize.root()`, `scipy.optimize.minimize()` requires a guess `x0`. (Note that this is the initial value of *both* variables rather than the value of the variable we happened to label x_0 .)

```
res = sp.optimize.minimize(f, x0=[0, 0])
res
```



```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.705780445775116e-16
   x: [ 1.000e+00  2.000e+00]
  nit: 2
  jac: [ 3.219e-09 -8.462e-09]
hess_inv: [[ 9.000e-01 -2.000e-01]
            [-2.000e-01  6.000e-01]]
 nfev: 9
 njev: 3
```

This barely scratches the surface of SciPy's optimization features, which include mixed integer linear programming, constrained nonlinear programming, and the solution of assignment problems. For much more information, see the documentation of `scipy.optimize` and the advanced chapter [Mathematical optimization: finding minima of functions](#).

Maximization?

Is `scipy.optimize.minimize()` restricted to the solution of minimization problems? Nope! To solve a maximization problem, simply minimize the *negative* of the original objective function.

Exercise 40

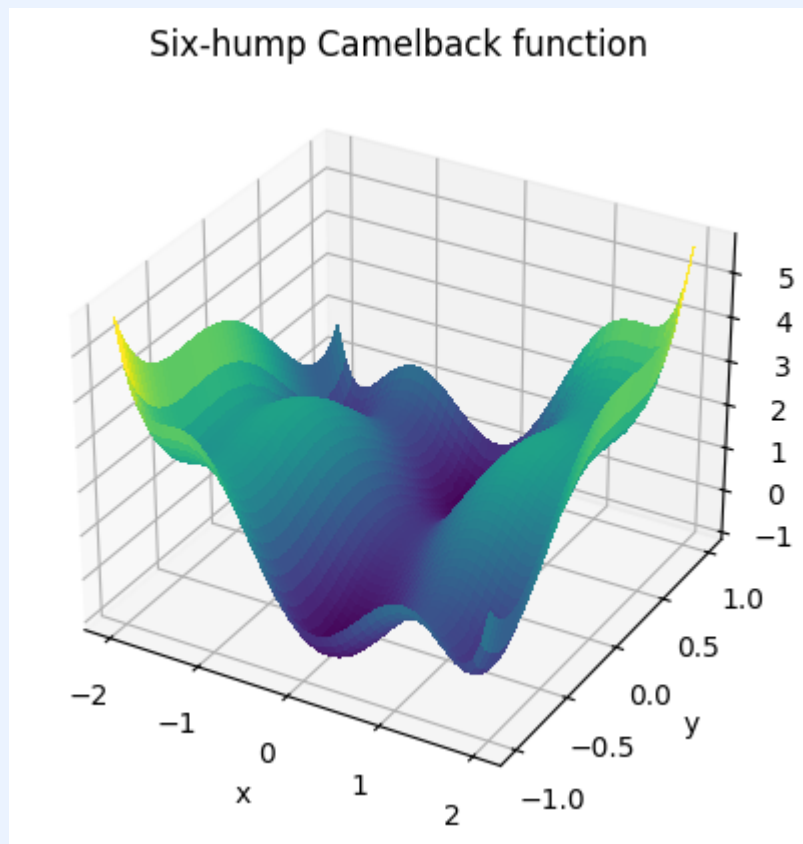
This is an exercise on 2-D minimization.

The six-hump camelback function

$$f(x, y) = (4 - 2.1x^2 + \frac{x^4}{3})x^2 + xy + (4y^2 - 4)y^2$$

has multiple local minima. Find a global minimum (there is more than one, each with the same value of the objective function) and at least one other local minimum.

Here's a plot of the function (taken from the exercise solution):



Hints:

- Variables can be restricted to $-2 < x < 2$ and $-1 < y < 1$.
- `numpy.meshgrid()` and `matplotlib.pyplot.imshow()` can help with visualization.
- Try minimizing with `scipy.optimize.minimize()` with an initial guess of $(x, y) = (0, 0)$. Does it find the global minimum, or converge to a local minimum? What about other initial guesses?
- Try minimizing with `scipy.optimize.differential_evolution()`.

Solution to [Exercise 40](#)

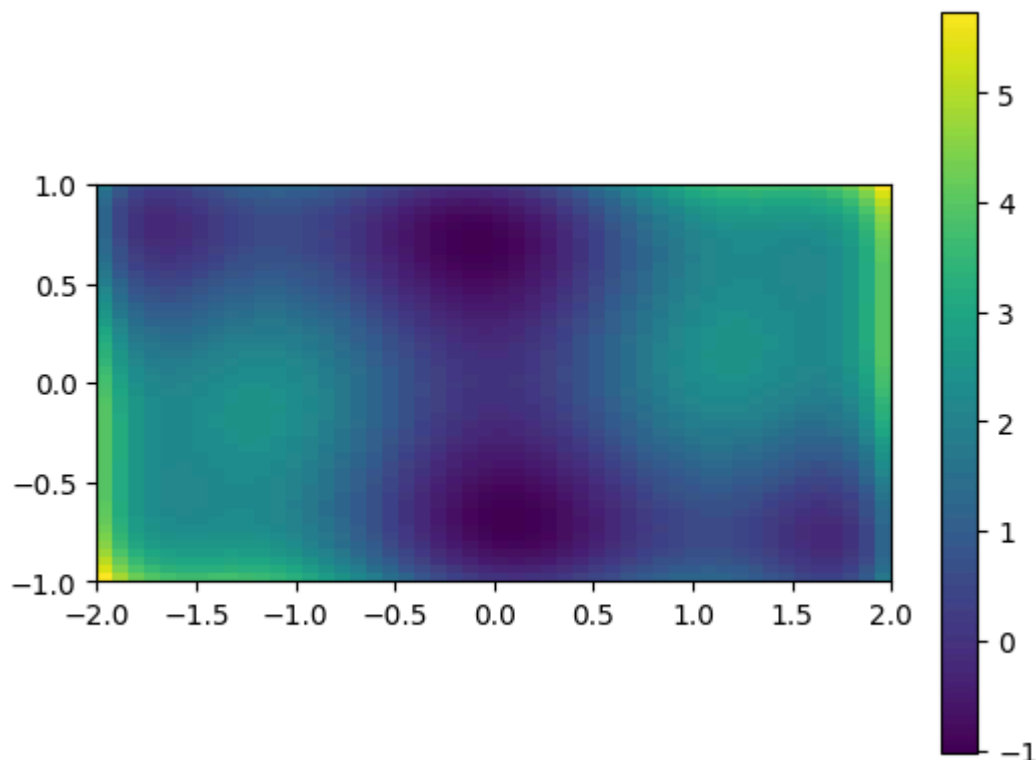
Optimization of a two-parameter function:

```
# Define the function that we are interested in
def sixhump(x):
    return (
        (4 - 2.1 * x[0] ** 2 + x[0] ** 4 / 3) * x[0] ** 2
        + x[0] * x[1]
        + (-4 + 4 * x[1] ** 2) * x[1] ** 2
    )

# Make a grid to evaluate the function (for plotting)
xlim = [-2, 2]
ylim = [-1, 1]
x = np.linspace(*xlim) # type: ignore[call-overload]
y = np.linspace(*ylim) # type: ignore[call-overload]
xg, yg = np.meshgrid(x, y)
```

A 2D image plot of the function:

```
# Simple visualization in 2D
plt.figure()
plt.imshow(sixhump([xg, yg]), extent=xlim + ylim, origin="lower") #
type: ignore[arg-type]
plt.colorbar();
```



A 3D surface plot of the function:

```

from mpl_toolkits.mplot3d import Axes3D

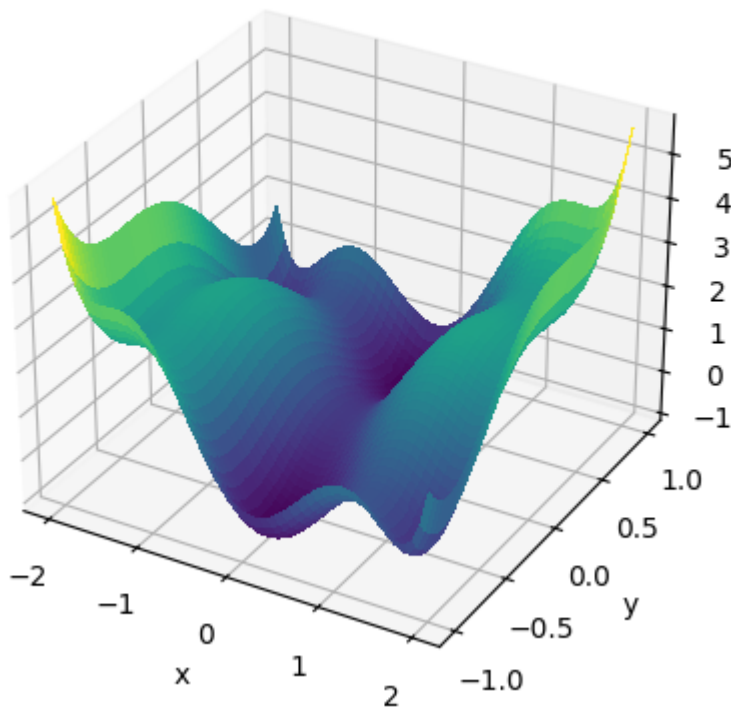
fig = plt.figure()
ax: Axes3D = fig.add_subplot(111, projection="3d")
surf = ax.plot_surface(
    xg,
    yg,
    sixhump([xg, yg]),
    rstride=1,
    cstride=1,
    cmap="viridis",
    linewidth=0,
    antialiased=False,
)

ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("f(x, y)")
ax.set_title("Six-hump Camelback function");

# You can ignore the code below - it's not part of the solution. It is
# only to
# allow us to use the plot from the solution as a graphic in the web
# page.
from myst_nb import glue
glue("plot_camel", fig, display=False)

```

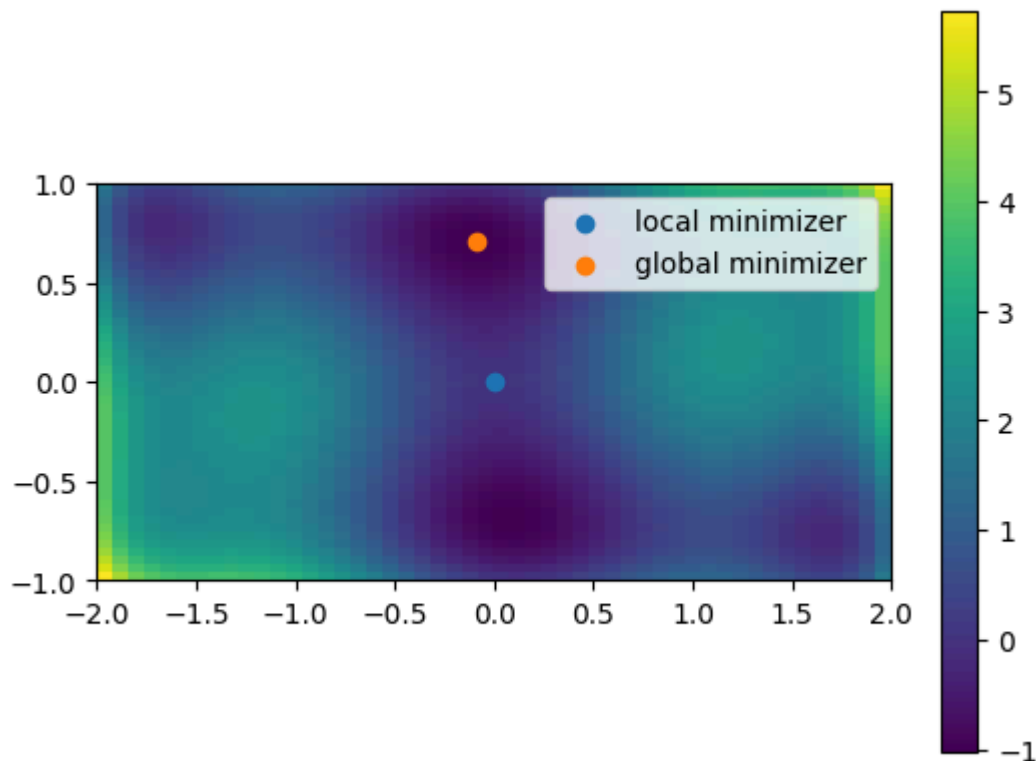
Six-hump Camelback function



Find minima:

```
# local minimization
res_local = sp.optimize.minimize(sixhump, x0=[0, 0])
# global minimization
res_global = sp.optimize.differential_evolution(sixhump, bounds=[xlim,
ylim])
```

► Show code cell source



See the summary exercise on [Non linear least squares curve fitting: application to point extraction in topographical lidar data](#) for another, more advanced example.

Statistics and random numbers: [scipy.stats](#)

[scipy.stats](#) contains fundamental tools for statistics in Python.

Statistical Distributions

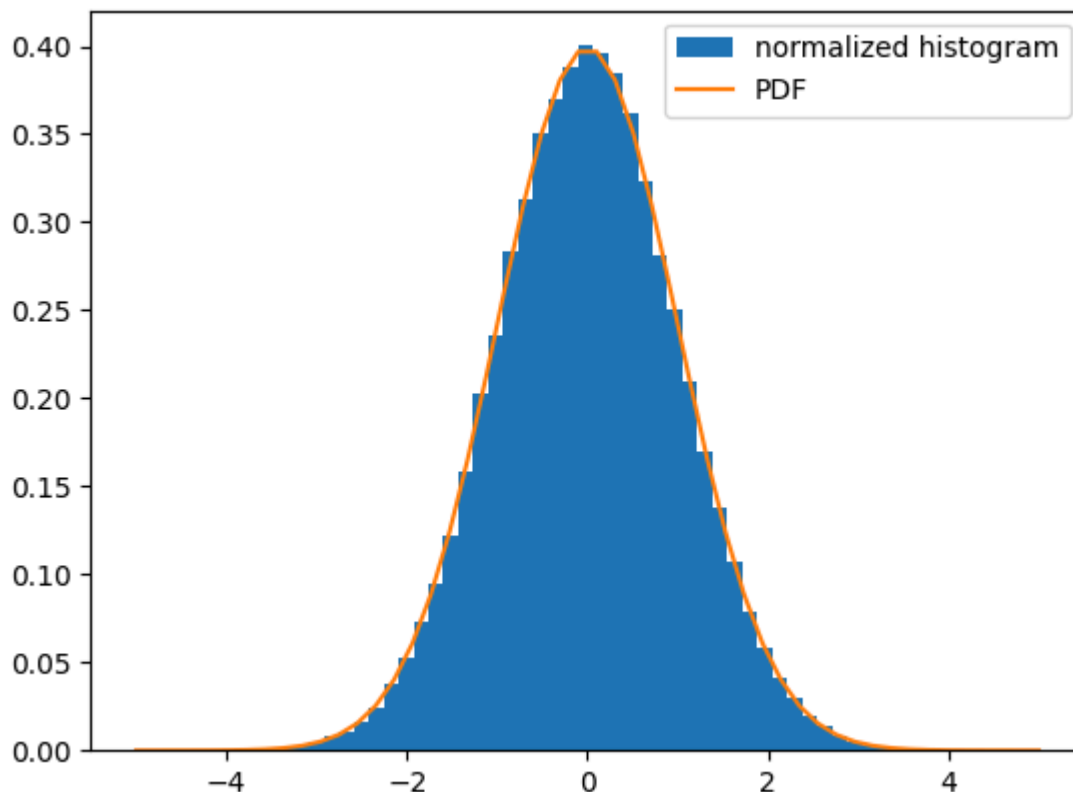
Consider a random variable distributed according to the standard normal. We draw a sample consisting of 100000 observations from the random variable. The normalized histogram of the sample is an estimator of the random variable's probability density function (PDF):

```

dist = sp.stats.norm(loc=0, scale=1) # standard normal distribution
sample = dist.rvs(size=100000) # "random variate sample"
plt.hist(sample, bins=50, density=True, label='normalized histogram')
x = np.linspace(-5, 5)
plt.plot(x, dist.pdf(x), label='PDF')
plt.legend()

```

<matplotlib.legend.Legend at 0x7fb37be24ce0>



Suppose we knew that the sample had been drawn from a distribution belonging to the family of normal distributions, but we did not know the particular distribution's location (mean) and scale (standard deviation). We perform maximum likelihood estimation of the unknown parameters using the distribution family's `fit` method:

Distribution objects and frozen distributions

Each of the 100+ `scipy.stats` distribution families is represented by an *object* with a `__call__` method. Here, we call the `scipy.stats.norm` object to specify its location and scale, and it returns a *frozen* distribution: a particular element of a distribution family with all parameters fixed. The frozen distribution object has methods to compute essential

functions of the particular distribution.

```
loc, scale = sp.stats.norm.fit(sample)
loc
```

```
np.float64(0.0028075448237651138)
```

```
scale
```

```
np.float64(0.9979740193044766)
```

Since we know the true parameters of the distribution from which the sample was drawn, we are not surprised that these estimates are similar.

Exercise 41

Generate 1000 random variates from a gamma distribution with a shape parameter of 1. *Hint: the shape parameter is passed as the first argument when freezing the distribution.* Plot the histogram of the sample, and overlay the distribution's PDF. Estimate the shape parameter from the sample using the `fit` method.

Extra: the distributions have many useful methods. Explore them using tab completion. Plot the cumulative density function of the distribution, and compute the variance.

Sample Statistics and Hypothesis Tests

The sample mean is an estimator of the mean of the distribution from which the sample was drawn:

```
np.mean(sample)
```

```
np.float64(0.0028075448237651138)
```

NumPy includes some of the most fundamental sample statistics (e.g. `numpy.mean()`, `numpy.var()`, `numpy.percentile()`); `scipy.stats` includes many more. For instance, the geometric mean is a common measure of central tendency for data that tends to be distributed over many orders of magnitude.

```
sp.stats.gmean(2**sample)
```

```
np.float64(1.0019479365470911)
```

SciPy also includes a variety of hypothesis tests that produce a sample statistic and a p-value. For instance, suppose we wish to test the null hypothesis that `sample` was drawn from a normal distribution:

```
res = sp.stats.normaltest(sample)
res.statistic
```

```
np.float64(0.8411130312432809)
```

```
res.pvalue
```

```
np.float64(0.6566812647236747)
```

Here, `statistic` is a sample statistic that tends to be high for samples that are drawn from non-normal distributions. `pvalue` is the probability of observing such a high value of the statistic for a sample that *has* been drawn from a normal distribution. If the p-value is unusually small, this may be taken as evidence that `sample` was *not* drawn from the normal distribution. Our statistic and p-value are moderate, so the test is inconclusive.

There are many other features of `scipy.stats`, including circular statistics, quasi-Monte Carlo methods, and resampling methods. For much more information, see the documentation of `scipy.stats` and the advanced chapter [statistics](#).

Numerical integration: `scipy.integrate`

Quadrature

Suppose we wish to compute the definite integral $\int_0^{\pi/2} \sin(t) dt$ numerically.

`scipy.integrate.quad()` chooses one of several adaptive techniques depending on the parameters, and is therefore the recommended first choice for integration of function of a single variable:


```
integral, error_estimate = sp.integrate.quad(np.sin, 0, np.pi / 2)
np.allclose(integral, 1) # numerical result ~ analytical result
```

True

```
abs(integral - 1) < error_estimate # actual error < estimated error
```

True

Other functions for *numerical quadrature*, including integration of multivariate functions and approximating integrals from samples, are available in [scipy.integrate](#).

Initial Value Problems

[scipy.integrate](#) also features routines for integrating [Ordinary Differential Equations \(ODE\)](#). For example, [scipy.integrate.solve_ivp\(\)](#) integrates ODEs of the form:

$$\frac{dy}{dt} = f(t, y(t))$$

from an initial time t_0 and initial state $y(t = t_0) = y_0$ to a final time t_f or until an event occurs (e.g. a specified state is reached).

As an introduction, consider the initial value problem given by $\frac{dy}{dt} = -2y$ and the initial condition $y(t = 0) = 1$ on the interval $t = 0 \dots 4$. We begin by defining a callable that computes $f(t, y(t))$ given the current time and state.

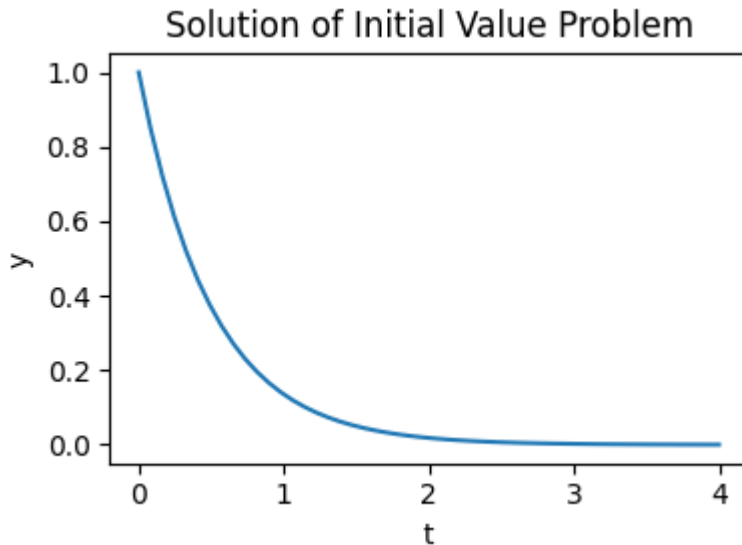
```
def f(t, y):
    return -2 * y
```

Then, to compute y as a function of time:

```
t_span = (0, 4) # time interval
t_eval = np.linspace(*t_span) # times at which to evaluate `y`
y0 = [1,] # initial state
res = sp.integrate.solve_ivp(f, t_span=t_span, y0=y0, t_eval=t_eval)
```

and plot the result:

```
plt.figure(figsize=(4, 3))
plt.plot(res.t, res.y[0])
plt.xlabel('t')
plt.ylabel('y')
plt.title('Solution of Initial Value Problem')
plt.tight_layout();
```



Let us integrate a more complex ODE: a [damped spring-mass oscillator](#). The position of a mass attached to a spring obeys the 2nd order ODE $\ddot{y} + 2\zeta\omega_0\dot{y} + \omega_0^2 y = 0$ with natural frequency $\omega_0 = \sqrt{k/m}$, damping ratio $\zeta = c/(2m\omega_0)$, spring constant k , mass m , and damping coefficient c .

Before using `scipy.integrate.solve_ivp()`, the 2nd order ODE needs to be transformed into a system of first-order ODEs. Note that

$$\frac{dy}{dt} = \dot{y} \frac{d\dot{y}}{dt} = \ddot{y} = -(2\zeta\omega_0\dot{y} + \omega_0^2 y)$$

If we define $z = [z_0, z_1]$ where $z_0 = y$ and $z_1 = \dot{y}$, then the first order equation:

$$\frac{dz}{dt} = \begin{bmatrix} \frac{dz_0}{dt} \\ \frac{dz_1}{dt} \end{bmatrix} = \begin{bmatrix} z_1 \\ -(2\zeta\omega_0 z_1 + \omega_0^2 z_0) \end{bmatrix}$$

is equivalent to the original second order equation.

We set:

```

m = 0.5 # kg
k = 4 # N/m
c = 0.4 # N s/m
zeta = c / (2 * m * np.sqrt(k/m))
omega = np.sqrt(k / m)

```

and define the function that computes $\dot{z} = f(t, z(t))$:

```

def f(t, z, zeta, omega):
    return (z[1], -2.0 * zeta * omega * z[1] - omega**2 * z[0])

```

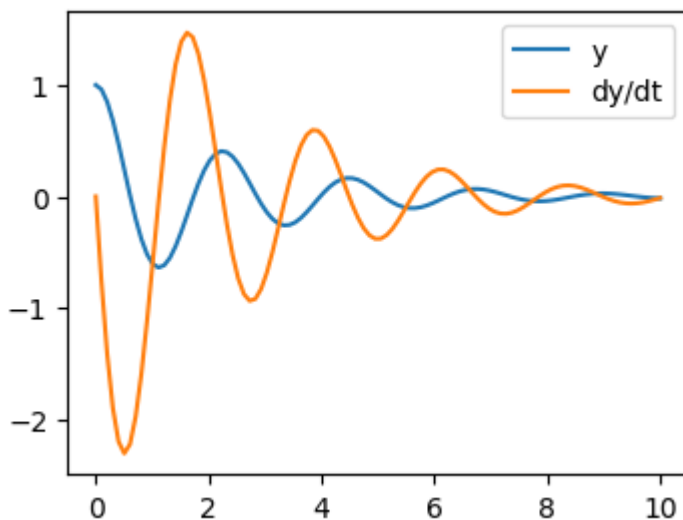
Integration of the system follows:

```

t_span = (0, 10)
t_eval = np.linspace(*t_span, 100)
z0 = [1, 0]
res = sp.integrate.solve_ivp(f, t_span, z0, t_eval=t_eval,
                             args=(zeta, omega), method='LSODA')

```

► Show code cell source



Note

With the option `method='LSODA'`, `scipy.integrate.solve_ivp()` uses the LSODA (Livermore Solver for Ordinary Differential equations with Automatic method switching for stiff and non-stiff problems). See the [ODEPACK Fortran library] for more details.

See also

Partial Differential Equations

There is no Partial Differential Equations (PDE) solver in SciPy. Some Python packages for solving PDE's are available, such as [fipy] or [SfePy].

Fast Fourier transforms: `scipy.fft`

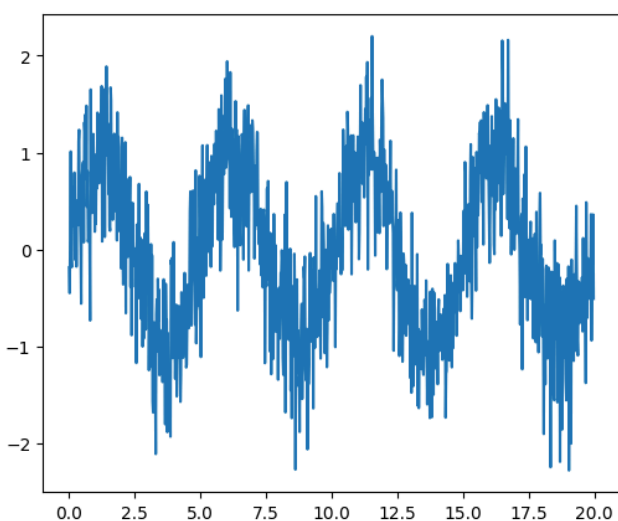
The `scipy.fft` module computes fast Fourier transforms (FFTs) and offers utilities to handle them. Some important functions are:

- `scipy.fft.fft()` to compute the FFT
- `scipy.fft.fftfreq()` to generate the sampling frequencies
- `scipy.fft.ifft()` to compute the inverse FFT, from frequency space to signal space

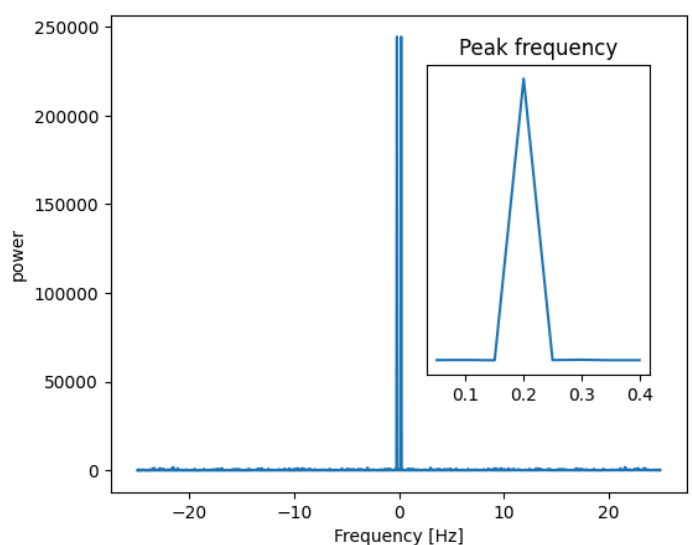
As an illustration, a example (noisy) input signal (`sig`), and its FFT:

```
# Time.
dt = 0.02 # Time step.
t = np.arange(0, 20, dt) # Time vector.
# An example noisy signal over time.
sig = np.sin(2 * np.pi / 5.0 * t) + 0.5 * rng.normal(size=t.size)
# FFT of signal.
sig_fft = sp.fft.fft(sig)
# Corresponding frequencies.
freqs = sp.fft.fftfreq(sig.size, d=dt)
```

Signal



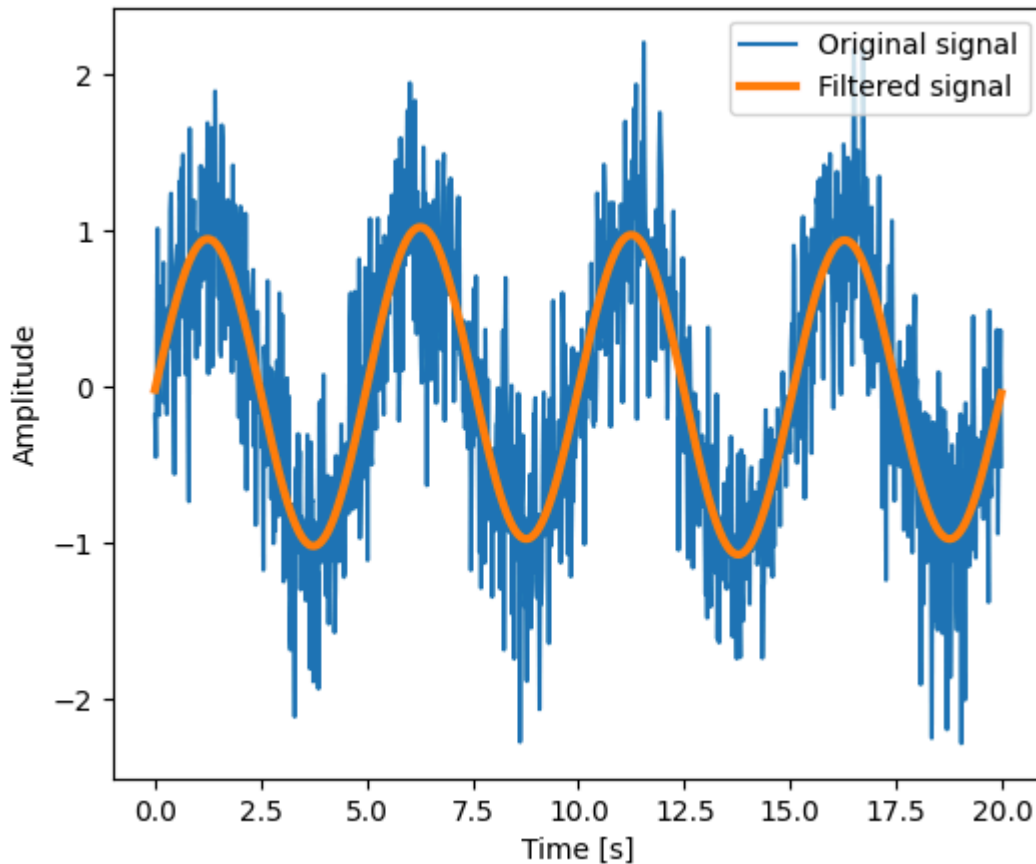
FFT



The peak signal frequency can be found with `freqs[power.argmax()]`.

The code of this example and the figures above can be found in the [SciPy FFT example](#).

Setting the Fourier component above this frequency to zero and inverting the FFT with `scipy.fft.ifft()`, gives a filtered signal (see the [example](#) for detail).



`numpy.fft`

NumPy also has an implementation of FFT (`numpy.fft`). However, the SciPy one should be preferred, as it uses more efficient underlying implementations.

Fully worked examples:

Crude periodicity finding

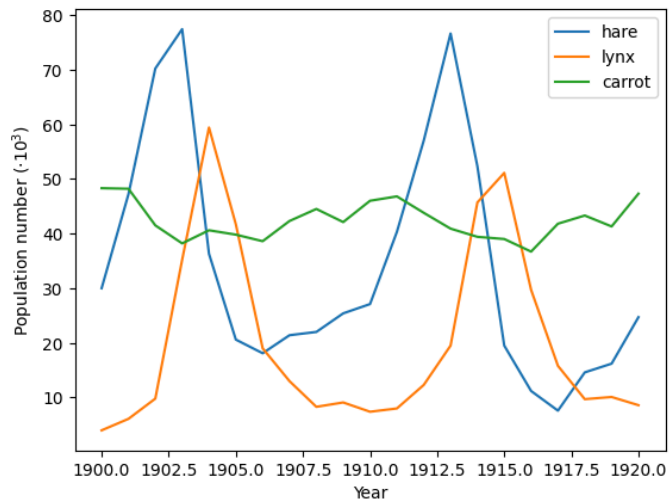
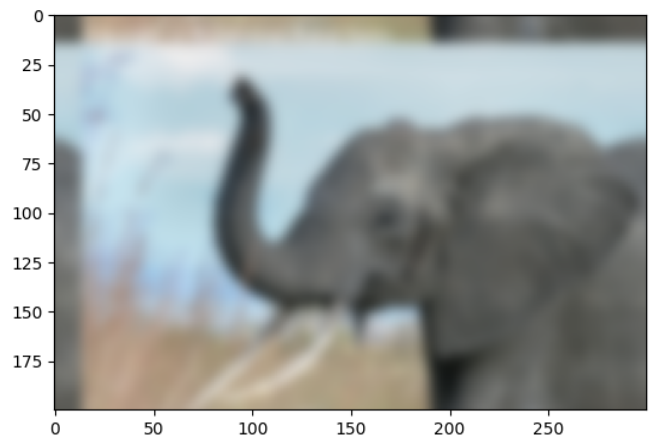
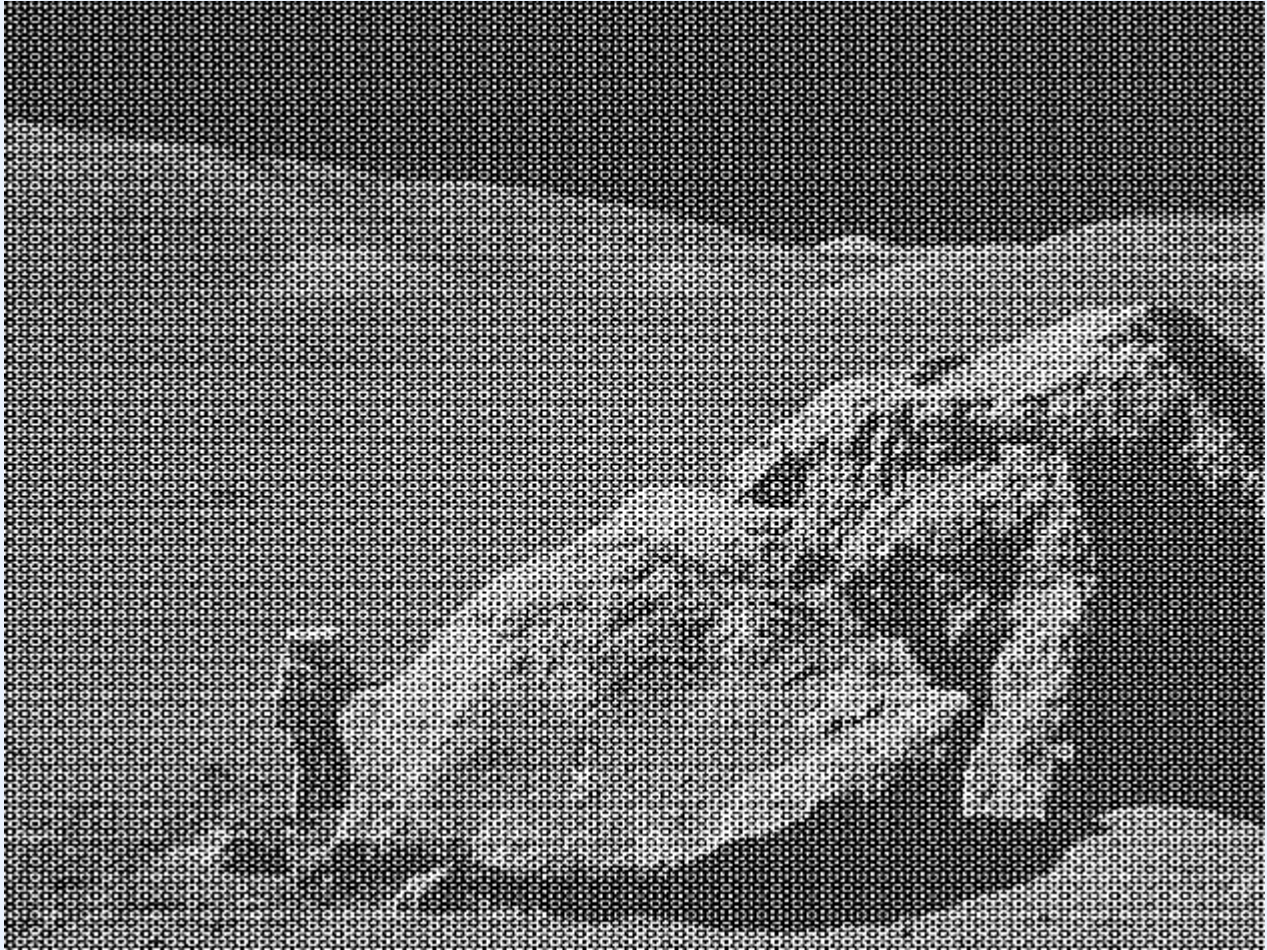


Image blur with FFT



Exercise 42



1. Examine the provided image [moonlanding.png](#), which is heavily contaminated with periodic noise. In this exercise, we aim to clean up the noise using the Fast Fourier Transform.
2. Load the image using `matplotlib.pyplot.imread()`.
3. Find and use the 2-D FFT function in `scipy.fft`, and plot the spectrum (Fourier transform of) the image. Do you have any trouble visualising the spectrum? If so, why?
4. The spectrum consists of high and low frequency components. The noise is contained in the high-frequency part of the spectrum, so set some of those components to zero (use array slicing).
5. Apply the inverse Fourier transform to see the resulting image.

Solution to [Exercise 42](#)

Implementing image denoising with FFT.

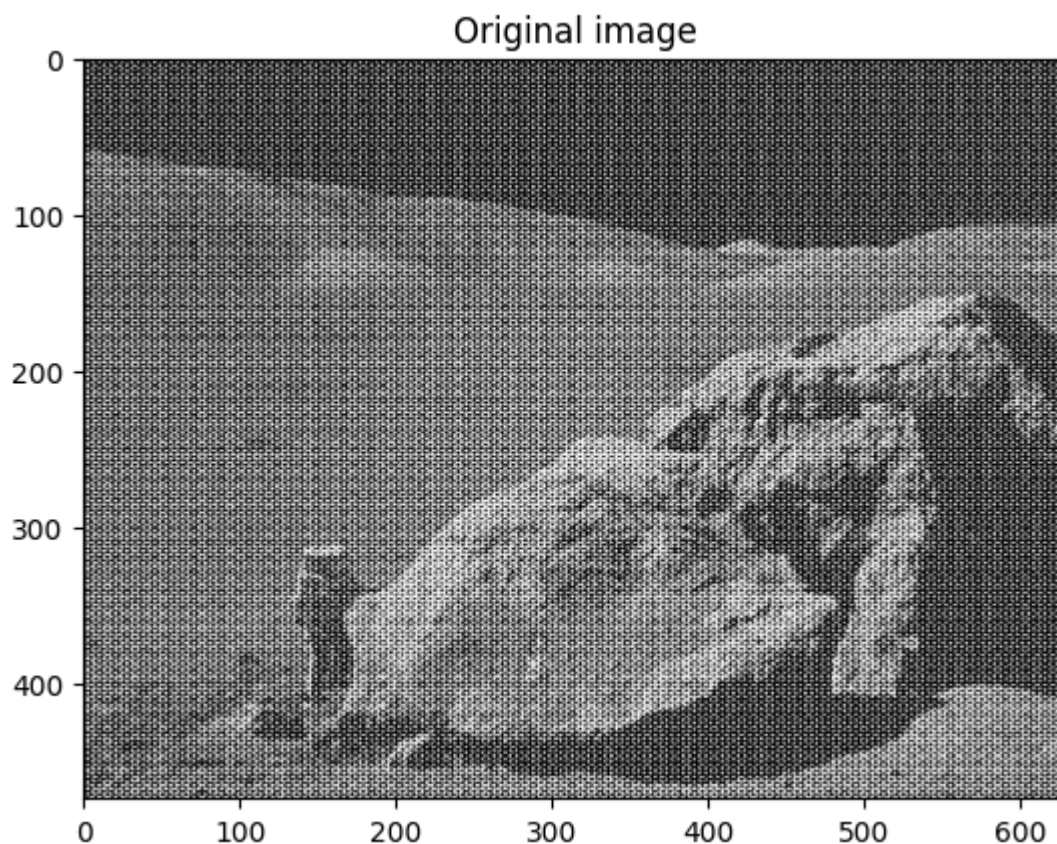
Denoise an image ([data/moonlanding.png](#)) by implementing a blur with an FFT.

Implements, via FFT, the following convolution:

$$f_1(t) = \int dt' K(t - t') f_0(t')$$
$$\tilde{f}_1(\omega) = \tilde{K}(\omega) \tilde{f}_0(\omega)$$

```
# Read and plot the image
im = plt.imread("data/moonlanding.png").astype(float)

plt.figure()
plt.imshow(im, "gray")
plt.title("Original image");
```




```

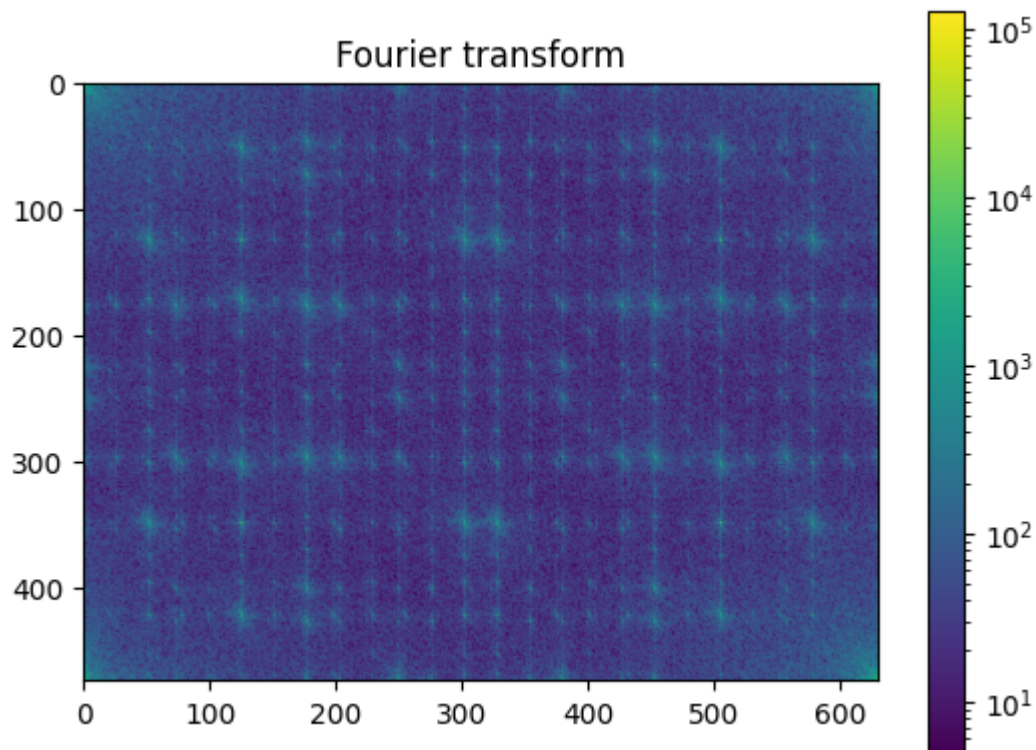
# Compute the 2d FFT of the input image
im_fft = sp.fft.fft2(im)

# Show the results
from matplotlib.colors import LogNorm

def plot_spectrum(im_fft):
    # A logarithmic colormap
    plt.imshow(np.abs(im_fft), norm=LogNorm(vmin=5))
    plt.colorbar()

plt.figure()
plot_spectrum(im_fft)
plt.title("Fourier transform");

```



Filter in FFT:

In the lines following, we'll make a copy of the original spectrum and truncate coefficients.

```

# Define the fraction of coefficients (in each direction) we keep
keep_fraction = 0.1

# Call ff a copy of the original transform. NumPy arrays have a copy
# method for this purpose.
im_fft2 = im_fft.copy()

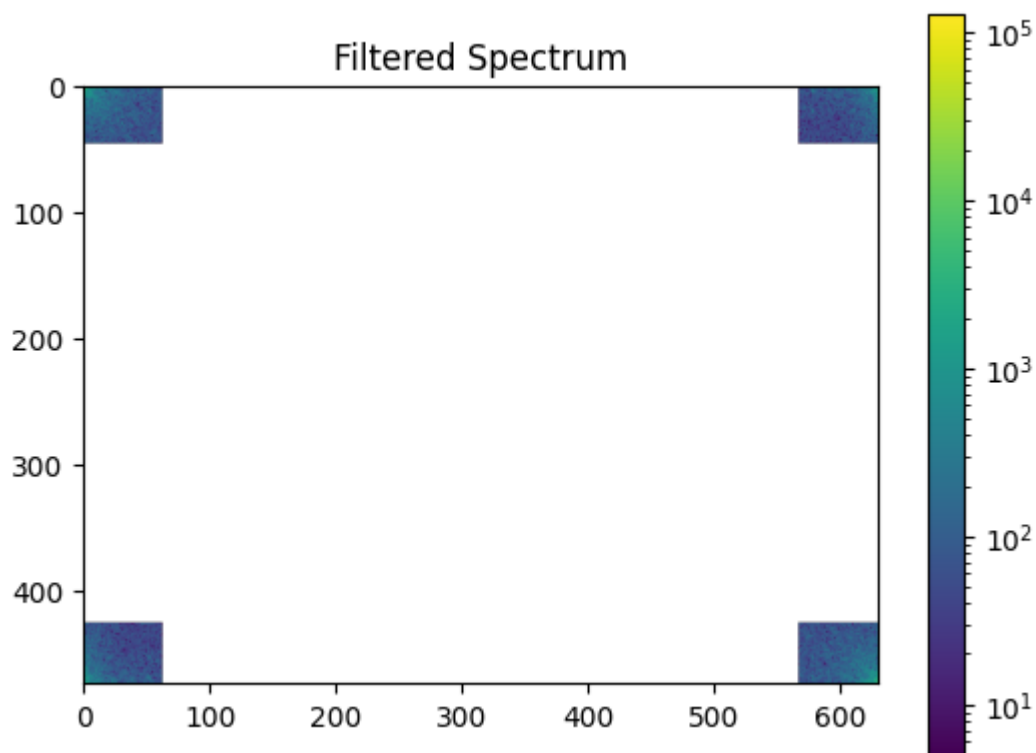
# Set r and c to be the number of rows and columns of the array.
r, c = im_fft2.shape

# Set to zero all rows with indices between r*keep_fraction and
# r*(1-keep_fraction):
im_fft2[int(r * keep_fraction) : int(r * (1 - keep_fraction))] = 0

# Similarly with the columns:
im_fft2[:, int(c * keep_fraction) : int(c * (1 - keep_fraction))] = 0

plt.figure()
plot_spectrum(im_fft2)
plt.title("Filtered Spectrum");

```



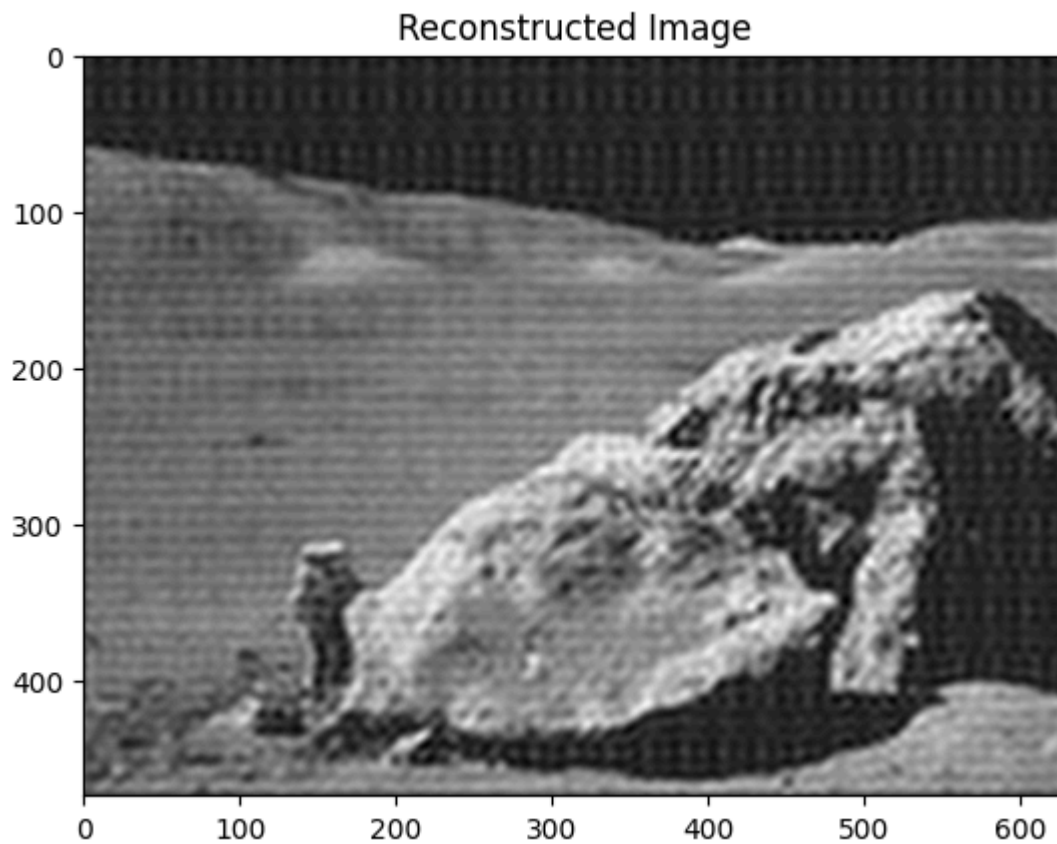
Reconstruct the final image

```

# Reconstruct the denoised image from the filtered spectrum, keep only
# the
# real part for display.
im_new = sp.fft.ifft2(im_fft2).real

plt.figure()
plt.imshow(im_new, "gray")
plt.title("Reconstructed Image");

```



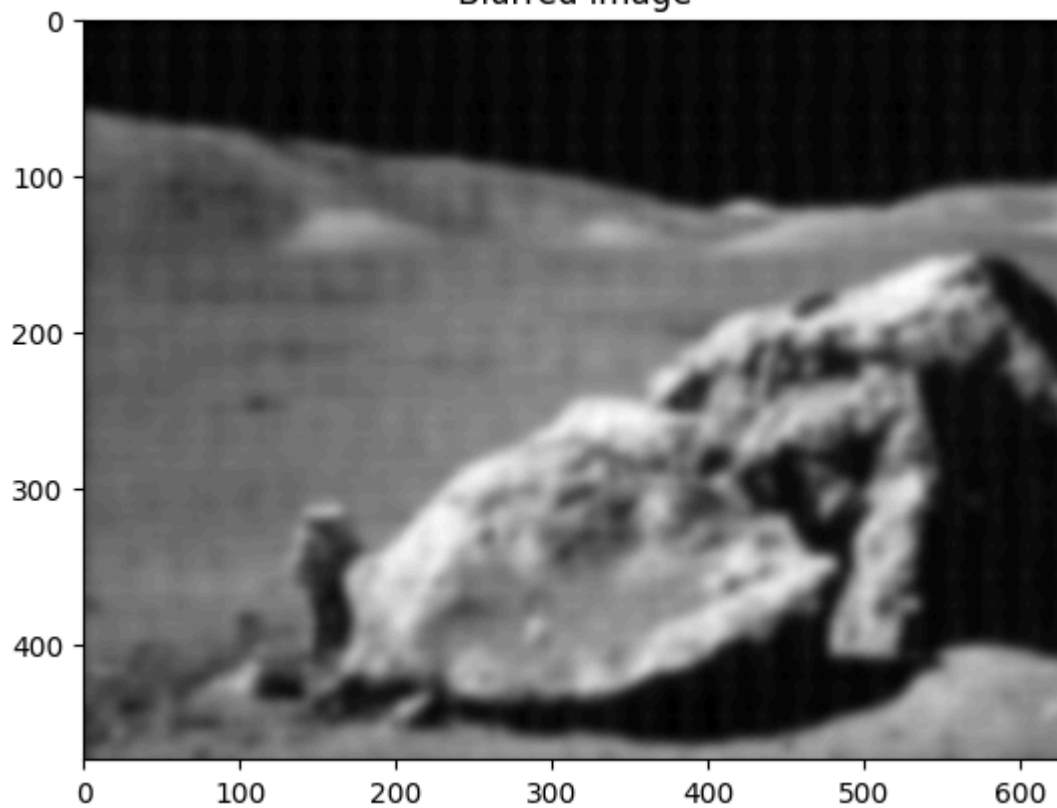
Easier and better: `scipy.ndimage.gaussian_filter()`

Implementing filtering directly with FFTs is tricky and time consuming. We can use the Gaussian filter from `scipy.ndimage`

```
im_blur = sp.ndimage.gaussian_filter(im, 4)

plt.figure()
plt.imshow(im_blur, "gray")
plt.title("Blurred image");
```

Blurred image



Signal processing: [scipy.signal](#)

Note

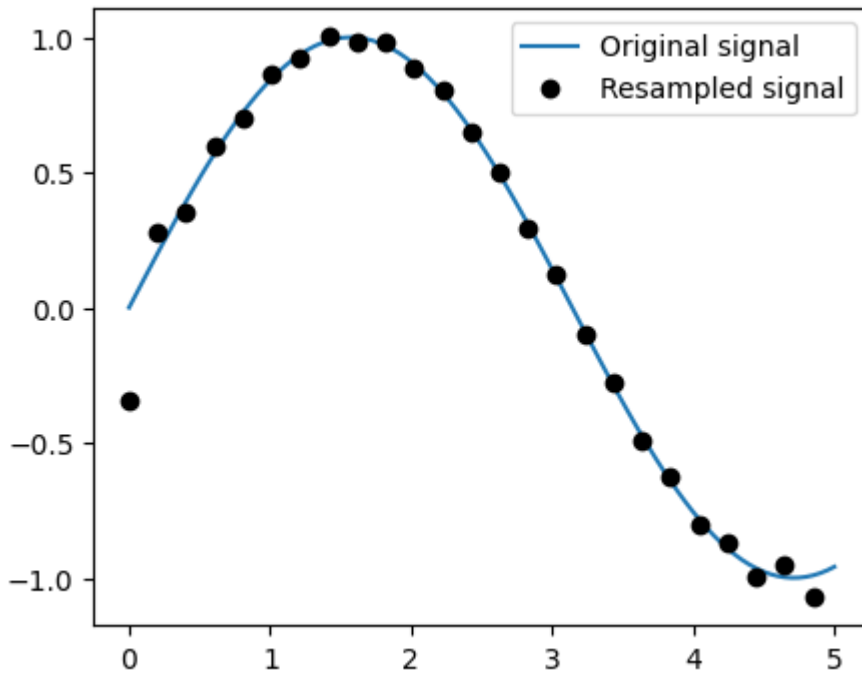
[scipy.signal](#) is for typical signal processing: 1D, regularly-sampled signals.

Resampling [scipy.signal.resample\(\)](#): resample a signal to `n` points using FFT.

```
t = np.linspace(0, 5, 100)
x = np.sin(t)

x_resampled = sp.signal.resample(x, 25)
```

► Show code cell source



Note

Notice how on the side of the window the resampling is less accurate and has a rippling effect.

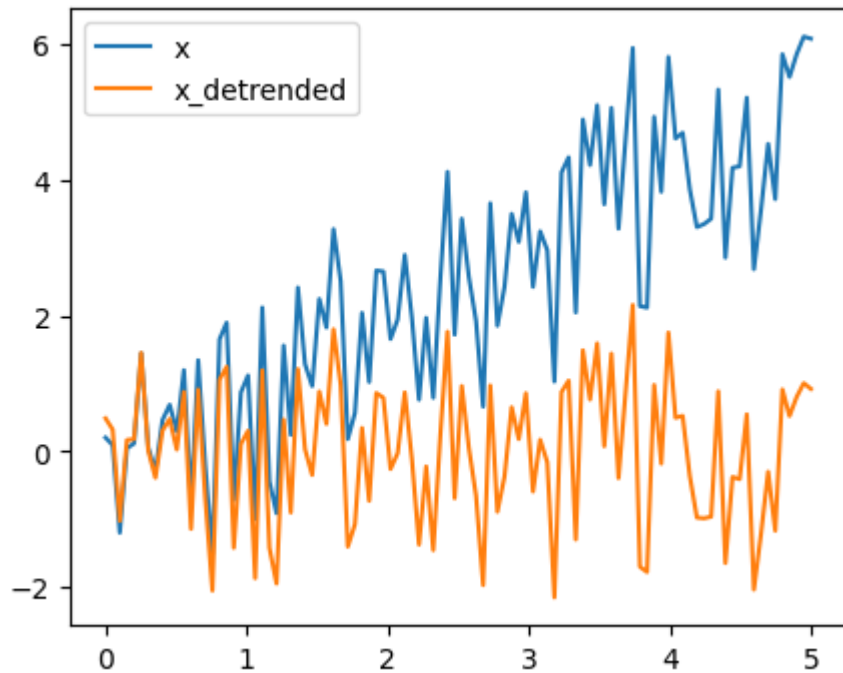
This resampling is different from the [interpolation](#) provided by `scipy.interpolate` as it only applies to regularly sampled data.

Detrending `scipy.signal.detrend()`: remove linear trend from signal:

```
t = np.linspace(0, 5, 100)
rng = np.random.default_rng()
x = t + rng.normal(size=100)

x_detrended = sp.signal.detrend(x)
```

► Show code cell source



Filtering:

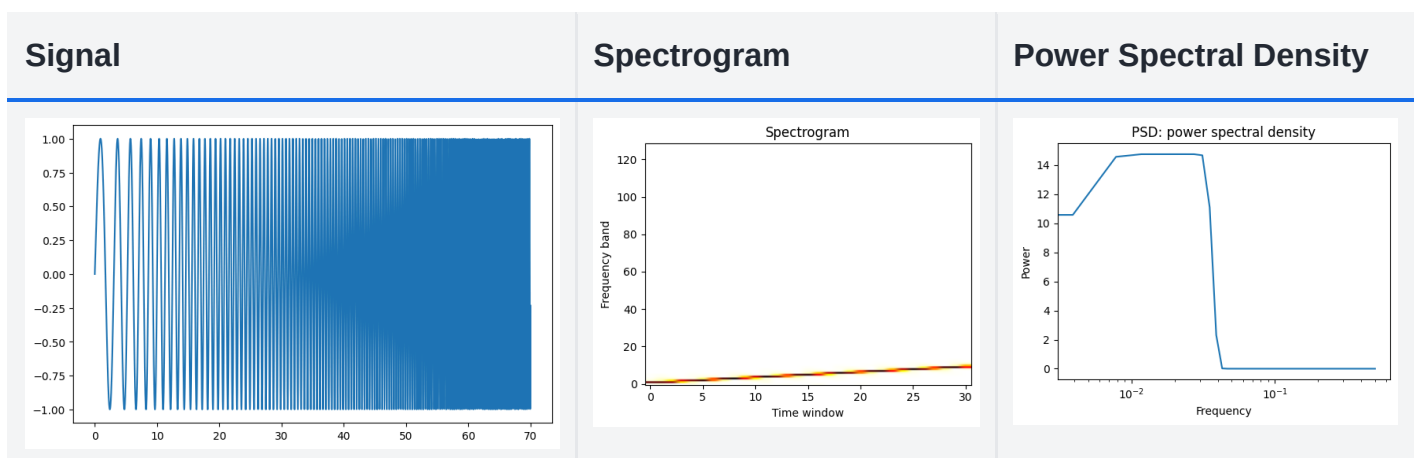
For non-linear filtering, `scipy.signal` has filtering (median filter `scipy.signal.medfilt()`, Wiener `scipy.signal.wiener()`), but we will discuss this in the image section.

Note

`scipy.signal` also has a full-blown set of tools for the design of linear filter (finite and infinite response filters), but this is out of the scope of this tutorial.

Spectral analysis:

`scipy.signal.spectrogram()` computes a spectrogram — frequency spectra over consecutive time windows — while `scipy.signal.welch()` computes a power spectral density (PSD).



See the [Spectrogram example](#).

Image manipulation: `scipy.ndimage`

See [Scipy image processing](#)

Summary exercises on scientific computing

The summary exercises use mainly NumPy, SciPy and Matplotlib. They provide some real-life examples of scientific computing with Python. Now that the basics of working with NumPy and SciPy have been introduced, the interested user is invited to try these exercises.

- [Statistical interpolation](#)
- [Non-linear fitting](#)
- [Image processing](#)



See also

References to go further

- Some chapters of the [advanced](#) and the [packages and applications](#) parts of the SciPy lectures.
- The [SciPy cookbook](#)

Other useful links

- [fipy](#)
- [odepack fortran library](#)
- [sfepy](#)

Getting help and finding documentation

Contents

- `help` in Jupyter and IPython
- Online documentation
- `psearch`
- If all else fails

Author: *Emmanuelle Gouillart*

Rather than knowing all functions in NumPy and SciPy, it is important to find information throughout the documentation and the available help. Here are some ways to get information:

`help` in Jupyter and IPython

In the Jupyter notebook, and in IPython terminals, one can use the `help` function to see the docstring of any particular function. For example:

```
import numpy as np
help(np.around)
```

Help on `_ArrayFunctionDispatcher` in module `numpy`:

```
around(a, decimals=0, out=None)
    Round an array to the given number of decimals.
```

```
`around` is an alias of `~numpy.round`.
```

See Also

```
ndarray.round : equivalent method
round : alias for this function
ceil, fix, floor, rint, trunc
```


Jupyter and IPython also recognize `?` at the end of the function name as a request to the function docstring, so executing:

```
np.around?
```

is equivalent to executing `help(around)`.

You only need type the beginning of the function's name and use tab completion to display the matching functions. For example, if you were interesting the `np.vander` function, you can type the Tab key after `np.van` to tab complete to the only function starting with `np.van` (`np.vander`).

```
# Uncomment, and press Tab at the end of `np.van` to show tab completion.  
# np.van
```

In the standard Ipython terminal, it is not possible to open a separate window for help and documentation; however one can always open a second `Ipython` shell just to display help and docstrings...

Online documentation

Numpy's and Scipy's documentations can be browsed online on <https://scipy.org> and <https://numpy.org>. The `search` button is quite useful inside the reference documentation of the two packages.

Tutorials on various topics as well as the complete API with all docstrings are found on this website.

The SciPy Cookbook <https://scipy-cookbook.readthedocs.io> gives recipes on many common problems frequently encountered, such as fitting data points, solving ODE, etc.

Matplotlib's website <https://matplotlib.org/> features a very nice **gallery** with a large number of plots, each of them shows both the source code and the resulting plot. This is very useful for learning by example. More standard documentation is also available.

psearch

Jupyter and IPython have a magic function `%psearch` to search for objects matching patterns. This is useful if, for example, one does not know the exact name of a function.

```
%psearch np.diag*
```

If all else fails

If everything listed above fails (and Google doesn't have the answer)... don't despair! There is a vibrant Scientific Python community. Scientific Python is present on various platform.

<https://scientific-python.org/community/>

Packages like SciPy and NumPy also have their own channels. Have a look at their respective websites to find out how to engage with users and maintainers.